

Thirty characters are not enou

CPC Reference Notebook

Table of Contents

1	Template and Stress	3	Centroid decomposition with updates.cpp	26
	Template.cpp	3	Centroid decomposition.cpp	27
	stress.bat	3	dijkstra.cpp	27
	brute.cpp	3	Dinic.cpp	27
	fast.cpp	3	DSU on trees.cpp	28
	generator.cpp	4	eulerWalk.cpp	29
2	Data structures	6	HLD.cpp	29
	2D sparse table.cpp	6	Hopcroftkarp.cpp	30
	bin_Trie.cpp	6	LCA and Binary lifting.cpp	30
	Cartesian Tree.cpp	7	MCMF.cpp	31
	CHT.cpp	8	Rerooting.cpp	32
	DSU rollbacks.cpp	9	SCC.cpp	33
	DSU.cpp	9	tree isomorphism.cpp	33
	Fenwick.cpp	10	Virtual Tree.cpp	34
	lichao segment with rollbacks.cpp	10	4 Math temps	36
	lichao with rollbacks.cpp	11	modnum helper	36
	merge sort tree.cpp	11	2D NTT.cpp	36
	Mo_s on tree.cpp	12	BerlekampMassey.cpp	37
	Mo_s with updates.cpp	13	comb.cpp	38
	Mo_s_algorithm.cpp	14	CRT.cpp	38
	Multiset lazy deletion.cpp	14	Discrete log.cpp	39
	Offline 2D BIT.cpp	14	Fast prime sieve.cpp	39
	ordered_set.cpp	15	FFT string_matching.cpp	40
	Persistent DSU.cpp	15	FFT with MOD.cpp	41
	PST.cpp	16	FFT.cpp	42
	seg_tree (lazy generic).cpp	17	FWHT.cpp	43
	seg_tree (lazy propagation).cpp	17	Gaussian with Doubles.cpp	44
	seg_tree (point, range query).cpp	18	Gaussian with MOD.cpp	44
	sparse_table.cpp	18	Kth root modulo.cpp	46
	topK.cpp	19	lagrange interpolation.cpp	47
	TwoStackQ.cpp	19	matrix_expo.cpp	48
3	graphs	21	NTT.cpp	48
	2SAT.cpp	21	NTT_CRT.cpp	50
	Articulation points.cpp	21	Number_theory.cpp	52
	Biconnected components.cpp	22	Pollard Rho Miller Rabin.cpp	54
	Bipartite check.cpp	23	Primitive root.cpp	55
	Blossom.cpp	23	rand.cpp	55
	Bridges.cpp	24	Segmented sieve.cpp	55
	Centroid decomposition buckets.cpp	25	SOS dp.cpp	56
			xor basis.cpp	57

5	strings	59	Z_algorithm.cpp	70
	Aho.cpp	59	6 DP Optimizations and Misc	71
	Bitset string matching.cpp	59	divide and conquer dp.cpp	71
	dynamic_aho.cpp	60	Custom hashing.cpp	71
	Generalized suffix automaton.cpp	61	Josephus.cpp	71
	KMP.cpp	63	Permutation cycles.cpp	72
	manacher.cpp	64	Lattice count under line	73
	palindromic tree.cpp	64	Minimum number of squares	73
	String_Hashing.cpp	66	Pell equation	74
	Suffixautomaton.cpp	67	Subsequences product at least K	75
	Trie_strings.cpp	69	Blue red hackenbush	76

1 Template and Stress

Template.cpp

Use: Starter file: put solution code inside SOLVE(), keep main(), and add snippets above SOLVE() as needed.

Tip: Use this as the wrapper for copied snippets; most snippets in the notebook have duplicate includes/main removed.

Before: Keep common includes, typedefs, and fast I/O here so snippets can stay short.

Pitfall: If a copied snippet needs globals, place them above SOLVE() and reset them for each test case.

```
#include <bits/stdc++.h>
#include <cassert>
#pragma GCC optimize("Ofast,unroll-loops")
#ifdef ONLINE_JUDGE
    #define debug(...) ((void)0)
#else
    #include "Template\debug.h"
#endif
using namespace std;
using ll = long long;
#define endl '\n'

void SOLVE(){

}

signed main(){
    ios_base::sync_with_stdio(false); cin.tie(nullptr);
    // freopen("txt.in", "r", stdin);
    int o_o; cin >> o_o; while(o_o--)
        SOLVE(); return 0;
}
```

stress.bat

Use: Put brute.cpp, fast.cpp, and generator.cpp beside it, then run stress.bat to repeatedly compare outputs.

Tip: When it reports a mismatch, rerun with the printed seed/input first, then shrink the generated constraints.

Before: Compile all three C++ files first, or keep the script compile commands in sync with your local flags.

Pitfall: A mismatch can come from nondeterministic output order, so sort outputs when any valid order is accepted.

```
@echo off
setlocal EnableDelayedExpansion

if not exist bin mkdir bin

g++ -std=c++17 -O2 generator.cpp -o bin\gen.exe || exit /b
g++ -std=c++17 -O2 brute.cpp -o bin\brute.exe || exit /b
g++ -std=c++17 -O2 fast.cpp -o bin\fast.exe || exit /b

set /a I=0
set LIM=%1
if "%LIM%"=="0" set LIM=0

:loop
set /a I+=1
echo Test !I!

bin\gen.exe !I! > input.txt
bin\brute.exe < input.txt > out_brute.txt
bin\fast.exe < input.txt > out_fast.txt
```

```
fc out_brute.txt out_fast.txt > nul || (
    echo MISMATCH ON SEED !I!
    echo ==== INPUT ==== & type input.txt
    echo ==== BRUTE ==== & type out_brute.txt
    echo ==== FAST ==== & type out_fast.txt
    pause
    exit /b
)

if not "%LIM%"=="0" if !I! GEQ %LIM% exit /b
goto loop
```

brute.cpp

Use: Write the slow trusted solution in solve(); stress.bat feeds it generated tests and compares against fast.cpp.

Care: Keep this solution obvious and trusted; slow exhaustive logic is fine because it is only an oracle for small tests.

In: Use the same input format as fast.cpp, even when constraints are intentionally tiny.

Pitfall: Do not optimize the brute until it becomes hard to trust; clarity matters more than speed here.

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

void solve() {
    int n;
    cin >> n;
    vector<i64> a(n);
    for (i64 &x : a) cin >> x;

    i64 best = LLONG_MIN;
    for (int l = 0; l < n; l++) {
        i64 sum = 0;
        for (int r = l; r < n; r++) {
            sum += a[r];
            best = max(best, sum);
        }
    }
    cout << best << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) solve();
}
```

fast.cpp

Use: Write the optimized solution in solve(); stress.bat checks its output against brute.cpp.

Care: Match brute.cpp output exactly, including spaces and newlines, so the diff points to algorithm bugs only.

In: Keep the generator limits visible, then paste the contest solution logic inside solve().

Pitfall: Undefined behavior may pass samples but usually appears quickly under repeated random stress.

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

void solve() {
    int n;
    cin >> n;
    vector<i64> a(n);
    for (i64 &x : a) cin >> x;

    i64 best = a[0], current = a[0];
    for (int i = 1; i < n; i++) {
        current = max(a[i], current + a[i]);
        best = max(best, current);
    }
    cout << best << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) solve();
}

```

generator.cpp

Use: Edit main()/helpers to print one random test; optional argv seed lets stress.bat reproduce failing tests.

Tip: Mix random cases with edge cases such as minimum sizes, duplicates, sorted data, and dense/sparse graphs.

Out: Print exactly one complete test file, including the test count if the solution expects one.

Pitfall: Use the argv seed while debugging so a failing random case can be reproduced exactly.

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

ll rnd(ll l, ll r) {
    return uniform_int_distribution<ll>(l, r)(rng);
}

int rnd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

bool coin(int p = 50) {
    return rnd(1, 100) <= p;
}

template <class T>
void print(vector<T> a) {
    for (int i = 0; i < (int)a.size(); i++) cout << a[i] << " \n"[i + 1 ==
        (int)a.size()];
}

```

```

vector<ll> arr(int n, ll l, ll r) {
    vector<ll> a(n);
    for (auto &x : a) x = rnd(l, r);
    return a;
}

vector<int> perm(int n) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

string str(int n, char l = 'a', char r = 'z') {
    string s;
    while (n--) s += char(rnd(l, r));
    return s;
}

vector<string> grid(int n, int m, string chars = ".#") {
    vector<string> g(n, string(m, '.'));
    for (auto &row : g) {
        for (char &c : row) c = chars[rnd(0, (int)chars.size() - 1)];
    }
    return g;
}

vector<pair<int, int>> tree(int n) {
    vector<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.push_back({rnd(1, i - 1), i});
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> path_tree(int n) {
    vector<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.push_back({i - 1, i});
    return e;
}

vector<pair<int, int>> star_tree(int n) {
    vector<pair<int, int>> e;
    int c = rnd(1, n);
    for (int i = 1; i <= n; i++) {
        if (i != c) e.push_back({c, i});
    }
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> graph(int n, int m) {
    vector<pair<int, int>> all, e;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) all.push_back({i, j});
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

```

```

}

vector<pair<int, int>> digraph(int n, int m) {
    vector<pair<int, int>> all, e;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i != j) all.push_back({i, j});
        }
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

vector<pair<int, int>> dag(int n, int m) {
    vector<int> p = perm(n);
    vector<pair<int, int>> all, e;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) all.push_back({p[i], p[j]});
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

vector<pair<int, int>> connected_graph(int n, int m) {
    vector<pair<int, int>> e = tree(n);
    set<pair<int, int>> used;
    for (auto [u, v] : e) {
        if (u > v) swap(u, v);
        used.insert({u, v});
    }
    m = min(m, n * (n - 1) / 2);
    while ((int)e.size() < m) {
        int u = rnd(1, n), v = rnd(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.insert({u, v}).second) e.push_back({u, v});
    }
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> queries(int q, int n) {
    vector<pair<int, int>> res;
    while (q--) {
        int l = rnd(1, n), r = rnd(1, n);
        if (l > r) swap(l, r);
        res.push_back({l, r});
    }
    return res;
}

void print_edges(vector<pair<int, int>> e) {
    for (auto [u, v] : e) cout << u << ' ' << v << '\n';
}

```

```

void print_weighted_edges(vector<pair<int, int>> e, int l, int r) {
    for (auto [u, v] : e) cout << u << ' ' << v << ' ' << rnd(l, r) << '\n';
}

void solve() {
    int n = rnd(1, 20);
    cout << n << '\n';
    print(arr(n, -20, 20));
}

int main(int argc, char **argv) {
    ios::sync_with_stdio(0);
    cin.tie(0);
    if (argc > 1) rng.seed(stoull(argv[1]));
    int t = 1; // TESTCASES
    if (argc > 2) t = stoi(argv[2]);
    cout << t << '\n';
    while (t--) solve();
}

```

2 Data structures

2D sparse table.cpp

Use: SparseTable2D<T> st(a, merge); query(r1,c1,r2,c2) returns the idempotent aggregate on an inclusive 0-indexed rectangle.

Care: The merge function must be idempotent, such as min, max, or gcd; do not use it for sum without changing the query logic.

Complexity: Build and memory are $O(n*m*\log n*\log m)$, query is $O(1)$, and the matrix must stay static.

Pitfall: Input rows must all have the same length; coordinates are inclusive and may be swapped if passed reversed.

```
template<class T>
struct SparseTable2D{
    int n, m, LOGN, LOGM;
    vector<int> lgN, lgM;
    vector<vector<vector<T>>> table;
    function<T(const T&, const T&)> f;
    SparseTable2D(){}
    SparseTable2D(const vector<vector<T>> &a, function<T(const T&, const T&)> func){
        init(a, func);
    }
    int rows(int kx) const{
        return n - (1 << kx) + 1;
    }
    int cols(int ky) const{
        return m - (1 << ky) + 1;
    }
    int id(int ky, int i, int j) const{
        return i * cols(ky) + j;
    }
    T& at(int kx, int ky, int i, int j){
        return table[kx][ky][id(ky, i, j)];
    }
    const T& at(int kx, int ky, int i, int j) const{
        return table[kx][ky][id(ky, i, j)];
    }
    void init(const vector<vector<T>> &a, function<T(const T&, const T&)> func){
        n = a.size();
        m = n ? a[0].size() : 0;
        assert(n > 0 && m > 0);
        for (int i = 0; i < n; i++){
            assert((int)a[i].size() == m);
        }
        f = func;
        lgN.assign(n + 1, 0);
        lgM.assign(m + 1, 0);
        for (int i = 2; i <= n; i++){
            lgN[i] = lgN[i >> 1] + 1;
        }
        for (int i = 2; i <= m; i++){
            lgM[i] = lgM[i >> 1] + 1;
        }
        LOGN = lgN[n] + 1;
        LOGM = lgM[m] + 1;
        table.assign(LOGN, vector<vector<T>>(LOGM));
        table[0][0].resize(n * m);
        for (int i = 0; i < n; i++){
            for (int j = 0; j < m; j++){
                at(0, 0, i, j) = a[i][j];
            }
        }
    }
};
```

```
    }
}
for (int kx = 0; kx < LOGN; kx++){
    for (int ky = 0; ky < LOGM; ky++){
        if (kx == 0 && ky == 0) continue;
        int R = rows(kx), C = cols(ky);
        table[kx][ky].resize(R * C);
        if (kx == 0){
            for (int i = 0; i < R; i++){
                for (int j = 0; j < C; j++){
                    at(kx, ky, i, j) = f(at(kx, ky - 1, i, j), at(kx, ky - 1, i,
                        ↪ j + (1 << (ky - 1))));
                }
            }
        }
        else{
            for (int i = 0; i < R; i++){
                for (int j = 0; j < C; j++){
                    at(kx, ky, i, j) = f(at(kx - 1, ky, i, j), at(kx - 1, ky, i +
                        ↪ (1 << (kx - 1)), j));
                }
            }
        }
    }
}
}
T query(int r1, int c1, int r2, int c2) const{
    if (r1 > r2) swap(r1, r2);
    if (c1 > c2) swap(c1, c2);
    assert(0 <= r1 && r2 < n && 0 <= c1 && c2 < m);
    int kx = lgN[r2 - r1 + 1];
    int ky = lgM[c2 - c1 + 1];
    int nr = r2 - (1 << kx) + 1;
    int nc = c2 - (1 << ky) + 1;
    T a = f(at(kx, ky, r1, c1), at(kx, ky, nr, c1));
    T b = f(at(kx, ky, r1, nc), at(kx, ky, nr, nc));
    return f(a, b);
}
};
```

bin_Trie.cpp

Use: Trie t(bits); call insert(x), del(x), cnt(x), query(x,k), mnXor(x), or mxXor(x) for nonnegative integers with that bit width.

Care: Values are stored with multiplicity; delete only inserted values and choose bits high enough for the maximum number.

Complexity: insert, del, cnt, and xor queries are $O(\text{bits})$, independent of how many numbers are stored.

Pitfall: query(x,k) counts stored y with $(x \text{ xor } y) < k$, so confirm strict inequality before using it.

```
struct Trie{
    struct Node{
        int child[2], freq;
        Node(){
            child[0] = child[1] = -1;
            freq = 0;
        }
    };
    vector<Node> trie;
    int bits;
    Trie(int x){
```

```

        bits = x;
        trie.emplace_back();
    }
    int newNode(){
        trie.emplace_back();
        return (int)trie.size() - 1;
    }
    void insert(ll val){
        int cur = 0;
        trie[cur].freq++;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            if (trie[cur].child[bit] == -1){
                trie[cur].child[bit] = newNode();
            }
            cur = trie[cur].child[bit];
            trie[cur].freq++;
        }
    }
    void del(ll val){
        if (!cnt(val)) return;
        int cur = 0;
        trie[cur].freq--;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            cur = trie[cur].child[bit];
            trie[cur].freq--;
        }
    }
    int cnt(ll val){
        int cur = 0;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            cur = trie[cur].child[bit];
            if (cur == -1) return 0;
        }
        return trie[cur].freq;
    }
    ll query(ll val, ll k){ // num ^ val < k
        int cur = 0;
        ll answer = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool b1 = (val >> i) & 1;
            bool b2 = (k >> i) & 1;
            if (b2){
                int same = trie[cur].child[b1];
                if (same != -1) answer += trie[same].freq;
                cur = trie[cur].child[b1 ^ 1];
            }else{
                cur = trie[cur].child[b1];
            }
        }
        return answer;
    }
    ll mxXor(ll val){
        int cur = 0;
        ll ret = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool bit = (val >> i) & 1;

```

```

            int nxt = trie[cur].child[bit ^ 1];
            if (nxt != -1 && trie[nxt].freq){
                ret |= (1LL << i);
                cur = nxt;
            }else{
                cur = trie[cur].child[bit];
            }
        }
        return ret;
    }
    ll mnXor(ll val){
        int cur = 0;
        ll ret = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool bit = (val >> i) & 1;
            int nxt = trie[cur].child[bit];
            if (nxt != -1 && trie[nxt].freq){
                cur = nxt;
            }else{
                ret |= (1LL << i);
                cur = trie[cur].child[bit ^ 1];
            }
        }
        return ret;
    }
};

```

Cartesian Tree.cpp

Use: Build MinCartesianTree<T> or MaxCartesianTree<T> ct(a); query_index/query_value answer RMQ and get_tree(directed) returns adjacency with root.

Out: Queries are 0-indexed inclusive; L[v], R[v] give the subtree interval, and tie behavior follows the better() comparator.

Complexity: Construction is O(n), build_info adds binary lifting, and range queries are O(log n).

When: Use it when RMQ index/value queries are static and you also want the Cartesian tree structure.

```

template<class T, bool is_min = true>
struct CartesianTree{
    int n, root, LOG;
    vector<T> a;
    vector<int> parent, left, right, L, R, depth;
    vector<vector<int>> up;
    CartesianTree(){}
    CartesianTree(const vector<T> &v){ init(v); }
    bool better(int i, int j) const{
        if (a[i] != a[j]) return is_min ? a[i] < a[j] : a[i] > a[j];
        return is_min ? i < j : i > j; // pair {a[i], i}
    }
    void init(const vector<T> &v){
        a = v;
        n = a.size();
        root = -1;
        parent.assign(n, -1);
        left.assign(n, -1); right.assign(n, -1);
        L.resize(n); R.resize(n);
        depth.assign(n, 0);
        vector<int> st;
        for (int i = 0; i < n; i++){
            int last = -1;

```

```

while (!st.empty() && better(i, st.back())){
    last = st.back();
    st.pop_back();
}
if (!st.empty()){
    right[st.back()] = i;
    parent[i] = st.back();
}
if (last != -1){
    left[i] = last;
    parent[last] = i;
}
st.push_back(i);
}
for (int i = 0; i < n; i++){
    if (parent[i] == -1) root = i;
}
build_info();
}
void build_info(){
    if (!n) return;
    LOG = __lg(n) + 1;
    up.assign(LOG, vector<int>(n, root));
    vector<int> order = {root};
    up[0][root] = root;
    for (int ptr = 0; ptr < (int)order.size(); ptr++){
        int v = order[ptr];
        for (int i = 1; i < LOG; i++) up[i][v] = up[i - 1][up[i - 1][v]];
        for (int u : {left[v], right[v]}){
            if (u == -1) continue;
            depth[u] = depth[v] + 1;
            up[0][u] = v;
            order.push_back(u);
        }
    }
    reverse(order.begin(), order.end());
    for (auto v : order){
        L[v] = R[v] = v;
        if (left[v] != -1) L[v] = L[left[v]];
        if (right[v] != -1) R[v] = R[right[v]];
    }
}
int lca(int u, int v) const{
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int i = 0; i < LOG; i++){
        if ((diff >> i) & 1) u = up[i][u];
    }
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--){
        if (up[i][u] != up[i][v]){
            u = up[i][u];
            v = up[i][v];
        }
    }
    return up[0][u];
}
int query_index(int l, int r) const{
    if (l > r) swap(l, r);

```

```

return lca(l, r);
}
T query_value(int l, int r) const{
    return a[query_index(l, r)];
}
pair<vector<vector<int>>, int> get_tree(bool directed = true) const {
    vector<vector<int>> adj(n);
    for (int i = 0; i < n; i++) {
        if (left[i] != -1) {
            adj[i].push_back(left[i]);
            if (!directed) adj[left[i]].push_back(i);
        }
        if (right[i] != -1) {
            adj[i].push_back(right[i]);
            if (!directed) adj[right[i]].push_back(i);
        }
    }
    return {adj, root};
}
};
template<class T>
using MinCartesianTree = CartesianTree<T, true>;
template<class T>
using MaxCartesianTree = CartesianTree<T, false>;

```

CHT.cpp

Use: CHT<true> for min or CHT<false> for max; add(k,m) inserts $y=kx+m$ and query(x) returns the best value.

Care: This is a dynamic integer LineContainer; queries may be arbitrary, but slopes/intersections use floored $\parallel$ division.

Complexity: Each add/query is $O(\log n)$; it handles arbitrary insertion order and arbitrary x queries.

Pitfall: For minimum queries the template negates lines internally, so do not negate answers yourself.

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

// CHT<true> for min, CHT<false> for max. add(k, m) adds  $y = kx + m$ 
template <bool is_min = false> struct CHT : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        if constexpr (is_min) {
            k = -k;

```



```

    m = -m;
}
auto z = insert({k, m, 0}), y = z++, x = y;
while (isect(y, z))
    z = erase(z);
if (x != begin() && isect(--x, y))
    isect(x, y = erase(y));
while ((y = x) != begin() && (--x)->p >= y->p)
    isect(x, erase(y));
}
ll query(ll x) {
    if (empty())
        return is_min ? LLONG_MAX : LLONG_MIN;
    auto l = *lower_bound(x);
    ll res = l.k * x + l.m;
    if constexpr (is_min) {
        return -res;
    }
    return res;
}
};

```

DSU rollbacks.cpp

Use: DSU d(n); save int t = d.time(), call unite(a,b), then rollback(t) to undo to that state.

Care: Rollback DSU intentionally has no path compression, so find() stays undoable but costs logarithmic-ish recursion.

Complexity: unite and rollback are undo-friendly without path compression; rollback is proportional to undone changes.

When: Use it for offline dynamic connectivity, divide-and-conquer on time, or backtracking over graph states.

```

struct DSU{
    int n, components;
    vector<int> parent, sz;
    vector<pair<int&, int>> history;
    DSU(){
        DSU(int n){
            init(n);
        }
        void init(int _n){
            n = _n;
            components = n;
            parent.resize(n + 1);
            sz.assign(n + 1, 1);
            history.clear();
            for (int i = 1; i <= n; i++){
                parent[i] = i;
            }
        }
        int find(int v){
            return parent[v] == v ? v : find(parent[v]);
        }
        bool same(int u, int v){
            return find(u) == find(v);
        }
        bool unite(int a, int b){
            a = find(a);
            b = find(b);
            if (a == b) return false;

```

```

            if (sz[a] < sz[b]) swap(a, b);
            history.push_back({sz[a], sz[a]});
            history.push_back({parent[b], parent[b]});
            history.push_back({components, components});
            parent[b] = a;
            sz[a] += sz[b];
            components--;
            return true;
        }
        bool union_sets(int a, int b){
            return unite(a, b);
        }
        int time(){
            return (int)history.size();
        }
        int get_size(int v){
            return sz[find(v)];
        }
        void rollback(int until){
            while ((int)history.size() > until){
                history.back().first = history.back().second;
                history.pop_back();
            }
        }
    };

```

DSU.cpp

Use: DSU d(n); use unite(a,b), same(a,b), get_size(v), and the components variable on 1-indexed nodes.

Care: Nodes are 1-indexed and components is a public variable, not a function call.

Complexity: find/unite are effectively $O(\alpha n)$ with path compression and union by size.

Pitfall: The structure is 1-indexed, so allocate with n and avoid node 0 unless you adapt init.

```

struct DSU{
    int n;
    vector<int> parent, size;
    int components;
    DSU(int n) : n(n), parent(n + 1), size(n + 1, 1), components(n) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int v){
        return parent[v] = (parent[v] == v ? v : find(parent[v]));
    }
    bool same(int u, int v) {
        return find(u) == find(v);
    }
    bool union_sets(int a, int b){
        a = find(a);
        b = find(b);
        if(a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        components--;
        return true;
    }
    int get_size(int v){
        return size[find(v)];
    }
}

```

```
};
```

Fenwick.cpp

Use: Fenwick<T> fw(n, ID, op); add(i, val), query(i), query(l, r), and lower_bound(sum) use 1-indexed positions.

Care: Range query uses subtraction, so op should behave like sum/xor with an inverse if you use query(l, r).

Complexity: add, prefix query, range query, and lower_bound are $O(\log n)$.

Pitfall: lower_bound assumes prefix values are monotone increasing, usually nonnegative point updates.

```
template <class T>
struct Fenwick { // 1-indexed
    int n, LOG; T ID;
    function<T(T, T)> op;
    vector<T> tree;
    Fenwick(int _n, T _ID, function<T(T, T)> _op) : n(_n + 5), ID(_ID), op(_op),
        LOG(log2(n) + 1) {
        tree.assign(n + 1, ID);
    }
    void add(int i, T val) {
        if (i <= 0) return;
        for (; i <= n; i += (i & -i)) tree[i] = op(tree[i], val);
    }
    T query(int i) {
        T res = ID;
        for (; i >= 1; i -= (i & -i)) res = op(res, tree[i]);
        return res;
    }
    T query(int l, int r){
        return query(r) - query(l - 1);
        // return query(r) ^ query(l - 1);
    }
    int lower_bound(ll val){
        T sum = 0;
        int pos = 0;
        for(int i = LOG; i >= 0; i--){
            if(pos + (1 << i) <= n && sum + tree[pos + (1 << i)] < val){
                pos += (1 << i);
                sum += tree[pos];
            }
        }
        return pos + 1;
    }
};
```

lichao segment with rollbacks.cpp

Use: LiChaoSegment<is_min> lc(L, R); add_line(a, b), add_segment(a, b, l, r), query(x), time(), and rollback(t).

Care: The domain is the integer interval [L, R]; use add_segment only when a line is valid on a subrange.

Complexity: Line insertion and query are $O(\log \text{coordinate_range})$; segment insertion adds another log factor.

When: Use the segment version when each line is valid only on [ql, qr], not on the whole domain.

```
template<bool is_min = true>
struct LiChaoSegment{
    const ll INF = 4e18;
    struct Line{
        ll a, b;
        Line(ll _a = 0, ll _b = 4e18) : a(_a), b(_b){}
    };
};
```

```
vector<Line> tree;
vector<pair<int, Line>> history;
ll L, R;
LiChaoSegment(){}
LiChaoSegment(ll _L, ll _R){
    init(_L, _R);
}
void init(ll _L, ll _R){
    L = _L;
    R = _R;
    tree.assign(4, Line());
    history.clear();
}
ll eval(Line line, ll x){
    __int128 ret = (__int128)line.a * x + line.b;
    if (ret > INF) return INF;
    if (ret < -INF) return -INF;
    return (ll)ret;
}
bool better(Line x, Line y, ll at){
    return eval(x, at) < eval(y, at);
}
void save(int node){
    if (node >= (int)tree.size()) tree.resize(node + 1, Line());
    history.push_back({node, tree[node]});
}
void add_line(ll a, ll b){
    if constexpr (!is_min) a = -a, b = -b;
    add_line(Line(a, b), 1, L, R);
}
void add_line(Line cur, int node, ll l, ll r){
    if (node >= (int)tree.size()) tree.resize(node + 1, Line());
    ll mid = l + (r - l) / 2;
    bool lef = better(cur, tree[node], l);
    bool m = better(cur, tree[node], mid);
    if (m){
        save(node);
        swap(cur, tree[node]);
    }
    if (l == r) return;
    if (lef != m) add_line(cur, node * 2, l, mid);
    else add_line(cur, node * 2 + 1, mid + 1, r);
}
void add_segment(ll a, ll b, ll ql, ll qr){
    if constexpr (!is_min) a = -a, b = -b;
    add_segment(Line(a, b), ql, qr, 1, L, R);
}
void add_segment(Line line, ll ql, ll qr, int node, ll l, ll r){
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr){
        add_line(line, node, l, r);
        return;
    }
    if (node >= (int)tree.size()) tree.resize(node + 1, Line());
    ll mid = l + (r - l) / 2;
    add_segment(line, ql, qr, node * 2, l, mid);
    add_segment(line, ql, qr, node * 2 + 1, mid + 1, r);
}
ll query(ll x){
```

```

    ll ret = query(x, 1, L, R);
    if constexpr (!is_min) ret = -ret;
    return ret;
}
ll query(ll x, int node, ll l, ll r){
    if (node >= (int)tree.size()) return INF;
    ll ret = eval(tree[node], x);
    if (l == r) return ret;
    ll mid = l + (r - l) / 2;
    if (x <= mid) ret = min(ret, query(x, node * 2, l, mid));
    else ret = min(ret, query(x, node * 2 + 1, mid + 1, r));
    return ret;
}
int time(){
    return (int)history.size();
}
void rollback(int t){
    while ((int)history.size() > t){
        auto [node, line] = history.back();
        history.pop_back();
        tree[node] = line;
    }
}
};

```

lichao with rollbacks.cpp

Use: LiChao<is_min> lc(L,R); add_line(a,b) inserts $y=ax+b$, query(x) asks best value, and rollback(snapshot()) undoes.

Care: Use snapshot() before tentative insertions; rollback(t) restores line nodes but not any variables outside the structure.

Complexity: add_line and query are $O(\log \text{coordinate_range})$; rollback cost is proportional to saved node changes.

Pitfall: Pick L and R to cover every x query exactly; outside-domain queries are not protected.

```

template<bool is_min = true>
struct LiChao{
    const ll INF = 4e18;
    struct Line{
        ll a, b;
        Line(ll _a = 0, ll _b = 4e18) : a(_a), b(_b){}
    };
    vector<Line> tree;
    vector<pair<int, Line>> history;
    ll L, R;
    LiChao(){
        LiChao(ll _L, ll _R){
            init(_L, _R);
        }
    void init(ll _L, ll _R){
        L = _L;
        R = _R;
        tree.assign(4, Line());
        history.clear();
    }
    ll eval(Line line, ll x){
        __int128 ret = (__int128)line.a * x + line.b;
        if (ret > INF) return INF;
        if (ret < -INF) return -INF;
    }
};

```

```

        return (ll)ret;
    }
    bool better(Line x, Line y, ll at){
        return eval(x, at) < eval(y, at);
    }
    void save(int node){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        history.push_back({node, tree[node]});
    }
    void add_line(ll a, ll b){
        if constexpr (!is_min) a = -a, b = -b;
        add_line(Line(a, b), 1, L, R);
    }
    void add_line(Line cur, int node, ll l, ll r){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        ll mid = l + (r - l) / 2;
        bool lef = better(cur, tree[node], l);
        bool m = better(cur, tree[node], mid);
        if (m){
            save(node);
            swap(cur, tree[node]);
        }
        if (l == r) return;
        if (lef != m) add_line(cur, node * 2, l, mid);
        else add_line(cur, node * 2 + 1, mid + 1, r);
    }
    ll query(ll x){
        ll ret = query(x, 1, L, R);
        if constexpr (!is_min) ret = -ret;
        return ret;
    }
    ll query(ll x, int node, ll l, ll r){
        if (node >= (int)tree.size()) return INF;
        ll ret = eval(tree[node], x);
        if (l == r) return ret;
        ll mid = l + (r - l) / 2;
        if (x <= mid) ret = min(ret, query(x, node * 2, l, mid));
        else ret = min(ret, query(x, node * 2 + 1, mid + 1, r));
        return ret;
    }
    int snapshot(){
        return (int)history.size();
    }
    void rollback(int t){
        while ((int)history.size() > t){
            auto [node, line] = history.back();
            history.pop_back();
            tree[node] = line;
        }
    }
};

```

merge sort tree.cpp

Use: merge_sort_tree mt(a); count_less/count_leq/count_greater(l,r,x) and sum_less/sum_leq use 0-indexed inclusive ranges.

Out: Counts and sums are for values compared to x on $[l,r]$; memory is $O(n \log n)$, so reserve it for static arrays.

Complexity: Build is $O(n \log n)$; each count/sum query is $O(\log^2 n)$.

Pitfall: The array is static; point updates require rebuilding or a different structure.

```

struct merge_sort_tree{
    int n;
    vector<vector<ll>> tree, pref;
    merge_sort_tree(){}
    merge_sort_tree(vector<ll> &a){
        init(a);
    }
    void init(vector<ll> &a){
        n = 1;
        while (n < (int)a.size()) n <= 1;
        tree.assign(n <= 1, {});
        pref.assign(n <= 1, {});
        build(0, 0, n, a);
    }
    void pull(int node){
        auto &l = tree[node * 2 + 1];
        auto &r = tree[node * 2 + 2];
        tree[node].resize(l.size() + r.size());
        merge(l.begin(), l.end(), r.begin(), r.end(), tree[node].begin());
        pref[node].resize(tree[node].size() + 1);
        for (int i = 0; i < (int)tree[node].size(); i++){
            pref[node][i + 1] = pref[node][i] + tree[node][i];
        }
    }
    void build(int node, int l, int r, vector<ll> &a){
        if (r - l == 1){
            if (l < (int)a.size()){
                tree[node] = {a[l]};
                pref[node] = {0, a[l]};
            }
            return;
        }
        int mid = (l + r) >> 1;
        build(node * 2 + 1, l, mid, a);
        build(node * 2 + 2, mid, r, a);
        pull(node);
    }
    int cnt_less(int node, ll x){
        return lower_bound(tree[node].begin(), tree[node].end(), x) - tree[node].begin();
    }
    int cnt_leq(int node, ll x){
        return upper_bound(tree[node].begin(), tree[node].end(), x) - tree[node].begin();
    }
    ll sum_first(int node, int cnt){
        return pref[node][cnt];
    }
    int count_less(int l, int r, ll x){
        return count_less(l, r + 1, x, 0, 0, n);
    }
    int count_less(int ql, int qr, ll x, int node, int l, int r){
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return cnt_less(node, x);
        int mid = (l + r) >> 1;
        return count_less(ql, qr, x, node * 2 + 1, l, mid) + count_less(ql, qr, x, node *
        ↪ 2 + 2, mid, r);
    }
    int count_leq(int l, int r, ll x){
        return count_leq(l, r + 1, x, 0, 0, n);
    }

```

```

}
int count_leq(int ql, int qr, ll x, int node, int l, int r){
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return cnt_leq(node, x);
    int mid = (l + r) >> 1;
    return count_leq(ql, qr, x, node * 2 + 1, l, mid) + count_leq(ql, qr, x, node * 2
    ↪ + 2, mid, r);
}
int count_greater(int l, int r, ll x){
    return r - l + 1 - count_leq(l, r, x);
}
ll sum_less(int l, int r, ll x){
    return sum_less(l, r + 1, x, 0, 0, n);
}
ll sum_less(int ql, int qr, ll x, int node, int l, int r){
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return sum_first(node, cnt_less(node, x));
    int mid = (l + r) >> 1;
    return sum_less(ql, qr, x, node * 2 + 1, l, mid) + sum_less(ql, qr, x, node * 2 +
    ↪ 2, mid, r);
}
ll sum_leq(int l, int r, ll x){
    return sum_leq(l, r + 1, x, 0, 0, n);
}
ll sum_leq(int ql, int qr, ll x, int node, int l, int r){
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return sum_first(node, cnt_leq(node, x));
    int mid = (l + r) >> 1;
    return sum_leq(ql, qr, x, node * 2 + 1, l, mid) + sum_leq(ql, qr, x, node * 2 +
    ↪ 2, mid, r);
}
};

```

Mo's on tree.cpp

Use: Fill add(v), del(v), get_answer(); add edges, build(root), add_query(u,v,id), then process() fills answer[id].
Care: Call clear(n) between tests; vertex toggling means add/del should be symmetric and based on node contribution.

Complexity: Processing is roughly $O((n+q) \sqrt{n})$ after Euler tour and LCA preprocessing.

Pitfall: Path queries need special LCA handling; verify whether the LCA node is toggled separately.

```

const int N = 2e5 + 5;
int n, q, timer, LOG, block;
vector<int> graph[N];
int tin[N], tout[N], euler[2 * N], depth[N], up[20][N];
bool active[N];
ll curAns, answer[N];

struct Query{
    int l, r, lca, id;
    bool operator<(const Query &other) const{
        if (l / block != other.l / block) return l < other.l;
        return (l / block & 1 ? r < other.r : other.r < r);
    }
};
vector<Query> queries;

void add(int v){

```

```

}
void del(int v){
}
ll get_answer(){
    return curAns;
}
void toggle(int idx){
    int v = euler[idx];
    if (active[v]){
        del(v);
    }else{
        add(v);
    }
    active[v] ^= 1;
}
void dfs(int v, int p){
    tin[v] = ++timer;
    euler[timer] = v;
    up[0][v] = p;
    for (int i = 1; i < LOG; i++) up[i][v] = up[i - 1][up[i - 1][v]];
    for (auto u : graph[v]){
        if (u == p) continue;
        depth[u] = depth[v] + 1;
        dfs(u, v);
    }
    tout[v] = ++timer;
    euler[timer] = v;
}
bool is_ancestor(int u, int v){
    return tin[u] <= tin[v] && tout[v] <= tout[u];
}
int lca(int u, int v){
    if (is_ancestor(u, v)) return u;
    if (is_ancestor(v, u)) return v;
    for (int i = LOG - 1; i >= 0; i--){
        if (!is_ancestor(up[i][u], v)) u = up[i][u];
    }
    return up[0][u];
}
void add_query(int u, int v, int id){
    if (tin[u] > tin[v]) swap(u, v);
    int w = lca(u, v);
    if (w == u){
        queries.push_back({tin[u], tin[v], 0, id});
    }else{
        queries.push_back({tout[u], tin[v], w, id});
    }
}
void build(int root = 1){
    timer = 0;
    LOG = __lg(max(1, n)) + 1;
    depth[root] = 0;
    dfs(root, root);
    block = max(1, (int)sqrt(timer));
}
void process(){
    sort(queries.begin(), queries.end());
    int l = 1, r = 0;

```

```

        for (auto query : queries){
            while (l > query.l) toggle(--l);
            while (r < query.r) toggle(++r);
            while (l < query.l) toggle(l++);
            while (r > query.r) toggle(r--);
            if (query.lca) toggle(tin[query.lca]);
            answer[query.id] = get_answer();
            if (query.lca) toggle(tin[query.lca]);
        }
    }
void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        active[i] = false;
    }
    queries.clear();
    curAns = 0;
    timer = 0;
}

```

Mo's with updates.cpp

Use: Fill add(i), del(i), and maintain curAns; store queries/updates, call process(), then read answer[id].

Care: The kept SOLVE() shows the required input handling for updates; do not remove its current-array bookkeeping.

Complexity: Typical cost depends on block sizes, often around $O((n+q)^{2/3})$ pointer/update movement.

Pitfall: apply/undo must change both the array and maintained answer consistently for active indices.

```

const int N = 2e5 + 5;
int n, q, block;
int a[N];
ll curAns, answer[N];

struct Query{
    int l, r, t, id;
    bool operator<(const Query &other) const{
        if (l / block != other.l / block) return l < other.l;
        if (r / block != other.r / block) return r < other.r;
        return t < other.t;
    }
};
struct Update{
    int i, old, val;
};

vector<Query> queries;
vector<Update> updates;
void add(int i){
}
void del(int i){
}
void apply(int t, int l, int r){
    auto &u = updates[t];
    if (l <= u.i && u.i <= r) del(u.i);
    a[u.i] = u.val;
    if (l <= u.i && u.i <= r) add(u.i);
}

```

```

void undo(int t, int l, int r){
    auto &u = updates[t];
    if (l <= u.i && u.i <= r) del(u.i);
    a[u.i] = u.old;
    if (l <= u.i && u.i <= r) add(u.i);
}
void process(){
    block = max(1, (int)pow(n, 2.0 / 3.0));
    sort(queries.begin(), queries.end());
    int l = 1, r = 0, t = 0;
    for (auto query : queries){
        while(t < query.t) apply(t++, l, r);
        while(t > query.t) undo(--t, l, r);
        while(r < query.r) add(++r);
        while(l > query.l) add(--l);
        while(r > query.r) del(r--);
        while(l < query.l) del(l++);
        answer[query.id] = curAns;
    }
}
void clear(){
    queries.clear();
    updates.clear();
    curAns = 0;
}
void SOLVE(){
    cin >> n >> q;
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> cur(a, a + n + 1);
    int id = 0;
    for (int i = 0; i < q; i++){
        char type;
        cin >> type;
        if (type == 'Q'){
            int l, r;
            cin >> l >> r;
            queries.push_back({l, r, (int)updates.size(), id++});
        }else{
            int idx, val;
            cin >> idx >> val;
            updates.push_back({idx, cur[idx], val});
            cur[idx] = val;
        }
    }
    process();
    for (int i = 0; i < id; i++){
        cout << answer[i] << endl;
    }
    clear();
}
}

```

Mo_s_algorithm.cpp

Use: Define add/remove logic around sorted query{*l,r,id*}; move current [*L,R*] while processing and write ans[*id*].

Care: The file is only the ordering skeleton; keep ans[*id*] outside and update the current range while iterating queries.

Complexity: After sorting, pointer moves dominate; total cost is about $O((n+q) \sqrt{n})$ for $O(1)$ add/remove.

Before: Store queries as inclusive [*l,r*], choose block size, and initialize current *L/R* before processing.

```

const int N = 1e5 + 5, sq = 317;
struct query{
    int l, r, i;
    bool operator<(const query &other) const {
        if (l / sq != other.l / sq)
            return l / sq < other.l / sq;
        return (l / sq & 1 ? r < other.r : other.r < r);
    }
};

```

Multiset lazy deletion.cpp

Use: MS<*T*> ms; insert(*x*), erase(*x*), top(), pop(), size(), empty(); normalize() lazily removes deleted elements.

Care: top()/pop() assume the structure is nonempty after normalize(); compare parameter makes min-heap or max-heap behavior.

Complexity: insert, erase, top, and pop are $O(\log n)$ amortized because stale deleted values are removed lazily.

Pitfall: If you erase a value never inserted, a future equal value may be accidentally removed.

```

template<typename T, typename Compare = less<T>>
struct MS {
    priority_queue<T, vector<T>, Compare> pq, del;
    void normalize(){
        while(!pq.empty() && !del.empty() && pq.top() == del.top()){
            pq.pop();
            del.pop();
        }
    }
    bool empty() { normalize(); return size() == 0; }
    int size() { return (int)pq.size() - (int)del.size(); }
    void insert(T x) { pq.push(x); }
    void erase(T x) { del.push(x); }
    T top() { normalize(); return pq.top(); }
    void pop() { normalize(); pq.pop(); }
    void clear() {
        while(!pq.empty()) pq.pop();
        while(!del.empty()) del.pop();
    }
};

```

Offline 2D BIT.cpp

Use: OfflineBIT2D<*T*> bit(*n*, points); add(*r,c,val*) and rect_sum(*r1,c1,r2,c2*) answer offline 2D point-add rectangle-sum queries.

Before: Pass every coordinate that can ever receive add() into todo during init, otherwise that point cannot be updated later.

Complexity: Memory is $O(\text{total compressed coordinates} \log n)$; add and rectangle sum are $O(\log^2 n)$.

When: Good for all queries known beforehand with point updates and rectangle sums on sparse coordinates.

```

template<typename T>
struct OfflineBIT2D{
    int n;
    vector<int> head, vals;
    vector<T> bit;
    OfflineBIT2D(){
        OfflineBIT2D(int _n, vector<array<int, 2>> todo){
            init(_n, todo);
        }
        int ind(int r, int c){

```

```

    int start = head[r], end = head[r + 1];
    return upper_bound(vals.begin() + start, vals.begin() + end, c) - vals.begin() -
        ↪ start - 1;
}

void init(int _n, vector<array<int, 2>> todo){
    n = _n;
    sort(todo.begin(), todo.end(), [](array<int, 2> &a, array<int, 2> &b){
        return a[1] < b[1];
    });
    vector<int> last(n + 1, -1), sz(n + 1, 1);
    for (auto [r, c] : todo){
        r++, c++;
        for (; r <= n; r += r & -r){
            if (last[r] != c){
                sz[r]++;
                last[r] = c;
            }
        }
    }
    head.assign(n + 2, 0);
    for (int i = 1; i <= n; i++){
        head[i + 1] = head[i] + sz[i];
    }
    vals.assign(head[n + 1], 0);
    bit.assign(head[n + 1], 0);
    last.assign(n + 1, -1);
    vector<int> cur = head;
    for (int i = 1; i <= n; i++){
        vals[cur[i]++] = 0;
    }
    for (auto [r, c] : todo){
        r++, c++;
        for (; r <= n; r += r & -r){
            if (last[r] != c){
                vals[cur[r]++] = c;
                last[r] = c;
            }
        }
    }
}

void add(int r, int c, T val){
    r++, c++;
    for (; r <= n; r += r & -r){
        int i = ind(r, c);
        int sz = head[r + 1] - head[r];
        for (; i < sz; i += i & -i){
            bit[head[r] + i] += val;
        }
    }
}

T rect_sum(int r, int c){
    r++, c++;
    T ret = 0;
    for (; r > 0; r -= r & -r){
        int i = ind(r, c);
        for (; i > 0; i -= i & -i){
            ret += bit[head[r] + i];
        }
    }
}

```

```

        return ret;
    }
    T rect_sum(int r1, int c1, int r2, int c2){
        if (r1 > r2 || c1 > c2) return 0;
        return rect_sum(r2, c2) - rect_sum(r2, c1 - 1) - rect_sum(r1 - 1, c2) +
            ↪ rect_sum(r1 - 1, c1 - 1);
    }
    T sum(int r, int c){
        return rect_sum(r, c);
    }
    T sum(int r1, int c1, int r2, int c2){
        return rect_sum(r1, c1, r2, c2);
    }
};

```

ordered_set.cpp

Use: ordered_set<T> supports find_by_order(k) for kth element and order_of_key(x) for count of elements < x.

Care: For duplicates, store pairs like {value, unique_id}; plain ordered_set<T> behaves like a mathematical set.

Complexity: insert, erase, order_of_key, and find_by_order are O(log n).

Pitfall: find_by_order(k) is 0-indexed and invalid when k is outside [0,size).

```

template<class T>
using ordered_set = tree<T, null_type, std::less<T>, rb_tree_tag,
    ↪ tree_order_statistics_node_update>;

```

Persistent DSU.cpp

Use: PersistentDSU d(n); add_edge/unite(a,b,t), same(a,b,t), get_size(v,t), components(t), and when_connected(u,v).

Care: Times are edge insertion versions; same/get_size/components answer the state at time t without mutating history.

Complexity: Queries are logarithmic over history vectors; when_connected does a binary search over time.

Before: Add edges in nondecreasing time so parent_time and component history stay meaningful.

```

// Add edges in nondecreasing time. Query with time 0 for the initial graph.
struct PersistentDSU{
    const int INF = 1e9;
    int n, timer;
    vector<int> parent, parent_time;
    vector<vector<pair<int, int>>> sz;
    vector<pair<int, int>> components_hist;
    PersistentDSU(){
        PersistentDSU(int _n){
            init(_n);
        }
        void init(int _n){
            n = _n;
            timer = 0;
            parent.resize(n + 1);
            parent_time.assign(n + 1, INF);
            sz.assign(n + 1, {});
            components_hist = {{0, n}};
            for (int i = 1; i <= n; i++){
                parent[i] = i;
                sz[i].push_back({0, 1});
            }
        }
    }
};

```

```

}
int find(int v, int t){
    return parent_time[v] > t ? v : find(parent[v], t);
}
int get_size(int v, int t){
    v = find(v, t);
    auto &h = sz[v];
    int p = upper_bound(h.begin(), h.end(), make_pair(t, INF)) - h.begin() - 1;
    return h[p].second;
}
int components(int t){
    int p = upper_bound(components_hist.begin(), components_hist.end(), make_pair(t,
    ↪ INF)) - components_hist.begin() - 1;
    return components_hist[p].second;
}
bool same(int u, int v, int t){
    return find(u, t) == find(v, t);
}
bool unite(int a, int b, int t){
    timer = max(timer, t);
    a = find(a, t);
    b = find(b, t);
    if (a == b) return false;
    int sa = get_size(a, t), sb = get_size(b, t);
    if (sa < sb){
        swap(a, b);
        swap(sa, sb);
    }
    parent[b] = a;
    parent_time[b] = t;
    sz[a].push_back({t, sa + sb});
    components_hist.push_back({t, components(t) - 1});
    return true;
}
bool add_edge(int a, int b, int t){
    return unite(a, b, t);
}
bool unite(int a, int b){
    return unite(a, b, timer + 1);
}
bool add_edge(int a, int b){
    return unite(a, b);
}
int when_connected(int u, int v, int hi){
    if (!same(u, v, hi)) return -1;
    int l = 0, r = hi;
    while (l < r){
        int mid = (l + r) >> 1;
        if (same(u, v, mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
int when_connected(int u, int v){
    return when_connected(u, v, timer);
}
};

```

PST.cpp

Use: PST pst(n); keep root 0 as empty, newRoot=pst.update(oldRoot,idx,val), query(root,l,r), and kth(L,R,k).

Care: Updates add val to one position and return a new root index; kth(L,R,k) compares two version roots.

Complexity: Each point update creates $O(\log n)$ nodes; each query and kth call is $O(\log n)$.

When: Use two roots for prefix versions to answer kth/order-statistics on subarrays.

```

struct PST{
    struct Node{ int l, r; ll val; };
    int n;
    vector<Node> tree;
    PST(){}
    PST(int _n, int m = 0){
        init(_n, m);
    }
    void init(int _n, int m = 0){
        n = _n;
        tree.clear();
        if(m) tree.reserve(m);
        tree.push_back({0, 0, 0});
    }
    int clone(int node){
        tree.push_back(tree[node]);
        return (int)tree.size() - 1;
    }
    int update(int node, int idx, ll val){
        return update(node, idx, val, 0, n - 1);
    }
    int update(int node, int idx, ll val, int l, int r){
        int cur = clone(node);
        tree[cur].val += val;
        if (l == r) return cur;
        int mid = (l + r) >> 1;
        if (idx <= mid) tree[cur].l = update(tree[cur].l, idx, val, l, mid);
        else tree[cur].r = update(tree[cur].r, idx, val, mid + 1, r);
        return cur;
    }
    ll query(int node, int ql, int qr){
        return query(node, ql, qr, 0, n - 1);
    }
    ll query(int node, int ql, int qr, int l, int r){
        if (!node || qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node].val;
        int mid = (l + r) >> 1;
        return query(tree[node].l, ql, qr, l, mid) + query(tree[node].r, ql, qr, mid + 1,
        ↪ r);
    }
    int kth(int L, int R, ll k){
        return kth(L, R, k, 0, n - 1);
    }
    int kth(int L, int R, ll k, int l, int r){
        if (l == r) return l;
        int mid = (l + r) >> 1;
        ll left = tree[tree[R].l].val - tree[tree[L].l].val;
        if (k <= left) return kth(tree[L].l, tree[R].l, k, l, mid);
        return kth(tree[L].r, tree[R].r, k - left, mid + 1, r);
    }
};

```


seg_tree (lazy generic).cpp

Use: Edit node::add and merge for the problem, then build(v), update(l,r,val), and query(l,r) on inclusive ranges.

Care: Public update/query convert [l,r] to internal half-open ranges, so call them with inclusive endpoints.

Complexity: Build is $O(n)$, range update/query are $O(\log n)$ when merge and lazy application are $O(1)$.

Before: Change neutral node, merge, and lazy apply together; mismatching them gives silent wrong answers.

```
struct node {
    ll lazy = 0, val = 0;
    bool isLazy = 0;
    node() {}
    node(ll x) : val(x) {}
    void add(ll x, int lx, int rx) {
        val += x; // for sum use: val += x * (rx - lx)
        lazy += x;
        isLazy = 1;
    }
};

#define lnode (x * 2 + 1)
#define rnode (x * 2 + 2)
#define md (lx + (rx - lx) / 2)
struct segment_tree {
    int n;
    vector<node> seg;
    segment_tree(int sz) {
        n = 1;
        while (n < sz) n *= 2;
        seg.assign(n * 2, node());
    }
    node merge(node &l, node &r) {
        node ret;
        ret.val = l.val + r.val; // min/max: change this
        return ret;
    }
    void build(vector<int> &v, int x, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < v.size()) seg[x] = node(v[lx]);
            return;
        }
        build(v, lnode, lx, md);
        build(v, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
    void build(vector<int> &v) { build(v, 0, 0, n); }
    void propagate(int x, int lx, int rx) {
        if (rx - lx == 1 || !seg[x].isLazy) return;
        seg[lnode].add(seg[x].lazy, lx, md);
        seg[rnode].add(seg[x].lazy, md, rx);
        seg[x].isLazy = seg[x].lazy = 0;
    }
    void update(int l, int r, ll val, int x, int lx, int rx) {
        propagate(x, lx, rx);
        if (lx >= r || rx <= l) return;
        if (lx >= l && rx <= r) return seg[x].add(val, lx, rx);
        update(l, r, val, lnode, lx, md);
        update(l, r, val, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
};
```

```
void update(int l, int r, ll val) { update(l, r + 1, val, 0, 0, n); }
node query(int l, int r, int x, int lx, int rx) {
    propagate(x, lx, rx);
    if (lx >= l && rx <= r) return seg[x];
    if (lx >= r || rx <= l) return node();
    node L = query(l, r, lnode, lx, md);
    node R = query(l, r, rnode, md, rx);
    return merge(L, R);
}
node query(int l, int r) {
    return query(l, r + 1, 0, 0, n);
}
};
```

seg_tree (lazy propagation).cpp

Use: Build with vector, then add/multiply/set/add_ap(l,r,...) and query(l,r) on inclusive 0-indexed ranges.

Care: Default value is range sum with affine index update $a*i+b$; change node if the problem needs another aggregate.

Complexity: Each add, multiply, set, add_ap, and query is $O(\log n)$.

Pitfall: The operations compose through lazyMul/lazyA/lazyB, so new update types need careful algebra.

```
struct node {
    ll lazyMul = 1, lazyA = 0, lazyB = 0, val = 0;
    bool isLazy = 0;
    node() {}
    node(ll x) : val(x) {}
    ll sumIdx(ll lx, ll rx) {
        return (lx + rx - 1) * (rx - lx) / 2;
    }
    void apply(ll mul, ll a, ll b, int lx, int rx) {
        val = val * mul + a * sumIdx(lx, rx) + b * (rx - lx);
        lazyMul *= mul;
        lazyA = lazyA * mul + a;
        lazyB = lazyB * mul + b;
        isLazy = 1;
    }
    void add(ll x, int lx, int rx) { apply(1, 0, x, lx, rx); }
    void multiply(ll x, int lx, int rx) { apply(x, 0, 0, lx, rx); }
    void set(ll x, int lx, int rx) { apply(0, 0, x, lx, rx); }
    void add_ap(ll first, ll diff, int l, int lx, int rx) {
        apply(1, diff, first - diff * l, lx, rx);
    }
};

#define lnode (x * 2 + 1)
#define rnode (x * 2 + 2)
#define md (lx + (rx - lx) / 2)
struct segment_tree {
    int n;
    vector<node> seg;
    segment_tree(int sz) {
        n = 1;
        while (n < sz) n *= 2;
        seg.assign(n * 2, node());
    }
    node merge(node &l, node &r) {
        node ret;
        ret.val = l.val + r.val;
    }
};
```

```

    return ret;
}
void build(vector<int> &v, int x, int lx, int rx) {
    if (rx - lx == 1) {
        if (lx < v.size()) seg[x] = node(v[lx]);
        return;
    }
    build(v, lnode, lx, md);
    build(v, rnode, md, rx);
    seg[x] = merge(seg[lnode], seg[rnode]);
}
void build(vector<int> &v) { build(v, 0, 0, n); }
void propagate(int x, int lx, int rx) {
    if (rx - lx == 1 || !seg[x].isLazy) return;
    seg[lnode].apply(seg[x].lazyMul, seg[x].lazyA, seg[x].lazyB, lx, md);
    seg[rnode].apply(seg[x].lazyMul, seg[x].lazyA, seg[x].lazyB, md, rx);
    seg[x].lazyMul = 1;
    seg[x].lazyA = seg[x].lazyB = 0;
    seg[x].isLazy = 0;
}
void update(int l, int r, ll mul, ll a, ll b, int x, int lx, int rx) {
    propagate(x, lx, rx);
    if (lx >= r || rx <= l) return;
    if (lx >= l && rx <= r) return seg[x].apply(mul, a, b, lx, rx);
    update(l, r, mul, a, b, lnode, lx, md);
    update(l, r, mul, a, b, rnode, md, rx);
    seg[x] = merge(seg[lnode], seg[rnode]);
}
void update(int l, int r, ll val) { add(l, r, val); }
void add(int l, int r, ll val) { update(l, r + 1, 1, 0, val, 0, 0, n); }
void multiply(int l, int r, ll val) { update(l, r + 1, val, 0, 0, 0, 0, n); }
void set(int l, int r, ll val) { update(l, r + 1, 0, 0, val, 0, 0, n); }
void add_ap(int l, int r, ll first, ll diff) {
    update(l, r + 1, 1, diff, first - diff * l, 0, 0, n);
}
}
node query(int l, int r, int x, int lx, int rx) {
    propagate(x, lx, rx);
    if (lx >= l && rx <= r) return seg[x];
    if (lx >= r || rx <= l) return node();
    node L = query(l, r, lnode, lx, md);
    node R = query(l, r, rnode, md, rx);
    return merge(L, R);
}
node query(int l, int r) {
    return query(l, r + 1, 0, 0, n);
}
};

```

seg_tree (point, range query).cpp

Use: segment_tree st(n, neutral, merge); set(i,val) point-updates and query(l,r) answers inclusive range queries.

Care: Give a true neutral node for empty segments; public query(l,r) is inclusive even though the recursion is half-open.

Complexity: Point set and range query are $O(\log n)$; memory is $O(\text{power_of_two } n)$.

When: Use this for associative range queries with point updates and no lazy propagation.

```

struct node{
    ll val;
    node(){

```

```

        node(ll v): val(v) {}
    };

struct segment_tree{
    int n;
    vector<node> tree;
    node NEUTRAL;
    function<node(const node&, const node&>> merge;
    segment_tree(int n_, node neutral,
        function<node(const node&, const node&>> merge_)
        : NEUTRAL(neutral), merge(merge_){
        n = 1;
        while (n < n_) n <= 1;
        tree.assign(n << 1, NEUTRAL);
    }
    void set(int i, ll val, int pos, int l, int r){
        if (r - l == 1) { tree[pos] = node(val); return; }
        int mid = (l + r) >> 1;
        if(i < mid) set(i, val, 2 * pos + 1, l, mid);
        else set(i, val, 2 * pos + 2, mid, r);
        tree[pos] = merge(tree[2 * pos + 1], tree[2 * pos + 2]);
    }
    void set(int i, ll val){ set(i, val, 0, 0, n); }
    node query(int l, int r, int pos, int nl, int nr){
        if (nr <= l || r <= nl) return NEUTRAL;
        if (l <= nl && nr <= r) return tree[pos];
        int mid = (nl + nr) >> 1;
        return merge(query(l, r, 2 * pos + 1, nl, mid), query(l, r, 2 * pos + 2, mid,
            nr));
    }
    node query(int l, int r){ return query(l, r + 1, 0, 0, n); }
};

```

sparse_table.cpp

Use: SparseTable<T> st(a, merge_func); query(l,r) answers inclusive idempotent range queries in $O(1)$.

Care: merge_func must be idempotent, like min/max/gcd, because query overlaps two blocks.

Complexity: Preprocessing is $O(n \log n)$, query is $O(1)$, and memory is $O(n \log n)$.

Pitfall: It cannot support updates; rebuild if the array changes.

```

template<class T>
struct SparseTable{
    int n, LOG;
    vector<int> lg;
    vector<vector<T>> table;
    function<T(T, T)> f;
    SparseTable(){
        SparseTable(vector<T> &v, function<T(T, T)> func){
            init(v, func);
        }
    }
    void init(vector<T> &v, function<T(T, T)> func){
        n = v.size();
        f = func;
        LOG = __lg(n) + 1;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++){
            lg[i] = lg[i >> 1] + 1;
        }
        table.assign(LOG, vector<T>(n));
    }
};

```

```

        table[0] = v;
        for (int j = 1; j < LOG; j++){
            for (int i = 0; i + (1 << j) <= n; i++){
                table[j][i] = f(table[j - 1][i], table[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    T query(int l, int r){
        if (l > r) swap(l, r);
        int j = lg[r - l + 1];
        return f(table[j][l], table[j][r - (1 << j) + 1]);
    }
};

```

topK.cpp

Use: TopKSum ds(k); insert/erase values, setK(k), and get() returns the sum of the current largest k elements.

Care: It supports duplicate values; erase(x) removes one copy if present, then rebalances the top-k set.

Complexity: insert, erase, and setK are $O(\log n)$ plus rebalancing moved elements.

When: Use it for sliding windows or dynamic sets where you need the sum of the largest K values.

```

struct TopKSum{
    int K;
    multiset<ll> top_k, rest;
    ll sum;
    TopKSum(){
        TopKSum(int k){
            init(k);
        }
        void init(int k){
            K = k;
            sum = 0;
            top_k.clear();
            rest.clear();
        }
        void fix(){
            while ((int)top_k.size() > K){
                auto it = top_k.begin();
                sum -= *it;
                rest.insert(*it);
                top_k.erase(it);
            }
            while ((int)top_k.size() < K && !rest.empty()){
                auto it = prev(rest.end());
                sum += *it;
                top_k.insert(*it);
                rest.erase(it);
            }
            while (!top_k.empty() && !rest.empty() && *top_k.begin() < *prev(rest.end())){
                auto small = top_k.begin();
                auto big = prev(rest.end());
                ll x = *small, y = *big;
                sum += y - x;
                top_k.erase(small);
                rest.erase(big);
                top_k.insert(y);
                rest.insert(x);
            }
        }
    }
};

```

```

void setK(int k){
    K = k;
    fix();
}
void insert(ll val){
    if (K == 0){
        rest.insert(val);
        return;
    }
    top_k.insert(val);
    sum += val;
    fix();
}
void erase(ll val){
    auto it = rest.find(val);
    if (it != rest.end()){
        rest.erase(it);
    }else{
        it = top_k.find(val);
        if (it != top_k.end()){
            sum -= val;
            top_k.erase(it);
        }
    }
    fix();
}
int size(){
    return (int)top_k.size() + (int)rest.size();
}
void clear(){
    top_k.clear();
    rest.clear();
    sum = 0;
}
ll get(){
    return sum;
}
};

```

TwoStackQ.cpp

Use: TwoStackQ q; edit Node and mrg for your aggregate, then push(x), pop(), and get() for the whole queue.

Care: Current Node tracks min and max; add more fields to Node and update mrg() before using other aggregates.

Complexity: push and pop are $O(1)$ amortized; get is $O(1)$.

Pitfall: This only supports queue order, not arbitrary deletions from the middle.

```

const ll inf = 1e18;
struct TwoStackQ {
    struct Node {
        ll mx = -inf, mn = inf, val;
        Node(): val(0) { }
        Node(ll x): mx(x), mn(x), val(x) { }
    };
    stack<Node> a, b;
    int size() { return a.size() + b.size(); }
    bool empty() { return size() == 0; }
    void clear() {
        while (!a.empty()) a.pop();
    }
};

```

```
        while (!b.empty()) b.pop();
    }
    void mrg(Node& a, Node& b) {
        a.mn = min(a.mn, b.mn);
        a.mx = max(a.mx, b.mx);
    }
    void push(ll val) {
        auto nd = Node(val);
        if (!a.empty()) mrg(nd, a.top());
        a.push(nd);
    }
    void move() {
        while (!a.empty()) {
            auto nd = Node(a.top().val);
            if (!b.empty()) mrg(nd, b.top());
            b.push(nd), a.pop();
        }
    }
    Node get() {
        Node res;
        if (!b.empty()) mrg(res, b.top());
        if (!a.empty()) mrg(res, a.top());
        return res;
    }
    void pop() {
        if (b.empty()) move();
        if (!b.empty()) b.pop();
    }
};
```

3 graphs

2SAT.cpp

Use: two_sat ts(n); add implications/clauses, then satisfiable() returns bool and fills answer[i].

Care: Variables are 0-indexed; add_or(i,val,j,val) represents literal choices directly through var(i,val).

Complexity: satisfiable() is $O(n + \text{clauses})$ after all implication edges are added.

Pitfall: Literal true/false convention is handled by var(i,val); avoid manual offsets unless adding raw edges.

```
struct two_sat{
    int n;
    vector<vector<int>> g, gr;
    vector<int> comp, top, answer;
    vector<bool> vis;
    two_sat(){
        two_sat(int _n){
            init(_n);
        }
        void init(int _n){
            n = _n;
            g.assign(2 * n, {});
            gr.assign(2 * n, {});
            comp.assign(2 * n, 0);
            vis.assign(2 * n, false);
            answer.assign(n, 0);
            top.clear();
        }
        int var(int i, bool val){
            return i + (val ? 0 : n);
        }
        void add_edge(int u, int v){
            g[u].push_back(v);
            gr[v].push_back(u);
        }
        void add_imply(int i, bool f, int j, bool k){
            add_edge(var(i, f), var(j, k));
        }
        void set_true(int i){
            add_imply(i, false, i, true);
        }
        void set_false(int i){
            add_imply(i, true, i, false);
        }
        void add_or(int i, bool f, int j, bool k){
            add_imply(i, !f, j, k);
            add_imply(j, !k, i, f);
        }
        void add_xor(int i, bool f, int j, bool k){
            add_or(i, f, j, k);
            add_or(i, !f, j, !k);
        }
        void add_xnor(int i, bool f, int j, bool k){
            add_or(i, f, j, !k);
            add_or(i, !f, j, k);
        }
        void add_and(int i, bool f, int j, bool k){
            add_imply(i, !f, i, f);
            add_imply(j, !k, j, k);
        }
        void dfs(int u){
```

```
            vis[u] = true;
            for (auto v : g[u]){
                if (!vis[v]) dfs(v);
            }
            top.push_back(u);
        }
        void scc(int u, int id){
            vis[u] = true;
            comp[u] = id;
            for (auto v : gr[u]){
                if (!vis[v]) scc(v, id);
            }
        }
        bool satisfiable(){
            top.clear();
            fill(vis.begin(), vis.end(), false);
            for (int i = 0; i < 2 * n; i++){
                if (!vis[i]) dfs(i);
            }
            fill(vis.begin(), vis.end(), false);
            reverse(top.begin(), top.end());
            int id = 0;
            for (auto v : top){
                if (!vis[v]) scc(v, id++);
            }
            for (int i = 0; i < n; i++){
                if (comp[i] == comp[i + n]) return false;
                answer[i] = comp[i] > comp[i + n];
            }
            return true;
        }
    };
```

Articulation points.cpp

Use: ArticulationPoints ap(n); add_edge(u,v), build() returns cut vertices and also fills isCut[v].

Care: The graph is undirected and 1-indexed; build() resets tin/low/isCut but keeps all added edges.

Complexity: build() is $O(n+m)$ and handles disconnected undirected graphs.

Pitfall: Parallel edges are safe because edge ids are tracked; self-loops are usually ignored before adding.

```
struct ArticulationPoints{
    int n, timer;
    vector<vector<pair<int, int>>> graph;
    vector<pair<int, int>> edges;
    vector<int> tin, low, isCut;
    ArticulationPoints(){
        ArticulationPoints(int _n){
            init(_n);
        }
        void init(int _n){
            n = _n;
            timer = 0;
            graph.assign(n + 1, {});
            edges.clear();
            tin.assign(n + 1, 0);
            low.assign(n + 1, 0);
            isCut.assign(n + 1, 0);
        }
        void add_edge(int u, int v){
```

```

        int id = edges.size();
        edges.push_back({u, v});
        graph[u].push_back({v, id});
        graph[v].push_back({u, id});
    }
    void dfs(int v, int pe = -1){
        tin[v] = low[v] = ++timer;
        int children = 0;
        for (auto [u, id] : graph[v]){
            if (id == pe) continue;
            if (tin[u]){
                low[v] = min(low[v], tin[u]);
            }else{
                dfs(u, id);
                low[v] = min(low[v], low[u]);
                if (pe != -1 && low[u] >= tin[v]){
                    isCut[v] = true;
                }
                children++;
            }
        }
        if (pe == -1 && children > 1){
            isCut[v] = true;
        }
    }
    vector<int> build(){
        timer = 0;
        fill(tin.begin(), tin.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(isCut.begin(), isCut.end(), 0);
        for (int i = 1; i <= n; i++){
            if (!tin[i]) dfs(i);
        }
        vector<int> ret;
        for (int i = 1; i <= n; i++){
            if (isCut[i]) ret.push_back(i);
        }
        return ret;
    }
};

```

Biconnected components.cpp

Use: BiconnectedComponents bc(n); add_edge(u,v), build(), then components and comp_edges store each biconnected block.

Out: components stores vertices and comp_edges stores original edge ids for each block; isolated vertices become singleton blocks.

Complexity: build() is $O(n+m)$; each edge enters exactly one component stack interval.

When: Use it for block-cut tree construction, articulation decomposition, or vertex-biconnected blocks.

```

struct BiconnectedComponents{
    int n, timer;
    vector<vector<pair<int, int>>> graph;
    vector<pair<int, int>> edges;
    vector<vector<int>> components, comp_edges;
    vector<int> tin, low, isCut, st;
    BiconnectedComponents(){
        BiconnectedComponents(int _n){
            init(_n);
        }
    }

```

```

}
void init(int _n){
    n = _n;
    timer = 0;
    graph.assign(n + 1, {});
    edges.clear();
    components.clear();
    comp_edges.clear();
    tin.assign(n + 1, 0);
    low.assign(n + 1, 0);
    isCut.assign(n + 1, 0);
    st.clear();
}
void add_edge(int u, int v){
    int id = edges.size();
    edges.push_back({u, v});
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
}
void add_component(int until){
    vector<int> verts, eids;
    while (true){
        int id = st.back();
        st.pop_back();
        eids.push_back(id);
        verts.push_back(edges[id].first);
        verts.push_back(edges[id].second);
        if (id == until) break;
    }
    sort(verts.begin(), verts.end());
    verts.erase(unique(verts.begin(), verts.end()), verts.end());
    components.push_back(verts);
    comp_edges.push_back(eids);
}
void dfs(int v, int pe = -1){
    tin[v] = low[v] = ++timer;
    int children = 0;
    for (auto [u, id] : graph[v]){
        if (id == pe) continue;
        if (!tin[u]){
            st.push_back(id);
            dfs(u, id);
            low[v] = min(low[v], low[u]);
            if (low[u] >= tin[v]){
                if (pe != -1 || children > 0) isCut[v] = true;
                add_component(id);
            }
            children++;
        }else if (tin[u] < tin[v]){
            st.push_back(id);
            low[v] = min(low[v], tin[u]);
        }
    }
    if (pe == -1 && children > 1){
        isCut[v] = true;
    }
}
void build(){
    timer = 0;

```

```

    components.clear();
    comp_edges.clear();
    st.clear();
    fill(tin.begin(), tin.end(), 0);
    fill(low.begin(), low.end(), 0);
    fill(isCut.begin(), isCut.end(), 0);
    for (int i = 1; i <= n; i++){
        if (tin[i]) continue;
        if (graph[i].empty()){
            components.push_back({i});
            comp_edges.push_back({});
        }else{
            dfs(i);
        }
    }
}
};

```

Bipartite check.cpp

Use: Bipartite bp(n); add_edge(u,v), build(), then ok says bipartite and color[v] gives the side.

Out: build() also stores ok and color; get_side(0/1) is meaningful after a successful build().

Complexity: build() is $O(n+m)$ over all connected components.

Pitfall: If ok is false, color is still filled but sides are not a valid bipartition.

```

struct Bipartite{
    int n;
    bool ok;
    vector<vector<int>> graph;
    vector<int> color;
    Bipartite(){}
    Bipartite(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        ok = true;
        graph.assign(n + 1, {});
        color.assign(n + 1, -1);
    }
    void add_edge(int u, int v){
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    bool build(){
        fill(color.begin(), color.end(), -1);
        ok = true;
        queue<int> q;
        for (int i = 1; i <= n; i++){
            if (color[i] != -1) continue;
            color[i] = 0;
            q.push(i);
            while (!q.empty()){
                int v = q.front();
                q.pop();
                for (auto u : graph[v]){
                    if (color[u] == -1){
                        color[u] = color[v] ^ 1;
                        q.push(u);
                    }
                }
            }
        }
    }
};

```

```

        }else if (color[u] == color[v]){
            ok = false;
        }
    }
}
return ok;
}
vector<int> get_side(int c){
    vector<int> ret;
    for (int i = 1; i <= n; i++){
        if (color[i] == c) ret.push_back(i);
    }
    return ret;
}
};

```

Blossom.cpp

Use: blossom bm(n); add_edge(u,v), then solve() returns maximum matching size and mate[v] gives partner or -1.

Care: Nodes are 0-indexed and edges are unweighted; use only for general graph maximum cardinality matching.

Complexity: This is polynomial but heavy; use Hopcroft-Karp instead when the graph is bipartite.

Pitfall: The implementation is for maximum cardinality matching, not weighted matching.

```

struct blossom {
    int n, m;
    vector<int> mate;
    vector<vector<int>> b;
    vector<int> p, d, bl;
    vector<vector<int>> g;
    blossom(int n) : n(n) {
        m = n + n / 2;
        mate.assign(n, -1);
        b.resize(m);
        p.resize(m);
        d.resize(m);
        bl.resize(m);
        g.assign(m, vector<int>(m, -1));
    }
    void add_edge(int u, int v) {
        g[u][v] = u;
        g[v][u] = v;
    }
    void match(int u, int v) {
        g[u][v] = g[v][u] = -1;
        mate[u] = v;
        mate[v] = u;
    }
    vector<int> trace(int x) {
        vector<int> vx;
        while(true) {
            while(bl[x] != x) x = bl[x];
            if(!vx.empty() && vx.back() == x) break;
            vx.push_back(x);
            x = p[x];
        }
        return vx;
    }
};

```

```

void contract(int c, int x, int y, vector<int> &vx, vector<int> &vy) {
    b[c].clear();
    int r = vx.back();
    while(!vx.empty() && !vy.empty() && vx.back() == vy.back()) {
        r = vx.back();
        vx.pop_back();
        vy.pop_back();
    }
    b[c].push_back(r);
    b[c].insert(b[c].end(), vx.rbegin(), vx.rend());
    b[c].insert(b[c].end(), vy.begin(), vy.end());
    for(int i = 0; i <= c; i++) {
        g[c][i] = g[i][c] = -1;
    }
    for(int z : b[c]) {
        bl[z] = c;
        for(int i = 0; i < c; i++) {
            if(g[z][i] != -1) {
                g[c][i] = z;
                g[i][c] = g[i][z];
            }
        }
    }
}

vector<int> lift(vector<int> &vx) {
    vector<int> A;
    while(vx.size() >= 2) {
        int z = vx.back(); vx.pop_back();
        if(z < n) {
            A.push_back(z);
            continue;
        }
        int w = vx.back();
        int i = (A.size() % 2 == 0 ? find(b[z].begin(), b[z].end(), g[z][w]) -
            ↪ b[z].begin() : 0);
        int j = (A.size() % 2 == 1 ? find(b[z].begin(), b[z].end(), g[z][A.back()]) -
            ↪ b[z].begin() : 0);
        int k = b[z].size();
        int dif = (A.size() % 2 == 0 ? i % 2 == 1 : j % 2 == 0) ? 1 : k - 1;
        while(i != j) {
            vx.push_back(b[z][i]);
            i = (i + dif) % k;
        }
        vx.push_back(b[z][i]);
    }
    return A;
}

int solve() {
    for(int ans = 0; ; ans++) {
        fill(d.begin(), d.end(), 0);
        queue<int> Q;
        for(int i = 0; i < m; i++) bl[i] = i;
        for(int i = 0; i < n; i++) {
            if(mate[i] == -1) {
                Q.push(i);
                p[i] = i;
                d[i] = 1;
            }
        }
    }
}

```

```

int c = n;
bool aug = false;
while(!Q.empty() && !aug) {
    int x = Q.front(); Q.pop();
    if(bl[x] != x) continue;
    for(int y = 0; y < c; y++) {
        if(bl[y] == y && g[x][y] != -1) {
            if(d[y] == 0) {
                p[y] = x;
                d[y] = 2;
                p[mate[y]] = y;
                d[mate[y]] = 1;
                Q.push(mate[y]);
            } else if(d[y] == 1) {
                vector<int> vx = trace(x);
                vector<int> vy = trace(y);
                if(vx.back() == vy.back()) {
                    contract(c, x, y, vx, vy);
                    Q.push(c);
                    p[c] = p[b[c][0]];
                    d[c] = 1;
                    c++;
                } else {
                    aug = true;
                    vx.insert(vx.begin(), y);
                    vy.insert(vy.begin(), x);
                    vector<int> A = lift(vx);
                    vector<int> B = lift(vy);
                    A.insert(A.end(), B.rbegin(), B.rend());
                    for(int i = 0; i < (int) A.size(); i += 2) {
                        match(A[i], A[i + 1]);
                        if(i + 2 < (int) A.size()) add_edge(A[i + 1], A[i +
                            ↪ 2]);
                    }
                }
            } else {
                break;
            }
        }
    }
}

if(!aug) return ans;
}
}
};

```

Bridges.cpp

Use: clear(n), add_edge(u,v), build(n), then bridges contains bridge endpoints and isBridge[id] marks bridge edge ids.

Care: Call clear(n) before rebuilding; bridge edge ids follow the order in which add_edge was called.

Complexity: build(n) is $O(n+m)$ and supports disconnected graphs.

Pitfall: Global arrays have fixed N; increase N before copying if constraints are larger.

```

const int N = 2e5 + 5;
vector<pair<int, int>> graph[N];
vector<pair<int, int>> edges, bridges;
vector<bool> isBridge;
int tin[N], low[N], timer;

```



```

void add_edge(int u, int v){
    int id = edges.size();
    edges.push_back({u, v});
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
}
void dfs(int v, int pe = -1){
    tin[v] = low[v] = ++timer;
    for (auto [u, id] : graph[v]){
        if (id == pe) continue;
        if (tin[u]){
            low[v] = min(low[v], tin[u]);
        }else{
            dfs(u, id);
            low[v] = min(low[v], low[u]);
            if (low[u] > tin[v]){
                isBridge[id] = true;
                bridges.push_back(edges[id]);
            }
        }
    }
}
void build(int n){
    timer = 0;
    bridges.clear();
    isBridge.assign(edges.size(), false);
    for (int i = 1; i <= n; i++){
        tin[i] = low[i] = 0;
    }
    for (int i = 1; i <= n; i++){
        if (!tin[i]) dfs(i);
    }
}
void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        tin[i] = low[i] = 0;
    }
    edges.clear();
    bridges.clear();
    isBridge.clear();
    timer = 0;
}

```

Centroid decomposition buckets.cpp

Use: Set graph/limit k, build(1), toggle(v) active nodes, and countWith(v) counts active nodes satisfying the bucket condition.

Out: paths is the maintained answer for active pairs with distance < k; set global k before toggles.

Complexity: build is $O(n \log n)$; each toggle/countWith touches $O(\log n)$ centroid ancestors with ordered_set logs.

Pitfall: Distances are weighted; make sure global k uses the same weight units as edge weights.

```

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    ↪ tree_order_statistics_node_update>;

```

```

const int N = 1e5 + 5;
vector<pair<int, ll>> graph[N];
vector<pair<int, ll>> up[N], side[N]; // {centroid / side id, distance}
ordered_set<pair<ll, int>> bucket[N];
vector<ordered_set<pair<ll, int>>> sideBucket(1);
bool rem[N], active[N];
int sz[N], par[N], sideId;
ll k, paths;

```

```

int pre(int v, int p = -1){
    sz[v] = 1;
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) sz[v] += pre(u, v);
    }
    return sz[v];
}
int cen(int v, int tree_size, int p = -1){
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u] && sz[u] * 2 > tree_size){
            return cen(u, tree_size, v);
        }
    }
    return v;
}
void dfs(int v, int p, int c, ll d, int id){
    up[v].push_back({c, d});
    side[v].push_back({id, d});
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) dfs(u, v, c, d + w, id);
    }
}
void build(int v = 1, int p = 0){
    int c = cen(v, pre(v));
    par[c] = p;
    rem[c] = true;
    up[c].push_back({c, 0});
    side[c].push_back({0, 0});
    for (auto [u, w] : graph[c]){
        if (rem[u]) continue;
        sideBucket.emplace_back();
        dfs(u, c, c, w, ++sideId);
    }
    for (auto [u, w] : graph[c]){
        if (!rem[u]) build(u, c);
    }
}
int get(ordered_set<pair<ll, int>> &st, ll x){
    if (x < 0) return 0;
    return st.order_of_key({x, INT_MAX});
}
ll countWith(int v){
    ll ret = 0;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        ret += get(bucket[c], k - d);
        if (id) ret -= get(sideBucket[id], k - d);
    }
}

```

```

    return ret;
}
void add(int v){
    if (active[v]) return;
    paths += countWith(v);
    active[v] = true;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        bucket[c].insert({d, v});
        if (id) sideBucket[id].insert({d, v});
    }
}
void erase(int v){
    if (!active[v]) return;
    active[v] = false;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        bucket[c].erase({d, v});
        if (id) sideBucket[id].erase({d, v});
    }
    paths -= countWith(v);
}
void toggle(int v){
    active[v] ? erase(v) : add(v);
}
void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        up[i].clear();
        side[i].clear();
        bucket[i].clear();
        rem[i] = active[i] = false;
        sz[i] = par[i] = 0;
    }
    sideBucket.assign(1, ordered_set<pair<ll, int>>());
    sideId = 0;
    paths = 0;
}

```

Centroid decomposition with updates.cpp

Use: Build after filling tree; add/erase/toggle active nodes and query(v) returns the best distance-style answer for v.

Out: query(v) returns the minimum distance from v to an active node, or INF when no active node exists.

Complexity: build is $O(n \log n)$; add, erase, toggle, and query are $O(\log^2 n)$ with multisets.

Pitfall: When erasing, remove only one matching distance from each multiset.

```

const int N = 1e5 + 5;
const ll INF = 4e18;
vector<pair<int, ll>> graph[N], path[N];
multiset<ll> st[N];
bool rem[N], active[N];
int sz[N], par[N];

int pre(int v, int p = -1){
    sz[v] = 1;
    for (auto [u, w] : graph[v]){

```

```

        if (u != p && !rem[u]) sz[v] += pre(u, v);
    }
    return sz[v];
}
int cen(int v, int tree_size, int p = -1){
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u] && sz[u] * 2 > tree_size){
            return cen(u, tree_size, v);
        }
    }
    return v;
}
void dfs(int v, int p, int c, ll d){
    path[v].push_back({c, d});
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) dfs(u, v, c, d + w);
    }
}
void build(int v = 1, int p = 0){
    int c = cen(v, pre(v));
    par[c] = p;
    rem[c] = true;
    dfs(c, 0, c, 0);
    for (auto [u, w] : graph[c]){
        if (!rem[u]) build(u, c);
    }
}
void add(int v){
    if (active[v]) return;
    active[v] = true;
    for (auto [c, d] : path[v]) st[c].insert(d);
}
void erase(int v){
    if (!active[v]) return;
    active[v] = false;
    for (auto [c, d] : path[v]){
        auto it = st[c].find(d);
        if (it != st[c].end()) st[c].erase(it);
    }
}
void toggle(int v){
    active[v] ? erase(v) : add(v);
}
ll query(int v){
    ll ans = INF;
    for (auto [c, d] : path[v]){
        if (!st[c].empty()) ans = min(ans, d + *st[c].begin());
    }
    return ans;
}
void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        path[i].clear();
        st[i].clear();
        rem[i] = active[i] = false;
        sz[i] = par[i] = 0;
    }
}

```

```
}
```

Centroid decomposition.cpp

Use: Fill process_centroid/dfs_calc/dfs_add for your counting task, then build(1) decomposes and processes the tree.

Care: This is a fill-in skeleton: put your counting logic in dfs_calc, dfs_add, and process_centroid.

Complexity: Centroid decomposition visits each level once, usually $O(n \log n)$ plus your custom work.

Before: Clear per-centroid temporary arrays after process_centroid to avoid state leaking between subtrees.

```
const int N = 2e5 + 5;
vector<int> graph[N];
bool rem[N];
int sz[N];
int answer;

int pre(int v, int p = -1){
    sz[v] = 1;
    for(auto &u : graph[v]){
        if(u == p || rem[u]) continue;
        sz[v] += pre(u, v);
    }
    return sz[v];
}

int cen(int v, int tree_size, int p = -1){
    for(auto &u : graph[v]){
        if(u == p || rem[u]) continue;
        if(sz[u] * 2 > tree_size) return cen(u, tree_size, v);
    }
    return v;
}

void process_centroid(int c);
void build(int v = 1){
    int centroid = cen(v, pre(v));
    process_centroid(centroid);
    rem[centroid] = true;
    for(auto &u : graph[centroid]){
        if(rem[u]) continue;
        build(u);
    }
}

void dfs_calc(int &v, int &p) {
    for(int &u : graph[v]){
        if(u == p || rem[u]) continue;
        dfs_calc(u, v);
    }
}

void dfs_add(int &v, int &p){
    for(int &u : graph[v]){
        if(u == p || rem[u]) continue;
        dfs_add(u, v);
    }
}

void process_centroid(int c){
    for(int &u : graph[c]) {
        if(rem[u]) continue;
        dfs_calc(u, c);
        dfs_add(u, c);
    }
}
```

```
for(int &u : graph[c]){
    if(rem[u]) continue;
    dfs_add(u, c);
}

void clear(int n){
    for(int i = 1; i <= n; i++){
        rem[i] = sz[i] = 0;
    }
}
```

dijkstra.cpp

Use: Fill graph adjacency with (to,w); dijkstra(src,parent) returns distances and parents, or dijkstra(src) returns distances only.

Care: Set global n and graph first; edge weights must be nonnegative or Dijkstra is invalid.

Complexity: Runs in $O((n+m) \log n)$ with the priority queue.

Out: Unreachable vertices keep INF distance and parent -1.

```
const ll INF = 4e18;
int n;
vector<vector<pair<int, ll>>> graph;

vector<ll> dijkstra(int src, vector<int> &parent){
    vector<ll> dist(n + 1, INF);
    parent.assign(n + 1, -1);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()){
        auto [d, v] = pq.top();
        pq.pop();
        if (d != dist[v]) continue;
        for (auto [u, w] : graph[v]){
            if (dist[u] > d + w){
                dist[u] = d + w;
                parent[u] = v;
                pq.push({dist[u], u});
            }
        }
    }
    return dist;
}

vector<ll> dijkstra(int src){
    vector<int> parent;
    return dijkstra(src, parent);
}
```

Dinic.cpp

Use: Dinic flow(n,s,t); add_edge(u,v,cap), then flow() returns max flow and residual data stays in E.

Out: getEdges() returns a min-cut edge set after flow(); getMatchingEdges(n,m) extracts bipartite matching edges.

Complexity: Fast in practice; standard bound is $O(V^2 E)$, often better on unit networks.

Pitfall: Constructor stores s and t, so create a new Dinic or reset flows before another max-flow run.

```
const ll inf = 1e18;
struct Edge {
    int u, v;
```

```

    ll cap, flow = 0;
    Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
};

struct Dinic {
    vector<Edge> E;
    vector<vector<int>>> adj;
    vector<int> level, ptr;
    queue<int> q;
    int n, m = 0, s, t;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }
    void add_edge(int u, int v, ll cap) {
        E.push_back(Edge(u, v, cap));
        E.push_back(Edge(v, u, 0));
        adj[u].push_back(m);
        adj[v].push_back(m + 1);
        m += 2;
    }
    bool bfs() {
        while (q.size()) {
            auto u = q.front();
            q.pop();
            for (auto &id : adj[u]) {
                if (E[id].cap == E[id].flow) continue;
                if (level[E[id].v] != -1) continue;
                level[E[id].v] = level[u] + 1;
                q.push(E[id].v);
            }
        }
        return level[t] != -1;
    }
    ll dfs(int u, ll pushed) {
        if (pushed == 0) return 0;
        if (u == t) return pushed;
        for (int &cid = ptr[u]; cid < (int)adj[u].size(); ++cid) {
            int id = adj[u][cid];
            ll v = E[id].v;
            if (level[u] + 1 != level[v]) continue;
            ll tr = dfs(v, min(pushed, E[id].cap - E[id].flow));
            if (tr == 0) continue;
            E[id].flow += tr;
            E[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }
    ll flow() {
        ll ret = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs()) break;
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, inf)) {
                ret += pushed;
            }
        }
    }
};

```

```

    }
    return ret;
}

// get any set of E to achieve the min cut (max flow)
vector<pair<int, int>> getEdges() {
    vector<int> srcSide(n, 0);
    queue<int> q;
    q.push(s);
    srcSide[s] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto &id : adj[u]) {
            if (!srcSide[E[id].v] && E[id].flow < E[id].cap) {
                srcSide[E[id].v] = 1;
                q.push(E[id].v);
            }
        }
    }
    vector<pair<int, int>> res;
    for (int u = 0; u < n; ++u) {
        if (!srcSide[u]) continue;
        for (auto &id : adj[u]) {
            if (!srcSide[E[id].v] && E[id].cap > 0)
                res.push_back({u, E[id].v});
        }
    }
    return res;
}

// Get The Edges Used In Max Bipartite Matching : s(0), t(n + m + 1)
vector<pair<int, int>> getMatchingEdges(int n, int m) {
    vector<pair<int, int>> ret;
    for (auto &e : E) {
        if (1 <= e.u && e.u <= n && n + 1 <= e.v && e.v <= n + m) {
            if (e.flow > 0) {
                ret.push_back({e.u, e.v - n});
            }
        }
    }
    return ret;
}
};

```

DSU on trees.cpp

Use: Fill add/remove/query logic inside the DFS lambdas; run dfs(root,parent,true) to keep the root subtree data.

Care: The kept SOLVE() contains the actual DSU-on-tree traversal; fill add/remove before dfs(1,-1,1).

Complexity: The small-to-large traversal is $O(n \log n)$ or $O(n)$ depending on add/remove work.

Pitfall: Only the heavy child data is kept; light subtrees are removed unless keep is true.

```

void SOLVE(){
    int n; cin >> n;
    vector<vector<int>>> graph(n + 1);
    for(int i = 0, u, v; i < n - 1; i++){
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
}

```

```

}
vector<int> tin(n + 1), tout(n + 1), vertex(n + 1), sz(n + 1, 1), d(n + 1, 1);
int timer = 0;
function<int(int, int)> euler = [&](int u, int p){
    tin[u] = ++timer;
    vertex[tin[u]] = u;
    for(auto v : graph[u]) if(v != p){
        d[v] = d[u] + 1;
        sz[u] += euler(v, u);
    }
    tout[u] = timer;
    return sz[u];
};
euler(1, -1);

auto add = [&](int u){};
auto remove = [&](int u){};

function<void(int, int, bool)> dfs = [&](int u, int p, bool keep){
    int big = -1;
    for(auto v : graph[u]){
        if(v == p) continue;
        if(big == -1 || sz[v] > sz[big]) big = v;
    }
    for(auto v : graph[u])
        if(v != p && v != big) dfs(v, u, 0);
    if(big != -1) dfs(big, u, 1);
    for(auto v : graph[u]){
        if(v == p || v == big) continue;
        for(int i = tin[v]; i <= tout[v]; i++){
            // here
            add(vertex[i]);
        }
    }
    add(u);
    if(!keep){
        for(int i = tin[u]; i <= tout[u]; i++) remove(vertex[i]);
    }
};
dfs(1, -1, 1);
}

```

eulerWalk.cpp

Use: Pass adjacency with (to,edge_id), edge count, and source; eulerWalk returns the Euler vertex order or empty if impossible.

Care: Use unique edge ids from 0 to nedges-1; for undirected graphs add both directions with the same id.

Complexity: Runs in $O(n+m)$ and consumes each edge id once.

Pitfall: The function checks degree feasibility through D; still choose a valid source for paths.

```

/*
undirected/cycle: every vertex is even
undirected/path: 0/2 vertices are odd.
directed/cycle: every vertex in == out
directed/path: exactly one (out - in == 1) -> src
                exactly one (in - out == 1) -> sink
                all other in == out.
*/
vector<int> eulerWalk(vector<vector<pair<int, int>>> &gr, int nedges, int src = 0) {

```

```

int n = gr.size();
vector<int> D(n), its(n), eu(nedges), ret, s = {src};
D[src]++; // to allow Euler paths, not just cycles
while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = gr[x].size();
    if (it == end) {
        ret.push_back(x);
        s.pop_back();
        continue;
    }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
        D[x]--, D[y]++;
        eu[e] = 1;
        s.push_back(y);
    }
}
for (int x : D) if (x < 0 || ret.size() != nedges + 1) return {};
return {ret.rbegin(), ret.rend()};
}

```

HLD.cpp

Use: HLD h(n); add edges, init(root), then tin/tout/head/par/d/path_sorted split path and subtree ranges.

Out: path_sorted(u,v) returns tin ranges plus a reversed flag, which is useful for ordered path queries.

Complexity: init is $O(n)$; each path is split into $O(\log n)$ tin intervals.

Pitfall: dfs removes parent edges from child adjacency, so do not reuse the original undirected adjacency after init.

```

struct HLD {
    int root = 0;
    vector<vector<int>>> g;
    explicit HLD(int n) : g(n) {}
    void add(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> &operator[](int u) {
        return g[u];
    }
    int timer = 0;
    vector<int> tin, tout, d, sz, head, par, vertex;
    void init(int rt = 0) {
        root = rt;
        tin = tout = d = head = par = vertex = vector<int>(g.size() + 1);
        sz.resize(g.size(), 1);
        par[root] = head[root] = root;
        dfs(root); dfs2(root);
    }
    void dfs(int u){
        for(int &v : g[u]){
            d[v] = d[u] + 1;
            par[v] = u;
            g[v].erase(find(g[v].begin(), g[v].end(), u));
            dfs(v);
            sz[u] += sz[v];
            if(sz[v] > sz[g[u][0]]) swap(v, g[u][0]);
        }
    }
}

```

```

void dfs2(int u){
    tin[u] = ++timer;
    vertex[tin[u]] = u;
    for(int v : g[u]){
        head[v] = v == g[u][0] ? head[u] : v;
        dfs2(v);
    }
    tout[u] = timer;
}

int jump(int u, int k) {
    if(k > d[u]) return -1;
    int depth = d[u] - k;
    while (d[head[u]] > depth) { u = par[head[u]]; }
    return vertex[tin[u] - d[u] + depth];
}

bool isAnc(int u, int v) {
    return tin[u] <= tin[v] && tin[v] <= tout[u];
}

int lca(int u, int v) {
    if(d[u] > d[v]) swap(u, v);
    if(isAnc(u, v)) return u;
    while(head[u] != head[v]){
        if (d[head[u]] > d[head[v]]) u = par[head[u]];
        else v = par[head[v]];
    }
    return d[u] < d[v] ? u : v;
}

int dis(int u, int v) {
    return d[u] + d[v] - 2 * d[lca(u, v)];
}

auto path_sorted(int u, int v) {
    vector<tuple<int, int, bool>> U, V; // from, to, reversed
    while(head[u] != head[v]){
        if(d[head[u]] > d[head[v]]){
            U.emplace_back(tin[head[u]], tin[u], true);
            u = par[head[u]];
        }else{
            V.emplace_back(tin[head[v]], tin[v], false);
            v = par[head[v]];
        }
    }
    if(d[u] < d[v]) V.emplace_back(tin[u], tin[v], false);
    else U.emplace_back(tin[v], tin[u], true);
    U.insert(U.end(), V.rbegin(), V.rend());
    return U;
}
};

```

Hopcroftkarp.cpp

Use: HopcroftKarp hk(nLeft,nRight); add_edge(l,r), then maximum_matching() returns size and l/r store matches.

Care: Left nodes are 1..nLeft and right nodes are passed as 1..nRight; add_edge shifts right nodes internally.

Complexity: maximum_matching is $O(E \sqrt{V})$, much faster than flow for large bipartite matching.

Out: If l[u] is nonzero, left node u is matched to right node l[u]-n.

```

#define ll long long
#define endl "\n"

```

```

struct HopcroftKarp {
    static const int inf = 1e9;
    int n;
    vector<int> l, r, d;
    vector<vector<int>> g;
    HopcroftKarp(int _n, int _m) {
        n = _n;
        int p = _n + _m + 1;
        g.resize(p); l.resize(p, 0); r.resize(p, 0); d.resize(p, 0);
    }
    void add_edge(int u, int v) {
        g[u].push_back(v + n);
    }
    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (!l[u]) d[u] = 0, q.push(u);
            else d[u] = inf;
        }
        d[0] = inf;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto v : g[u]) {
                if (d[r[v]] == inf) { d[r[v]] = d[u] + 1; q.push(r[v]); }
            }
        }
        return d[0] != inf;
    }
    bool dfs(int u) {
        if (!u) return true;
        for (auto v : g[u]) {
            if(d[r[v]] == d[u] + 1 && dfs(r[v])) {
                l[u] = v; r[v] = u; return true;
            }
        }
        d[u] = inf;
        return false;
    }
    int maximum_matching() {
        int ans = 0;
        while (bfs()) {
            for(int u = 1; u <= n; u++) if (!l[u] && dfs(u)) ans++;
        }
        return ans;
    }
};

```

LCA and Binary lifting.cpp

Use: LCA lca(g,root); use lca(u,v), dist(u,v), distance(u,v), kth(u,k), jump(u,k), and go(u,v,k).

Care: The tree is 0-indexed and unweighted by default; distance() uses distRoot, which currently adds 1 per edge.

Complexity: Preprocessing is $O(n \log n)$; lca, kth, and go are $O(\log n)$.

Pitfall: go(u,v,k) asserts k is on the path length; pass 0 for u and dist(u,v) for v.

```

struct LCA{
    int n, Log, timer;
    vector<vector<int>> up;
    vector<int> depth, tin, tout;

```

```

vector<ll> distRoot;
LCA(){}
LCA(vector<vector<int>> &g, int root = 0){
    init(g, root);
}
void init(vector<vector<int>> &g, int root = 0){
    n = g.size();
    Log = __lg(n) + 1;
    timer = 0;
    up.assign(n, vector<int>(Log, root));
    depth.assign(n, 0);
    tin.assign(n, 0);
    tout.assign(n, 0);
    distRoot.assign(n, 0);
    dfs(root, root, g);
}
void dfs(int u, int p, vector<vector<int>> &g){
    tin[u] = ++timer;
    up[u][0] = p;
    for (int l = 1; l < Log; l++){
        up[u][l] = up[up[u][l-1]][l-1];
    }
    for (int v : g[u]){
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        distRoot[v] = distRoot[u] + 1;
        dfs(v, u, g);
    }
    tout[u] = timer;
}
int kth(int u, int k){
    while (k){
        u = up[u][__builtin_ctz(k)];
        k &= k - 1;
    }
    return u;
}
int jump(int u, int k){
    return kth(u, k);
}
bool isAnc(int u, int v){
    return tin[u] <= tin[v] && tout[v] <= tout[u];
}
int lca(int u, int v){
    if (depth[u] < depth[v]) swap(u, v);
    u = kth(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int l = Log - 1; l >= 0; l--){
        if (up[u][l] != up[v][l]){
            u = up[u][l];
            v = up[v][l];
        }
    }
    return up[u][0];
}
int dist(int u, int v, int l = -1){
    if (l == -1) l = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[l];
}

```

```

ll distance(int u, int v, int l = -1){
    if (l == -1) l = lca(u, v);
    return distRoot[u] + distRoot[v] - 2 * distRoot[l];
}
int go(int u, int v, int k){
    int l = lca(u, v);
    int d = dist(u, v, l);
    assert(k <= d);
    if (depth[l] + k <= depth[u]) return kth(u, k);
    k -= depth[u] - depth[l];
    return kth(v, depth[v] - depth[l] - k);
}
};

```

MCMF.cpp

Use: MCMF mf(n,s,t); addEdge(u,v,cost,cap), then minCostMaxFlow(K) returns {flow,cost}.

Care: addEdge parameter order is cost then cap; get_paths(k) assumes cost==1 edges mark the wanted path edges.

Complexity: Each augmentation uses SPFA-style shortest path, so use carefully on large dense graphs.

Out: minCostMaxFlow(K) may return flow < K if the sink cannot receive enough capacity.

```

const int inf = 1000000010;
struct Edge {
    int to, cost, cap, flow, backEdge;
};
struct MCMF {
    int s, t, n;
    vector<vector<Edge>> g;
    MCMF(int _n, int _s, int _t){
        s = _s, t = _t, n = _n + 1;
        g = vector<vector<Edge>>(n);
    }
    void addEdge(int u, int v, int cost, int cap) {
        g[u].push_back({ v, cost, cap, 0, (int)g[v].size() });
        g[v].push_back({ u, -cost, 0, 0, (int)g[u].size() - 1});
    }
    pair<int, int> minCostMaxFlow(int K) {
        int flow = 0, cost = 0;
        vector<int> state(n), from(n), from_edge(n), d(n);
        deque<int> q;

        while(flow < K){
            for (int i = 0; i < n; i++){
                state[i] = 2, d[i] = inf, from[i] = -1;
            }
            state[s] = 1; q.clear(); q.push_back(s); d[s] = 0;
            while (!q.empty()) {
                int v = q.front(); q.pop_front(); state[v] = 0;
                for (int i = 0; i < (int) g[v].size(); i++) {
                    Edge e = g[v][i];
                    if (e.flow >= e.cap || d[e.to] <= d[v] + e.cost)
                        continue;
                    int to = e.to; d[to] = d[v] + e.cost;
                    from[to] = v; from_edge[to] = i;
                    if(state[to] == 1) continue;
                    q.push_back(to);
                    state[to] = 1;
                }
            }
        }
    }
};

```

```

        if (d[t] == inf) break;
        int it = t, addflow = inf;
        while (it != s) {
            addflow = min(addflow, g[from[it]][from_edge[it]].cap
                           - g[from[it]][from_edge[it]].flow);
            it = from[it];
        }
        addflow = min(addflow, K - flow);
        it = t;
        while (it != s) {
            g[from[it]][from_edge[it]].flow += addflow;
            g[it][g[from[it]][from_edge[it]].backEdge].flow -= addflow;
            cost += g[from[it]][from_edge[it]].cost * addflow;
            it = from[it];
        }
        flow += addflow;
    }
    return {flow, cost};
}

vector<vector<int>> get_paths(int k) {
    vector<vector<int>> paths;
    for(int i = 0; i < k; i++){
        vector<int> path;
        int curr = s;
        path.push_back(curr);
        while(curr != t){
            for(auto &e : g[curr]){
                if(e.cost == 1 && e.flow > 0){
                    e.flow--;
                    curr = e.to;
                    path.push_back(curr);
                    break;
                }
            }
        }
        paths.push_back(path);
    }
    return paths;
}
};

```

Rerooting.cpp

Use: Edit base(), merge_into(), finalize_merge(); fill graph and call solve_reroot() to get dp for every root.

Care: Edit Agg, Val, base, merge_into, and finalize_merge; graph must be 0-indexed and connected.

Complexity: The generic reroot pass is $O(n \log \text{degree})$ due the stack split, usually near $O(n)$.

Out: Returns root_dp plus edge_dp/redge_dp, useful when answers also need directed edge contributions.

```

const int N = 2e5 + 5;
using Agg = ll; // in-progress dp of a node
using Val = ll; // final dp of a node
vector<vector<int>> graph; // 0-based nodes
int a[N];
// Initialize the DP state for a node
Agg base(int node) {
    return 0;
}
// Merge a child's finalized value into the current node's aggregate
Agg merge_into(Agg cur_dp, Val ch_val, int node, int edge_idx) {

```

```

    return cur_dp + ch_val;
}
// Finalize the aggregate before passing it up (add node/edge weights)
Val finalize_merge(Agg cur_dp, int node, int par_edge_idx) {
    return cur_dp;
}
// root_dp[u]: dp of the tree when rooted at u
// edge_dp[u][i]: dp of the subtree of node graph[u][i] excluding the branch of u
// redge_dp[u][i]: dp of the tree rooted at u excluding the branch of graph[u][i]
auto solve_reroot() {
    int n = graph.size();
    vector<Val> root_dp(n), dp(n);
    vector<vector<Val>> edge_dp(n), redge_dp(n);
    vector<int> bfs, par(n, -1);
    bfs.reserve(n);
    bfs.push_back(0);
    for(int i = 0; i < n; i++){
        int u = bfs[i];
        for(int v : graph[u]){
            if(v == par[u]) continue;
            par[v] = u;
            bfs.push_back(v);
        }
    }
    for(int i = n - 1; i >= 0; i--){
        int u = bfs[i], p_idx = -1;
        Agg agg = base(u);
        for(int j = 0; j < (int)graph[u].size(); j++){
            int v = graph[u][j];
            if(v == par[u]) p_idx = j;
            else agg = merge_into(agg, dp[v], u, j);
        }
        dp[u] = finalize_merge(agg, u, p_idx);
    }
    for(int u : bfs){
        int deg = graph[u].size();
        edge_dp[u].resize(deg);
        redge_dp[u].resize(deg);
        Agg tot = base(u);
        for (int j = 0; j < deg; j++){
            int v = graph[u][j];
            edge_dp[u][j] = (v == par[u] ? dp[u] : dp[v]);
            tot = merge_into(tot, edge_dp[u][j], u, j);
        }
        root_dp[u] = finalize_merge(tot, u, -1);
    }
    if(deg) {
        struct Frame { int l, r; Agg out; };
        vector<Frame> st;
        st.reserve(deg * 2);
        st.push_back({0, deg - 1, base(u)});
        while(!st.empty()) {
            auto [l, r, out] = st.back();
            st.pop_back();
            if(l == r){
                redge_dp[u][l] = finalize_merge(out, u, l);
                if(graph[u][l] != par[u]) dp[graph[u][l]] = redge_dp[u][l];
                continue;
            }
            int mid = l + ((r - l) >> 1);

```



```

        Agg L_out = out, R_out = out;
        for(int i = mid + 1; i <= r; i++) L_out = merge_into(L_out,
            ↪ edge_dp[u][i], u, i);
        for(int i = 1; i <= mid; i++) R_out = merge_into(R_out, edge_dp[u][i], u,
            ↪ i);
        st.push_back({mid + 1, r, R_out});
        st.push_back({1, mid, L_out});
    }
}
return make_tuple(root_dp, edge_dp, redge_dp);
}

```

SCC.cpp

Use: SCC scc(n); add_edge(u,v), build(), then comp[v] is component id and comps/condensation describe the DAG.

Out: After build(), comp[v] is the component id, groups lists vertices, and dag is the condensed graph.

Complexity: Kosaraju build is $O(n+m)$, then condensation DAG is deduplicated by sorting edges.

Pitfall: Nodes are 1-indexed; component ids are 0..components-1 in the second-pass order.

```

struct SCC{
    int n, components;
    vector<vector<int>> graph, rev, groups, dag;
    vector<int> comp, order, vis;
    SCC(){}
    SCC(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        components = 0;
        graph.assign(n + 1, {});
        rev.assign(n + 1, {});
        comp.assign(n + 1, -1);
        vis.assign(n + 1, 0);
        order.clear();
        groups.clear();
        dag.clear();
    }
    void add_edge(int u, int v){
        graph[u].push_back(v);
        rev[v].push_back(u);
    }
    void dfs1(int v){
        vis[v] = 1;
        for (auto u : graph[v]){
            if (!vis[u]) dfs1(u);
        }
        order.push_back(v);
    }
    void dfs2(int v, int id){
        comp[v] = id;
        groups[id].push_back(v);
        for (auto u : rev[v]){
            if (comp[u] == -1) dfs2(u, id);
        }
    }
    void build(){

```

```

        fill(vis.begin(), vis.end(), 0);
        fill(comp.begin(), comp.end(), -1);
        order.clear();
        for (int i = 1; i <= n; i++){
            if (!vis[i]) dfs1(i);
        }
        reverse(order.begin(), order.end());
        components = 0;
        groups.clear();
        for (auto v : order){
            if (comp[v] == -1){
                groups.push_back({});
                dfs2(v, components++);
            }
        }
        dag.assign(components, {});
        vector<pair<int, int>> edges;
        for (int v = 1; v <= n; v++){
            for (auto u : graph[v]){
                if (comp[v] != comp[u]) edges.push_back({comp[v], comp[u]});
            }
        }
        sort(edges.begin(), edges.end());
        edges.erase(unique(edges.begin(), edges.end()), edges.end());
        for (auto [u, v] : edges){
            dag[u].push_back(v);
        }
    }
};

```

tree isomorphism.cpp

Use: Call tree_hash(graph) for canonical center hashes or isomorphic(a,b) to compare two unrooted trees.

Care: Global ids are shared across calls; that is intended for comparison, but clear ids/ID if you need a fresh hash universe.

Complexity: Hashing is $O(n \log n)$ mainly from sorting child hashes.

Pitfall: The hash ids are deterministic inside one run but not canonical across different programs unless built in same order.

```

map<vector<int>, int> ids;
int ID = 1;

int hash_node(vector<int> ch){
    sort(ch.begin(), ch.end());
    if (!ids.count(ch)) ids[ch] = ID++;
    return ids[ch];
}

int rooted_hash(int v, int p, vector<vector<int>> &graph){
    vector<int> ch;
    for (auto u : graph[v]){
        if (u != p) ch.push_back(rooted_hash(u, v, graph));
    }
    return hash_node(ch);
}

vector<int> centers(vector<vector<int>> &graph){
    int n = graph.size() - 1;
    vector<int> deg(n + 1);
    queue<int> q;
    for (int i = 1; i <= n; i++){

```

```

        deg[i] = graph[i].size();
        if (deg[i] <= 1) q.push(i);
    }
    int left = n;
    while (left > 2){
        int sz = q.size();
        left -= sz;
        while (sz--){
            int v = q.front();
            q.pop();
            for (auto u : graph[v]){
                if (--deg[u] == 1) q.push(u);
            }
        }
    }
    vector<int> ret;
    while (!q.empty()){
        ret.push_back(q.front());
        q.pop();
    }
    return ret;
}
vector<int> tree_hash(vector<vector<int>> &graph){
    vector<int> ret;
    for (auto c : centers(graph)){
        ret.push_back(rooted_hash(c, 0, graph));
    }
    sort(ret.begin(), ret.end());
    return ret;
}
bool isomorphic(vector<vector<int>> &a, vector<vector<int>> &b){
    if (a.size() != b.size()) return false;
    return tree_hash(a) == tree_hash(b);
}

```

Virtual Tree.cpp

Use: Build LCA on the 0-indexed tree, then VirtualTree vt(nodes,lca) fills vt.nodes and weighted vt.edges.

Care: Input nodes may be unsorted; constructor adds needed LCAs, sorts by tin, and stores edges as parent-child-distance triples.

Complexity: Build is $O(k \log k)$ for k marked nodes after LCA preprocessing.

When: Use it when DP only cares about a subset of important nodes and their connecting LCAs.

```

struct LCA{
    int n, Log;
    vector<vector<int>> up;
    vector<int> depth, tin, tout;
    LCA(vector<vector<int>> &g, int root = 0)
        : n((int)g.size()), Log(__lg(n) + 1), up(n, vector<int>(Log, root)), depth(n),
          tin(n), tout(n) {
        int timer = 0;
        function<void(int)> dfs = [&](int u){
            tin[u] = ++timer;
            for (int v : g[u]){
                if (v == up[u][0]) continue;
                depth[v] = depth[u] + 1;
                up[v][0] = u;
                for (int l = 1; l < Log; l++){
                    up[v][l] = up[up[v][l - 1]][l - 1];
                }
            }
        };
        dfs(root);
        tout[u] = timer;
    }
};

```

```

    }
    dfs(v);
}
tout[u] = timer;
};
dfs(root);
}
int kth(int u, int k){
    while (k){
        u = up[u][__builtin_ctz(k)];
        k &= k - 1;
    }
    return u;
}
bool isAnc(int u, int v){
    return tin[u] <= tin[v] && tout[v] <= tout[u];
}
int lca(int u, int v){
    if (depth[u] < depth[v]) swap(u, v);
    u = kth(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int l = Log - 1; l >= 0; l--){
        if (up[u][l] != up[v][l]){
            u = up[u][l];
            v = up[v][l];
        }
    }
    return up[u][0];
}
int dist(int u, int v){
    int l = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[l];
}
};

struct VirtualTree{
    vector<int> nodes;
    vector<tuple<int, int, int>> edges; // u, v, distance
    VirtualTree(vector<int> subset, LCA &lca){
        if (subset.empty()) return;
        auto cmp = [&](int u, int v){
            return lca.tin[u] < lca.tin[v];
        };
        sort(subset.begin(), subset.end(), cmp);
        int k = subset.size();
        for (int i = 0; i + 1 < k; i++){
            subset.push_back(lca.lca(subset[i], subset[i + 1]));
        }
        sort(subset.begin(), subset.end(), cmp);
        subset.erase(unique(subset.begin(), subset.end()), subset.end());
        nodes = subset;
        vector<int> st;
        st.push_back(subset[0]);
        for (int i = 1; i < (int)subset.size(); i++){
            int u = subset[i];
            while (!st.empty() && !lca.isAnc(st.back(), u)) st.pop_back();
            edges.push_back({st.back(), u, lca.dist(st.back(), u)});
            st.push_back(u);
        }
    }
};

```

}
};

4 Math temps

modnum helper

Use: Shared Mint / modnum helper; templates marked with Needs use this MOD/mint code before their snippet.

Before: Change MOD and the mint alias once here; all dependent snippets should use the same modulus.

Pitfall: inv() uses Fermat, so MOD must be prime unless you replace the inverse logic.

```
template<int MOD> struct modnum {
    int v;
    modnum(ll x = 0) : v(x % MOD) { if(v < 0) v += MOD; }
    explicit operator int() const { return v; }
    friend ostream& operator<<(ostream& out, const modnum& n) { return out << n.v; }
    friend istream& operator>>(istream& in, modnum& n) { ll x; in >> x; n = modnum(x);
        ↪ return in; }
    friend bool operator==(const modnum& a, const modnum& b) { return a.v == b.v; }
    friend bool operator!=(const modnum& a, const modnum& b) { return a.v != b.v; }
    modnum pow(ll b) const {
        modnum a = *this, r = 1; while(b) { if(b & 1) r *= a; a *= a; b >>= 1; }
        return r;
    }
    modnum inv() const { return pow(MOD - 2); }
    modnum operator-() const { return modnum(v ? MOD - v : 0); }
    modnum& operator+=(const modnum& o) { v += o.v; if(v >= MOD) v -= MOD; return *this; }
    modnum& operator-=(const modnum& o) { v -= o.v; if(v < 0) v += MOD; return *this; }
    modnum& operator*=(const modnum& o) { v = int(ll(v) * o.v % MOD); return *this; }
    modnum& operator/=(const modnum& o) { return *this *= o.inv(); }
    friend modnum operator+(modnum a, const modnum& b) { return a += b; }
    friend modnum operator-(modnum a, const modnum& b) { return a -= b; }
    friend modnum operator*(modnum a, const modnum& b) { return a *= b; }
    friend modnum operator/(modnum a, const modnum& b) { return a /= b; }
};
```

2D NTT.cpp

Use: conv2d(a,b) pads both dimensions and returns the 2D convolution modulo 998244353; inputs may be rectangular.

Care: The modulus/root are fixed to 998244353 and 3; the 1D dft/idft helpers cache roots and bit-reversal.

Complexity: Build/evaluate cost is $O(H*W*(\log H + \log W))$, where H,W are padded powers of two.

Pitfall: Both dimensions must fit the NTT modulus limit; negative inputs are normalized modulo MOD before transforming.

```
const int MOD = 998244353;
const int root = 3;
using Mat = vector<vector<int>>>;

int mod_norm(ll x){ x %= MOD; if (x < 0) x += MOD; return x; }
int addmod(int a, int b){ a += b; if (a >= MOD) a -= MOD; return a; }
int submod(int a, int b){ a -= b; if (a < 0) a += MOD; return a; }
int mulmod(int a, int b){ return 1LL * a * b % MOD; }
int fp(int b, ll p){
    int res = 1;
    while (p){
        if (p & 1) res = mulmod(res, b);
        b = mulmod(b, b);
        p >>= 1;
    }
    return res;
}
```

```
int inv(int x){ return fp(x, MOD - 2); }
bool is_power_two(int n){ return n > 0 && (n & (n - 1)) == 0; }
int next_power_two(int n){
    int p = 1;
    while (p < n) p <<= 1;
    return p;
}
vector<int> roots{0, 1};
void prepare_roots(int n){
    if ((int)roots.size() >= n) return;
    int sz = roots.size();
    roots.resize(n);
    for (int len = sz; len < n; len <<= 1){
        int w = fp(root, (MOD - 1) / (len << 1));
        for (int i = len / 2; i < len; i++){
            roots[2 * i] = roots[i];
            roots[2 * i + 1] = mulmod(roots[i], w);
        }
    }
}
void ntt(vector<int> &a){
    int n = a.size();
    if (n <= 1) return;
    assert(is_power_two(n));
    static vector<int> revs;
    if ((int)revs.size() != n){
        int lg = __builtin_ctz(n);
        revs.assign(n, 0);
        for (int i = 0; i < n; i++){
            revs[i] = (revs[i >> 1] >> 1) | ((i & 1) << (lg - 1));
        }
    }
    prepare_roots(n);
    for (int i = 0; i < n; i++){
        if (i < revs[i]) swap(a[i], a[revs[i]]);
    }
    for (int len = 1; len < n; len <<= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                int u = a[i + j], v = mulmod(a[i + j + len], roots[len + j]);
                a[i + j] = addmod(u, v);
                a[i + j + len] = submod(u, v);
            }
        }
    }
}
void intt(vector<int> &a){
    if ((int)a.size() <= 1) return;
    reverse(a.begin() + 1, a.end());
    ntt(a);
    int iv = inv(a.size());
    for (auto &x : a) x = mulmod(x, iv);
}
void dft(vector<int> &a){
    for (int &x : a) x = mod_norm(x);
    ntt(a);
}
void idft(vector<int> &a){
    intt(a);
```

```

}
void ntt2d(Mat &a, bool inverse = false){
    int n = a.size(), m = a[0].size();
    assert(is_power_two(n) && is_power_two(m));
    assert(n <= (1 << 23) && m <= (1 << 23));
    for (int i = 0; i < n; i++){
        if (inverse) idft(a[i]);
        else dft(a[i]);
    }
    vector<int> col(n);
    for (int j = 0; j < m; j++){
        for (int i = 0; i < n; i++) col[i] = a[i][j];
        if (inverse) idft(col);
        else dft(col);
        for (int i = 0; i < n; i++) a[i][j] = col[i];
    }
}
Mat conv2d(const Mat &a, const Mat &b){
    if (a.empty() || b.empty() || a[0].empty() || b[0].empty()) return {};
    int n = a.size(), m = a[0].size();
    int r = b.size(), c = b[0].size();
    for (int i = 0; i < n; i++) assert((int)a[i].size() == m);
    for (int i = 0; i < r; i++) assert((int)b[i].size() == c);
    int H = next_power_two(n + r - 1);
    int W = next_power_two(m + c - 1);
    Mat A(H, vector<int>(W)), B(H, vector<int>(W));
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++){
            A[i][j] = mod_norm(a[i][j]);
        }
    }
    for (int i = 0; i < r; i++){
        for (int j = 0; j < c; j++){
            B[i][j] = mod_norm(b[i][j]);
        }
    }
    ntt2d(A);
    ntt2d(B);
    for (int i = 0; i < H; i++){
        for (int j = 0; j < W; j++){
            A[i][j] = mulmod(A[i][j], B[i][j]);
        }
    }
    ntt2d(A, true);
    A.resize(n + r - 1);
    for (auto &row : A) row.resize(m + c - 1);
    return A;
}

```

BerlekampMassey.cpp

Use: BerlekampMassey(S) returns recurrence tr; LinearRecurrence(init,tr,k) returns $S[k]$ using the first tr.size() terms.

Care: Provide enough initial terms, usually at least twice the recurrence degree; guess(S,k,ans) validates before answering.

Complexity: Berlekamp-Massey is $O(n^2)$; LinearRecurrence is $O(d^2 \log k)$ for recurrence degree d.

Pitfall: If the sampled prefix is too short, the guessed recurrence may fit samples but fail later.

Needs: Mint / modnum helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){ if (b & 1) r *= a; a *= a; b >>= 1; }
    return r;
}

vector<mint> BerlekampMassey(vector<mint> S){
    int n = (int)S.size(), L = 0, m = 0;
    vector<mint> C(n), B(n), T;
    C[0] = B[0] = 1;
    mint b = 1;
    for (int i = 0; i < n; i++){
        ++m;
        mint d = S[i];
        for (int j = 1; j <= L; j++) d += C[j] * S[i - j];
        if (d == 0) continue;
        T = C;
        mint coef = d / b;
        for (int j = m; j < n; j++) C[j] -= coef * B[j - m];
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }
    C.resize(L + 1);
    C.erase(C.begin());
    for (auto &x : C) x = -x;
    return C;
}

vector<mint> combine(int n, vector<mint> &a, vector<mint> &b, vector<mint> &tr){
    vector<mint> res(n * 2 + 1, 0);
    for (int i = 0; i < n + 1; i++){
        for (int j = 0; j < n + 1; j++) res[i + j] += a[i] * b[j];
    }
    for (int i = 2 * n; i > n; i--){
        for (int j = 0; j < n; j++) res[i - 1 - j] += res[i] * tr[j];
    }
    res.resize(n + 1);
    return res;
}

mint LinearRecurrence(vector<mint> &S, vector<mint> &tr, ll k){
    int n = S.size();
    assert(n == (int)tr.size());
    if (n == 0) return 0;
    if (k < n) return S[k];
    vector<mint> pol(n + 1), e(n + 1);
    pol[0] = e[1] = 1;
    for (++k; k; k >>= 1){
        if (k & 1) pol = combine(n, pol, e, tr);
        e = combine(n, e, e, tr);
    }
    mint res = 0;
    for (int i = 0; i < n; i++) res += pol[i + 1] * S[i];
    return res;
}

bool guess(vector<mint> S, ll k, mint &ans){

```

```

vector<mint> tr = BerlekampMassey(S);
int d = tr.size();
if (d == 0){
    ans = 0;
    return true;
}
if ((int)S.size() < 2 * d) return false;
for (int i = d; i < (int)S.size(); i++){
    mint cur = 0;
    for (int j = 0; j < d; j++){
        cur += tr[j] * S[i - j - 1];
    }
    if (cur != S[i]) return false;
}
vector<mint> init(S.begin(), S.begin() + d);
ans = LinearRecurrence(init, tr, k);
return true;
}

```

comb.cpp

Use: counting<N> C; call choose(n,k), stars_and_bars(n,k), P(n,k), catalan(n), fact, inv, and invfact under mint MOD.

Care: Set MOD and N before copying; all n used in combinations/catalan must be less than N.

Complexity: Precomputation is O(N), while choose, P, and catalan queries are O(1).

Pitfall: stars_and_bars(n,k) assumes positive boxes; handle k=0 separately if needed.

Needs: Mint / modnum helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){ if (b & 1) r *= a; a *= a; b >>= 1; }
    return r;
}

template<int N>
struct counting{
    mint fact[N], inv[N], invfact[N];
    counting(){
        fact[0] = invfact[0] = 1;
        for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i;
        invfact[N - 1] = fact[N - 1].inv();
        for (int i = N - 2; i >= 0; i--) invfact[i] = invfact[i + 1] * (i + 1);
        for (int i = 1; i < N; i++) inv[i] = fact[i - 1] * invfact[i];
    }
    mint choose(int n, int r){
        if (n < 0 || r < 0 || n < r) return 0;
        return fact[n] * invfact[n - r] * invfact[r];
    }
    mint stars_and_bars(int n, int k){
        return choose(n + k - 1, k - 1);
    }
    mint P(int n, int r){
        if (n < 0 || r < 0 || r > n) return 0;
        return fact[n] * invfact[n - r];
    }
}

```

```

mint catalan(int n){
    return fact[2 * n] * invfact[n] * invfact[n + 1];
}
};
const int N = 1e6 + 5;
counting<N> comb;

```

CRT.cpp

Use: CRT({{a,m},...}) returns {x,lcm} for $x = a \bmod m$ or $\{0,-1\}$; mod_inv requires $\gcd(a,m)=1$.

Care: Works for non-coprime moduli using __int128; print results carefully because T is not directly stream-friendly everywhere.

Complexity: Combining equations is $O(\text{number_of_equations} * \log \text{modulus})$.

Out: intersect(a1,d1,a2,d2) returns the first common value at least $\max(a1,a2)$ and the new period.

```

using T = __int128;

T norm(T a, T m){
    a %= m;
    if (a < 0) a += m;
    return a;
}

T exgcd(T a, T b, T &x, T &y){
    T xx = y = 0;
    T yy = x = 1;
    while (b){
        T q = a / b, t = b;
        b = a % b; a = t;
        t = xx;
        xx = x - q * xx;
        x = t; t = yy;
        yy = y - q * yy;
        y = t;
    }
    return a;
}

pair<T, T> CRT(T a1, T m1, T a2, T m2){
    a1 = norm(a1, m1);
    a2 = norm(a2, m2);
    T p, q;
    T g = exgcd(m1, m2, p, q);
    if (a1 % g != a2 % g) return {0, -1};
    T m = m1 / g * m2;
    p = norm(p, m);
    q = norm(q, m);
    T x = (p * a2 % m * (m1 / g) % m + q * a1 % m * (m2 / g) % m) % m;
    return {x, m};
}

pair<T, T> CRT(vector<pair<T, T>> eq){
    pair<T, T> ret = {0, 1};
    for (auto [a, m] : eq){
        ret = CRT(ret.first, ret.second, a, m);
        if (ret.second == -1) return ret;
    }
    return ret;
}

pair<T, T> intersect(T a1, T d1, T a2, T d2){
    auto x = CRT(a1, d1, a2, d2);
    T a = x.first, d = x.second;
}

```

```

    if (d == -1) return {0, 0};
    T st = max(a1, a2);
    if (a < st) a += (st - a + d - 1) / d * d;
    return {a, d};
}

template<typename T>
T mod_inv_in_range(T a, T m){
    T x = a, y = m;
    T vx = 1, vy = 0;
    while (x){
        T k = y / x;
        y %= x;
        vy -= k * vx;
        swap(x, y);
        swap(vx, vy);
    }
    assert(y == 1);
    return vy < 0 ? m + vy : vy;
}

template<typename T>
struct extended_gcd_result{
    T gcd, coeff_a, coeff_b;
};

template<typename T>
extended_gcd_result<T> extended_gcd(T a, T b){
    T x = a, y = b;
    T ax = 1, ay = 0;
    T bx = 0, by = 1;
    while (x){
        T k = y / x;
        y %= x;
        ay -= k * ax;
        by -= k * bx;
        swap(x, y);
        swap(ax, ay);
        swap(bx, by);
    }
    return {y, ay, by};
}

template<typename T>
T mod_inv(T a, T m){
    a %= m;
    if (a < 0) a += m;
    return mod_inv_in_range(a, m);
}

```

Discrete log.cpp

Use: `discrete_log(a,b,mod)` returns the minimum $x \geq 0$ with $a^x = b \bmod \text{mod}$, or -1 if none.

Care: Handles `gcd(a,mod)>1` cases; `unordered_map` memory is $O(\sqrt{\text{mod}})$, so avoid it for many huge calls.

Complexity: Baby-step giant-step uses $O(\sqrt{\text{mod}})$ time and memory.

Pitfall: For non-coprime cases the function reduces the modulus; test $a=0$ or $b=0$ cases separately.

```

ll mul_mod(ll a, ll b, ll mod){
    return (__int128)a * b % mod;
}

ll fp(ll a, ll b, ll mod){
    a %= mod;

```

```

    ll res = 1 % mod;
    while (b){
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}

// returns minimum x >= 0 with a^x = b (mod mod), or -1 if none
ll discrete_log(ll a, ll b, ll mod){
    a %= mod;
    b %= mod;
    if (mod == 1) return 0;
    ll add = 0, k = 1, g;
    while ((g = gcd(a, mod)) > 1){
        if (b == k) return add;
        if (b % g) return -1;
        b /= g;
        mod /= g;
        add++;
        k = mul_mod(k, a / g, mod);
    }
    if (b == k) return add;
    ll n = sqrtl(mod) + 1;
    unordered_map<ll, ll> vals;
    ll cur = b;
    for (ll q = 0; q <= n; q++){
        vals[cur] = q;
        cur = mul_mod(cur, a, mod);
    }
    ll an = fp(a, n, mod);
    cur = k;
    for (ll p = 1; p <= n + 1; p++){
        cur = mul_mod(cur, an, mod);
        auto it = vals.find(cur);
        if (it != vals.end()){
            ll ans = n * p - it->second + add;
            if (ans >= 0) return ans;
        }
    }
    return -1;
}

```

Fast prime sieve.cpp

Use: `sieve(N)` returns all primes $\leq N$ quickly; tune Q/L only if memory or speed needs it.

Care: Designed for large single limits; for small repeated limits, the simpler sieve in `Number_theory` may be easier.

Complexity: Optimized for huge N with wheel/blocking; memory is roughly $O(\sqrt{\text{N}} + \text{block})$.

Out: Returns a vector of primes in increasing order.

```

// credit: min_25, about 0.5s for n = 1e9
vector<int> sieve(const int N, const int Q = 17, const int L = 1 << 15) {
    if (N < 2) return {};
    if (N < 7){
        vector<int> primes;
        for (int p : {2, 3, 5}){
            if (p <= N) primes.push_back(p);
        }
    }

```

```

    return primes;
}
static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
auto approx_prime_count = [] (const int N) -> int {
    return N > 60184 ? N / (log(N) - 1.1)
        : max(1., N / (log(N) - 1.11)) + 1;
};

const int v = sqrt(N), vv = sqrt(v);
vector<bool> isp(v + 1, true);
if (v >= 0) isp[0] = false;
if (v >= 1) isp[1] = false;
for (int i = 2; i <= vv; ++i) if (isp[i]) {
    for (int j = i * i; j <= v; j += i) isp[j] = false;
}

const int rsize = approx_prime_count(N + 30);
vector<int> primes = {2, 3, 5};
int psize = 3;
primes.resize(rsize);

vector<int> sprimes_p;
vector<array<int, 8>> sprimes_pos;
size_t pbeg = 0;
int prod = 1;
for (int p = 7; p <= v; ++p) {
    if (!isp[p]) continue;
    if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
    array<int, 8> pos;
    for (int t = 0; t < 8; ++t) {
        int j = (p <= Q) ? p : p * p;
        while (j % 30 != rs[t]) j += p << 1;
        pos[t] = j / 30;
    }
    sprimes_p.push_back(p);
    sprimes_pos.push_back(pos);
}

vector<unsigned char> pre(prod, 0xFF);
for (size_t pi = 0; pi < pbeg; ++pi) {
    const int p = sprimes_p[pi];
    for (int t = 0; t < 8; ++t) {
        const unsigned char m = ~(1 << t);
        for (int i = sprimes_pos[pi][t]; i < prod; i += p) pre[i] &= m;
    }
}

const int block_size = (L + prod - 1) / prod * prod;
vector<unsigned char> block(block_size);
unsigned char* pblock = block.data();
const int M = (N + 29) / 30;

for (int beg = 0; beg < M; beg += block_size, pblock -= block_size) {
    int end = min(M, beg + block_size);
    for (int i = beg; i < end; i += prod) {
        copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes_p.size(); ++pi) {

```

```

        const int p = sprimes_p[pi];
        for (int t = 0; t < 8; ++t) {
            int i = sprimes_pos[pi][t];
            const unsigned char m = ~(1 << t);
            for (; i < end; i += p) pblock[i] &= m;
            sprimes_pos[pi][t] = i;
        }
    }
    for (int i = beg; i < end; ++i) {
        for (int m = pblock[i]; m > 0; m &= m - 1) {
            primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
        }
    }
}
assert(psize <= rsize);
while (psize > 0 && primes[psize - 1] > N) --psize;
primes.resize(psize);
return primes;
}

```

FFT string_matching.cpp

Use: conv(a,b) multiplies integer vectors; wildcard_matching(s,t) returns starts where pattern t matches text s with '?' wildcards.

Out: wildcard_matching(text,pattern) returns all starts where every non-'?' aligned character matches.

Complexity: Convolutions cost $O(n \log n)$, with three convolutions for wildcard matching.

Pitfall: The matching formula assumes lowercase letters mapped to positive values and '?' as wildcard.

```

using cd = complex<double>;
const double PI = acos(-1);

int next_power_two(int n){
    int p = 1;
    while (p < n) p <= 1;
    return p;
}

vector<cd> fft_roots{0, 1};
void prepare_roots(int n){
    if ((int)fft_roots.size() >= n) return;
    int old_size = fft_roots.size();
    fft_roots.resize(n);
    for (int len = old_size; len < n; len <= 1){
        cd w = polar(1.0, PI / len);
        for (int i = len / 2; i < len; i++){
            fft_roots[2 * i] = fft_roots[i];
            fft_roots[2 * i + 1] = fft_roots[i] * w;
        }
    }
}

void fft(vector<cd> &a){
    int n = a.size();
    if (n <= 1) return;
    assert((n & (n - 1)) == 0);
    static vector<int> cached_rev;
    if ((int)cached_rev.size() != n){
        int lg = __builtin_ctz(n);
        cached_rev.assign(n, 0);
    }
}

```



```

        for (int i = 0; i < n; i++){
            cached_rev[i] = (cached_rev[i >> 1] >> 1) | ((i & 1) << (lg - 1));
        }
    }
    prepare_roots(n);
    for (int i = 0; i < n; i++){
        if (i < cached_rev[i]) swap(a[i], a[cached_rev[i]]);
    }
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                cd z = fft_roots[len + j] * a[i + j + len], &ai = a[i + j];
                a[i + j + len] = ai - z;
                ai += z;
            }
        }
    }
}

void ifft(vector<cd> &a){
    if ((int)a.size() <= 1) return;
    reverse(a.begin() + 1, a.end());
    fft(a);
    for (cd &x : a) x /= (double)a.size();
}

void dft(vector<cd> &a){
    fft(a);
}

void idft(vector<cd> &a){
    ifft(a);
}

vector<ll> conv(const vector<ll> &a, const vector<ll> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int n = next_power_two(s);
    vector<cd> L(n), R(n);
    for (int i = 0; i < (int)a.size(); i++) L[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) R[i] = b[i];
    dft(L);
    dft(R);
    for (int i = 0; i < n; i++) L[i] *= R[i];
    idft(L);
    vector<ll> ret(s);
    for (int i = 0; i < s; i++){
        ret[i] = llround(real(L[i]));
    }
    return ret;
}

// match iff sum s*t*(s-t)^2 over aligned non-'?' chars is 0
vector<int> wildcard_matching(const string &s, const string &t){
    int n = s.size(), m = t.size();
    if (m > n) return {};
    vector<ll> s1(n), s2(n), s3(n), t1(m), t2(m), t3(m);
    for (int i = 0; i < n; i++){
        s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;

```

```

        s2[i] = s1[i] * s1[i];
        s3[i] = s1[i] * s2[i];
    }
    for (int i = 0; i < m; i++){
        t1[i] = t[i] == '?' ? 0 : t[i] - 'a' + 1;
        t2[i] = t1[i] * t1[i];
        t3[i] = t1[i] * t2[i];
    }
    reverse(t1.begin(), t1.end());
    reverse(t2.begin(), t2.end());
    reverse(t3.begin(), t3.end());
    vector<ll> a = conv(s1, t3);
    vector<ll> b = conv(s2, t2);
    vector<ll> c = conv(s3, t1);
    vector<int> ret;
    for (int i = m - 1; i < n; i++){
        if (a[i] - 2 * b[i] + c[i] == 0) ret.push_back(i - m + 1);
    }
    return ret;
}

```

FFT with MOD.cpp

Use: conv<M>(a,b) returns convolution modulo M using FFT splitting; inputs are integer coefficient vectors.

Care: Use this when target modulus is not NTT-friendly; coefficients are reduced modulo template parameter M.

Complexity: Convolution is $O(n \log n)$, with coefficient splitting to control precision under modulus M.

Pitfall: Still uses doubles, so for extreme lengths or coefficients compare with CRT/NTT if suspicious.

```

using C = complex<double>;
using ld = long double;
const ld PI = acosl(-1.0L);
int nxt_pw(int n){
    int p = 1;
    while (p < n) p <= 1;
    return p;
}

vector<complex<ld>> roots_ld{0, 1};
vector<C> roots{0, 1};
void prepare_roots(int n){
    if ((int)roots.size() >= n) return;
    int old_size = roots.size();
    roots_ld.resize(n);
    roots.resize(n);
    for (int len = old_size; len < n; len <= 1){
        auto w = polar(1.0L, PI / len);
        for (int i = len / 2; i < len; i++){
            roots_ld[2 * i] = roots_ld[i];
            roots_ld[2 * i + 1] = roots_ld[i] * w;
            roots[2 * i] = C((double)roots_ld[2 * i].real(), (double)roots_ld[2 * i].imag());
            roots[2 * i + 1] = C((double)roots_ld[2 * i + 1].real(), (double)roots_ld[2 * i + 1].imag());
        }
    }
}

void fft(vector<C> &a){

```

```

int n = a.size();
if (n <= 1) return;
assert((n & (n - 1)) == 0);
static vector<int> revs;
if ((int)revs.size() != n){
    int lg = __builtin_ctz(n);
    revs.assign(n, 0);
    for (int i = 0; i < n; i++){
        revs[i] = (revs[i >> 1] >> 1) | ((i & 1) << (lg - 1));
    }
}
prepare_roots(n);
for (int i = 0; i < n; i++){
    if (i < revs[i]) swap(a[i], a[revs[i]]);
}
for (int len = 1; len < n; len <= 1){
    for (int i = 0; i < n; i += 2 * len){
        for (int j = 0; j < len; j++){
            auto x = (double *)&roots[j + len], y = (double *)&a[i + j + len];
            C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
            a[i + j + len] = a[i + j] - z;
            a[i + j] += z;
        }
    }
}
}
void ifft(vector<C> &a){
    if ((int)a.size() <= 1) return;
    reverse(a.begin() + 1, a.end());
    fft(a);
    for (C &x : a) x /= (double)a.size();
}
template<int M>
vector<int> conv(const vector<int> &a, const vector<int> &b){
    if (a.empty() || b.empty()) return {};
    vector<int> res(a.size() + b.size() - 1);
    int n = nxt_pw(res.size()), cut = sqrt(M);
    vector<C> L(n), R(n), outl(n), outs(n);
    for (int i = 0; i < (int)a.size(); i++){
        ll x = a[i] % M;
        if (x < 0) x += M;
        L[i] = C((int)x / cut, (int)x % cut);
    }
    for (int i = 0; i < (int)b.size(); i++){
        ll x = b[i] % M;
        if (x < 0) x += M;
        R[i] = C((int)x / cut, (int)x % cut);
    }
    fft(L); fft(R);
    C I(0, 1);
    for (int i = 0; i < n; i++){
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / I;
    }
    fft(outl); fft(outs);
    for (int i = 0; i < (int)res.size(); i++){
        ll av = ll(real(outl[i]) + 0.5), cv = ll(imag(outs[i]) + 0.5);
        ll bv = ll(imag(outl[i]) + 0.5) + ll(real(outs[i]) + 0.5);

```

```

        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

```

FFT.cpp

Use: conv(a,b) returns integer convolution of vectors a and b using doubles, best when coefficients/results fit rounding safety.

Care: Rounding is the danger point; split coefficients or use NTT/CRT when products can exceed double precision safety.

Complexity: Convolution is $O(n \log n)$ after padding to the next power of two.

Pitfall: Negative coefficients are fine, but very large absolute values can break llround accuracy.

```

using cd = complex<double>;
const double PI = acos(-1);

int nxt_pw(int n){
    int p = 1;
    while (p < n) p <= 1;
    return p;
}
vector<cd> roots{0, 1};
void prepare_roots(int n){
    if ((int)roots.size() >= n) return;
    int old_size = roots.size();
    roots.resize(n);
    for (int len = old_size; len < n; len <= 1){
        cd w = polar(1.0, PI / len);
        for (int i = len / 2; i < len; i++){
            roots[2 * i] = roots[i];
            roots[2 * i + 1] = roots[i] * w;
        }
    }
}
void fft(vector<cd> &a){
    int n = a.size();
    if (n <= 1) return;
    assert((n & (n - 1)) == 0);
    static vector<int> revs;
    if ((int)revs.size() != n){
        int lg = __builtin_ctz(n);
        revs.assign(n, 0);
        for (int i = 0; i < n; i++){
            revs[i] = (revs[i >> 1] >> 1) | ((i & 1) << (lg - 1));
        }
    }
    prepare_roots(n);
    for (int i = 0; i < n; i++){
        if (i < revs[i]) swap(a[i], a[revs[i]]);
    }
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                cd z = roots[len + j] * a[i + j + len], &ai = a[i + j];
                a[i + j + len] = ai - z;
                ai += z;
            }
        }
    }
}

```

```

    }
}

void ifft(vector<cd> &a){
    if ((int)a.size() <= 1) return;
    reverse(a.begin() + 1, a.end());
    fft(a);
    for (cd &x : a) x /= (double)a.size();
}

vector<ll> conv(const vector<ll> &a, const vector<ll> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int n = nxt_pw(s);
    vector<cd> L(n), R(n);
    for (int i = 0; i < (int)a.size(); i++) L[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) R[i] = b[i];
    fft(L); fft(R);
    for (int i = 0; i < n; i++) L[i] *= R[i];
    ifft(L);
    vector<ll> ret(s);
    for (int i = 0; i < s; i++){
        ret[i] = llround(real(L[i]));
    }
    return ret;
}

```

FWHT.cpp

Use: Vectors must have power-of-two size; transforms mutate in place and convolution helpers return a new vector.

Apps: xor_convolution/and_convolution/or_convolution combine by bitwise operation; subset_convolution combines masks by disjoint submasks.

Care: Transforms mutate arrays in place; pass copies if you still need the original vectors after convolution.

Complexity: xor/and/or transforms are $O(n \log n)$; subset_convolution is $O(n \log^2 n)$ by mask layers.

Pitfall: For modular arithmetic, T must support +, -, *, and division by n for inverse xor.

```

bool is_power_two(int n){
    return n > 0 && (n & (n - 1)) == 0;
}

template<typename T>
void xor_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                T x = a[i + j], y = a[i + j + len];
                a[i + j] = x + y;
                a[i + j + len] = x - y;
            }
        }
    }
    if (inv){
        T iv = T(n);
        for (auto &x : a) x /= iv;
    }
}

template<typename T>

```

```

void and_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                if (inv) a[i + j] -= a[i + j + len];
                else a[i + j] += a[i + j + len];
            }
        }
    }
}

template<typename T>
void or_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                if (inv) a[i + j + len] -= a[i + j];
                else a[i + j + len] += a[i + j];
            }
        }
    }
}

template<typename T>
vector<T> xor_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    xor_transform(a);
    xor_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    xor_transform(a, true);
    return a;
}

template<typename T>
vector<T> and_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    and_transform(a);
    and_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    and_transform(a, true);
    return a;
}

template<typename T>
vector<T> or_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    or_transform(a);
    or_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    or_transform(a, true);
    return a;
}

template<typename T>
vector<T> subset_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size() && is_power_two(a.size()));
    int n = a.size(), lg = __builtin_ctz(n);
    vector<vector<T>> A(lg + 1, vector<T>(n)), B(lg + 1, vector<T>(n));
    for (int mask = 0; mask < n; mask++){
        A[__builtin_popcount(mask)][mask] = a[mask];

```

```

    B[__builtin_popcount(mask)][mask] = b[mask];
}
for (int i = 0; i <= lg; i++){
    or_transform(A[i]);
    or_transform(B[i]);
}
vector<T> tmp(n), ret(n);
for (int k = 0; k <= lg; k++){
    fill(tmp.begin(), tmp.end(), T(0));
    for (int i = 0; i <= k; i++){
        for (int mask = 0; mask < n; mask++){
            tmp[mask] += A[i][mask] * B[k - i][mask];
        }
    }
    or_transform(tmp, true);
    for (int mask = 0; mask < n; mask++){
        if (__builtin_popcount(mask) == k) ret[mask] = tmp[mask];
    }
}
return ret;
}

```

Gaussian with Doubles.cpp

Use: gauss(a,ans) solves linear equations over doubles and returns 0 none, 1 unique, or INF many solutions.

Care: Matrix a is augmented with b as the last column; tune EPS for geometry or near-singular systems.

Complexity: Gaussian elimination is $O(\text{rows} * \text{cols}^2)$ for an augmented matrix.

Pitfall: Floating comparisons depend on EPS; too small or too large EPS changes rank decisions.

```

const double EPS = 1e-9;
const int INF = 2;
int gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = a.size(), m = (int)a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int sel = row;
        for (int i = row; i < n; i++) {
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        }
        if (abs(a[sel][col]) < EPS) continue;
        for (int j = col; j <= m; j++) swap(a[sel][j], a[row][j]);
        where[col] = row;

        for (int i = 0; i < n; i++) {
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; j++) {
                    a[i][j] -= a[row][j] * c;
                }
            }
        }
        row++;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++) {
        if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    }
    for (int i = 0; i < n; i++) {
        double sum = 0;

```

```

        for (int j = 0; j < m; j++) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > EPS) return 0;
    }
    for (int i = 0; i < m; i++) {
        if (where[i] == -1) return INF;
    }
    return 1;
}

```

Gaussian with MOD.cpp

Use: Set MOD; Gauss(a,ans) solves $Ax=b$, determinant(a) returns det, and matrix_rank(a) returns rank modulo MOD.

Apps: Kirchhoff helpers count weighted spanning trees, rooted forests, and directed arborescences using Laplacian cofactors.

Out: Gauss returns -1 for no solution, otherwise the number of free variables and one solution in ans.

Kirchhoff: Use 0-indexed edges {u,v,w}; undirected multiedges are allowed, and weights are accumulated modulo MOD.

Complexity: Gauss, determinant, and matrix_rank are $O(n*m*\min(n,m))$ style elimination.

Pitfall: MOD should be prime because inverses use binpow(x, MOD-2).

Matrix Tree: count_spanning_trees removes one row/column; count_rooted_forests removes all roots and counts forests with one root per component.

Directed: count_arborescences_to_root follows edge directions into root; count_arborescences_from_root counts trees reachable from root.

```

const ll MOD = 1e9 + 7;
int norm(ll x){
    x %= MOD;
    if (x < 0) x += MOD;
    return x;
}
int norm(int x){
    return norm((ll)x);
}
ll binpow(ll a, ll b){
    ll res = 1;
    while(b){
        if(b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}
int Gauss(vector<vector<int>> a, vector<int> &ans){
    int n = a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    int free_var = 0;
    const ll MODSQ = (ll)MOD * MOD;
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a[k][col] > a[mx][col]) mx = k;
        if (a[mx][col] == 0) continue;
        for (int j = col; j <= m; j++) swap(a[mx][j], a[row][j]);
        pos[col] = row;
        int inv = binpow(a[row][col], MOD - 2);
        for (int i = 0; i < n && inv; i++){
            if (i != row && a[i][col]) {
                int x = ((ll)a[i][col] * inv) % MOD;
                for (int j = col; j <= m && x; j++){

```

```

        if (a[row][j]) a[i][j] = (MODSQ + a[i][j] - ((1LL)a[row][j] * x)) %
        ↪ MOD;
    }
}
row++;
}
ans.assign(m, 0);
for (int i = 0; i < m; i++){
    if (pos[i] == -1) free_var++;
    else ans[i] = ((1LL)a[pos[i]][m] * binpow(a[pos[i]][i], MOD - 2)) % MOD;
}
for (int i = 0; i < n; i++) {
    11 val = 0;
    for (int j = 0; j < m; j++) val = (val + ((1LL)ans[j] * a[i][j])) % MOD;
    if (val != a[i][m]) return -1; //no solution
}
return free_var; //has solution
}
int determinant(vector<vector<int>> a){
    int n = a.size();
    if (n == 0) return 1;
    assert((int)a[0].size() == n);
    int det = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++) a[i][j] = norm(a[i][j]);
    }
    for (int col = 0, row = 0; col < n; col++, row++){
        int sel = row;
        while (sel < n && a[sel][col] == 0) sel++;
        if (sel == n) return 0;
        if (sel != row){
            swap(a[sel], a[row]);
            det = norm(-det);
        }
        det = 1LL * det * a[row][col] % MOD;
        int inv = binpow(a[row][col], MOD - 2);
        for (int i = row + 1; i < n; i++){
            int c = 1LL * a[i][col] * inv % MOD;
            for (int j = col; j < n; j++){
                a[i][j] = norm(a[i][j] - 1LL * a[row][j] * c);
            }
        }
    }
    return det;
}
int matrix_rank(vector<vector<int>> a){
    int n = a.size();
    if (n == 0) return 0;
    int m = a[0].size(), rank = 0;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++) a[i][j] = norm(a[i][j]);
    }
    for (int col = 0, row = 0; col < m && row < n; col++){
        int sel = row;
        while (sel < n && a[sel][col] == 0) sel++;
        if (sel == n) continue;
        swap(a[sel], a[row]);
        int inv = binpow(a[row][col], MOD - 2);

```

```

        for (int i = row + 1; i < n; i++){
            int c = 1LL * a[i][col] * inv % MOD;
            for (int j = col; j < m; j++){
                a[i][j] = norm(a[i][j] - 1LL * a[row][j] * c);
            }
        }
        row++;
        rank++;
    }
    return rank;
}
vector<vector<int>> erase_rows_cols(const vector<vector<int>> &a, vector<int> rem){
    int n = a.size();
    vector<int> bad(n, 0);
    for(int x : rem){
        assert(0 <= x && x < n);
        bad[x] = 1;
    }
    vector<vector<int>> b;
    for(int i = 0; i < n; i++){
        if(bad[i]) continue;
        vector<int> row;
        for(int j = 0; j < n; j++){
            if(!bad[j]) row.push_back(a[i][j]);
        }
        b.push_back(row);
    }
    return b;
}
int cofactor_det(const vector<vector<int>> &lap, vector<int> rem){
    return determinant(erase_rows_cols(lap, rem));
}
void add_undirected_edge(vector<vector<int>> &lap, int u, int v, int w = 1){
    int n = lap.size();
    assert(0 <= u && u < n && 0 <= v && v < n);
    if(u == v) return;
    w = norm(w);
    lap[u][u] = norm(lap[u][u] + w);
    lap[v][v] = norm(lap[v][v] + w);
    lap[u][v] = norm(lap[u][v] - w);
    lap[v][u] = norm(lap[v][u] - w);
}
vector<vector<int>> undirected_laplacian(int n, const vector<array<int, 3>> &edges){
    vector<vector<int>> lap(n, vector<int>(n, 0));
    for(auto [u, v, w] : edges) add_undirected_edge(lap, u, v, w);
    return lap;
}
int count_spanning_trees(int n, const vector<array<int, 3>> &edges){
    if(n == 0) return 0;
    auto lap = undirected_laplacian(n, edges);
    return cofactor_det(lap, {n - 1});
}
int count_spanning_trees(int n, const vector<pair<int, int>> &edges){
    vector<array<int, 3>> e;
    e.reserve(edges.size());
    for(auto [u, v] : edges) e.push_back({u, v, 1});
    return count_spanning_trees(n, e);
}
int count_rooted_forests(int n, const vector<array<int, 3>> &edges, vector<int> roots){

```

```

    auto lap = undirected_laplacian(n, edges);
    return cofactor_det(lap, roots);
}

void add_directed_to_root_edge(vector<vector<int>> &lap, int u, int v, int w = 1){
    int n = lap.size();
    assert(0 <= u && u < n && 0 <= v && v < n);
    if(u == v) return;
    w = norm(w);
    lap[u][u] = norm(lap[u][u] + w);
    lap[u][v] = norm(lap[u][v] - w);
}

void add_directed_from_root_edge(vector<vector<int>> &lap, int u, int v, int w = 1){
    int n = lap.size();
    assert(0 <= u && u < n && 0 <= v && v < n);
    if(u == v) return;
    w = norm(w);
    lap[v][v] = norm(lap[v][v] + w);
    lap[v][u] = norm(lap[v][u] - w);
}

vector<vector<int>> directed_laplacian_to_root(int n, const vector<array<int, 3>> &edges){
    vector<vector<int>> lap(n, vector<int>(n, 0));
    for(auto [u, v, w] : edges) add_directed_to_root_edge(lap, u, v, w);
    return lap;
}

vector<vector<int>> directed_laplacian_from_root(int n, const vector<array<int, 3>>
↳ &edges){
    vector<vector<int>> lap(n, vector<int>(n, 0));
    for(auto [u, v, w] : edges) add_directed_from_root_edge(lap, u, v, w);
    return lap;
}

int count_arborescences_to_root(int n, const vector<array<int, 3>> &edges, int root){
    auto lap = directed_laplacian_to_root(n, edges);
    return cofactor_det(lap, {root});
}

int count_arborescences_from_root(int n, const vector<array<int, 3>> &edges, int root){
    auto lap = directed_laplacian_from_root(n, edges);
    return cofactor_det(lap, {root});
}

int count_directed_forests_to_roots(int n, const vector<array<int, 3>> &edges,
↳ vector<int> roots){
    auto lap = directed_laplacian_to_root(n, edges);
    return cofactor_det(lap, roots);
}

```

Kth root modulo.cpp

Use: kth_root_mod(a,k,p) returns one x with $x^k = a \bmod \text{prime } p$, or -1; kth_roots_mod returns all roots sorted.

Care: This version assumes p is prime; for $a=0$ the only returned root is 0, and many roots are possible when $\gcd(k,p-1) > 1$.

Complexity: primitive_root factors $p-1$ by trial division; discrete log is $O(\sqrt{p})$ memory/time for one query.

Before: Use only when the modulus is prime. For many queries under the same p , cache the primitive root or adapt the file.

Out: kth_roots_mod solves $k \cdot z = \log_g(a) \bmod p-1$, then maps exponents back by g^z and sorts unique roots.

```

ll normalize(ll x, ll mod){
    x %= mod;
    if (x < 0) x += mod;

```

```

    return x;
}

ll mult(ll a, ll b, ll mod){
    return (__int128)normalize(a, mod) * normalize(b, mod) % mod;
}

ll fp(ll a, ll b, ll mod){
    a = normalize(a, mod);
    ll res = 1 % mod;
    while (b){
        if (b & 1) res = mult(res, a, mod);
        a = mult(a, a, mod);
        b >>= 1;
    }
    return res;
}

ll exgcd(ll a, ll b, ll &x, ll &y){
    if (b == 0){
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}

ll inv_mod(ll a, ll mod){
    ll x, y;
    ll g = exgcd(a, mod, x, y);
    assert(abs(g) == 1);
    return normalize(x, mod);
}

// assumes p is prime
ll primitive_root(ll p){
    if (p == 2) return 1;
    ll phi = p - 1, x = phi;
    vector<ll> fact;
    for (ll d = 2; d <= x / d; d++){
        if (x % d == 0){
            fact.push_back(d);
            while (x % d == 0) x /= d;
        }
    }
    if (x > 1) fact.push_back(x);
    for (ll g = 2; ; g++){
        bool ok = true;
        for (ll q : fact){
            if (fp(g, phi / q, p) == 1){
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
}

```

// assumes $\gcd(a, \text{mod}) = 1$; returns minimum $x \geq 0$ with $a^x = b \bmod \text{mod}$, or -1

```

11 discrete_log_coprime(11 a, 11 b, 11 mod){
    a = normalize(a, mod);
    b = normalize(b, mod);
    if (mod == 1) return 0;
    11 n = sqrtl(mod - 1) + 1;
    unordered_map<11, 11> baby;
    baby.reserve(n * 2 + 1);
    11 cur = 1 % mod;
    for (11 j = 0; j < n; j++){
        if (!baby.count(cur)) baby[cur] = j;
        cur = mult(cur, a, mod);
    }
    11 factor = inv_mod(fp(a, n, mod), mod);
    cur = b;
    for (11 i = 0; i <= n; i++){
        auto it = baby.find(cur);
        if (it != baby.end()){
            11 ans = i * n + it->second;
            if (ans < mod) return ans;
        }
        cur = mult(cur, factor, mod);
    }
    return -1;
}

// returns all x with x^k = a mod prime p; may be large if gcd(k, p - 1) is large
vector<11> kth_roots_mod(11 a, 11 k, 11 p){
    assert(k >= 1 && p >= 2);
    a = normalize(a, p);
    if (p == 2) return {a};
    if (a == 0) return {0};
    11 g = primitive_root(p);
    11 y = discrete_log_coprime(g, a, p);
    if (y == -1) return {};
    11 order = p - 1, d = gcd(k, order);
    if (y % d != 0) return {};
    11 kk = k / d, yy = y / d, mod = order / d;
    11 z = 0;
    if (mod > 1) z = mult(yy, inv_mod(kk % mod, mod), mod);
    vector<11> roots;
    roots.reserve(d);
    for (11 i = 0; i < d; i++){
        roots.push_back(fp(g, z + i * mod, p));
    }
    sort(roots.begin(), roots.end());
    roots.erase(unique(roots.begin(), roots.end()), roots.end());
    return roots;
}

// returns one root, or -1 if no root exists
11 kth_root_mod(11 a, 11 k, 11 p){
    auto roots = kth_roots_mod(a, k, p);
    return roots.empty() ? -1 : roots[0];
}

```

lagrange interpolation.cpp

Use: interpolate(y,x,first,step) evaluates equally spaced points, interpolate_brute(points,x) arbitrary points, sum_pows(n,k) gives sum i^k .

Care: interpolate(y,x,first,step) assumes $y[i]=f(\text{first}+i*\text{step})$; step must be nonzero modulo MOD.

Complexity: Equally spaced interpolation is $O(n)$; brute interpolation is $O(n^2)$.

When: Use sum_pows when you need $1^k + \dots + n^k$ for huge n under MOD.

Needs: Mint / modnum helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, 11 b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

// y[i] = f(first + i * step)
mint interpolate(const vector<mint> &y, 11 x, mint first = 0, mint step = 1){
    int n = y.size() - 1;
    assert(n >= 0 && step != 0);
    if (first == 0 && step == 1 && 0 <= x && x <= n) return y[x];
    vector<mint> pref(n + 1, 1), suff(n + 1, 1), fact(n + 1, 1), finv(n + 1, 1);
    for (int i = 1; i <= n; i++){
        pref[i] = pref[i - 1] * (mint(x) - (first + mint(i - 1) * step));
        fact[i] = fact[i - 1] * step * i;
    }
    finv[n] = fact[n].inv();
    for (int i = n - 1; i >= 0; i--){
        suff[i] = suff[i + 1] * (mint(x) - (first + mint(i + 1) * step));
        finv[i] = finv[i + 1] * (i + 1) * step;
    }
    mint ans = 0;
    for (int i = 0; i <= n; i++){
        mint cur = y[i] * pref[i] * suff[i] * finv[i] * finv[n - i];
        if ((n - i) & 1) ans -= cur;
        else ans += cur;
    }
    return ans;
}

mint interpolate_brute(const vector<pair<mint, mint>> &p, mint x){
    int n = p.size();
    mint ans = 0;
    for (int i = 0; i < n; i++){
        mint num = 1, den = 1;
        for (int j = 0; j < n; j++){
            if (i == j) continue;
            num *= x - p[j].first;
            den *= p[i].first - p[j].first;
        }
        ans += p[i].second * num / den;
    }
    return ans;
}

mint sum_pows(11 n, int k){
    vector<mint> y(k + 2);
    for (int i = 1; i <= k + 1; i++){
        y[i] = y[i - 1] + binpow(i, k);
    }
}

```

```

    return interpolate(y, n);
}

```

matrix_expo.cpp

Use: `Mat<mint> A(r,c)`; set `A.a`, then `mul(A,B)` or `matpow(A,e)` for matrix multiplication/exponentiation.

Care: `matpow` requires a square matrix; `mul_vec` applies a matrix to a state vector after exponentiation.

Complexity: `matpow` is $O(\dim^3 \log \text{exponent})$ with the provided dense multiplication.

Pitfall: Use sparse/custom multiplication for very large states; this helper is for small to medium matrices.

Needs: `Mint` / `modnum` helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

template<typename T>
struct Mat{
    int r, c;
    vector<vector<T>>> a;
    Mat(int r = 0, int c = 0) : r(r), c(c), a(r, vector<T>(c)) {}
    static Mat iden(int n){
        Mat res(n, n);
        for (int i = 0; i < n; i++) res.a[i][i] = 1;
        return res;
    }
    vector<T> mul_vec(const vector<T> &v) const {
        assert(c == (int)v.size());
        vector<T> res(r);
        for (int i = 0; i < r; i++){
            for (int j = 0; j < c; j++){
                res[i] += a[i][j] * v[j];
            }
        }
        return res;
    }
};

template<typename T>
Mat<T> mul(const Mat<T> &a, const Mat<T> &b){
    assert(a.c == b.r);
    Mat<T> res(a.r, b.c);
    for (int i = 0; i < a.r; i++){
        for (int k = 0; k < a.c; k++){
            if (a.a[i][k] == T(0)) continue;
            for (int j = 0; j < b.c; j++){
                res.a[i][j] += a.a[i][k] * b.a[k][j];
            }
        }
    }
    return res;
}

```

```

template<typename T>
Mat<T> matpow(Mat<T> a, ll b){
    assert(a.r == a.c && b >= 0);
    Mat<T> res = Mat<T>::iden(a.r);
    while (b){
        if (b & 1) res = mul(res, a);
        a = mul(a, a);
        b >>= 1;
    }
    return res;
}

```

NTT.cpp

Use: `conv(a,b)` pads and multiplies modulo 998244353; `dft(a)/idft(a)` mutate power-of-two arrays, and all `poly_*` return trimmed prefixes.

Apps: `poly_inv` needs `F[0]!=0`; `poly_ln` needs `F[0]=1`; `poly_exp` needs `A[0]=0`; `poly_pow/pow2` exponentiate; `poly_sqrt` returns empty if no sqrt.

Care: `dft` normalizes inputs and `idft` performs the reverse/scale step; raw `dft/idft` still require power-of-two length.

Complexity: `dft/idft` are $O(n \log n)$; `conv` is $O(s \log s)$ after padding `s=a.size()+b.size()-1` to a power of two.

Before: Call `conv` for normal multiplication; use `dft/idft` only when you manually keep vectors already padded to power-of-two length.

Pitfall: Roots and bit-reversal are cached; modulus/root are fixed to 998244353 and 3 unless you rebuild the whole setup.

```

const int MOD = 998244353;
const int root = 3;

int normalize(ll x){ x %= MOD; if (x < 0) x += MOD; return x; }
int add(int a, int b){ a += b; if (a >= MOD) a -= MOD; return a; }
int sub(int a, int b){ a -= b; if (a < 0) a += MOD; return a; }
int mult(int a, int b){ return 1LL * a * b % MOD; }
int fp(int b, ll p){
    int res = 1;
    while (p){
        if (p & 1) res = mult(res, b);
        b = mult(b, b);
        p >>= 1;
    }
    return res;
}

int inv(int x){ return fp(x, MOD - 2); }
int nxt_pw(int n){
    int p = 1;
    while (p < n) p <<= 1;
    return p;
}

vector<int> roots{0, 1};
void prepare_roots(int n){
    if ((int)roots.size() >= n) return;
    int sz = roots.size();
    roots.resize(n);
    for (int len = sz; len < n; len <<= 1){
        int w = fp(root, (MOD - 1) / (len << 1));
        for (int i = len / 2; i < len; i++){
            roots[2 * i] = roots[i];
            roots[2 * i + 1] = mult(roots[i], w);
        }
    }
}

```



```

    }
}

void ntt(vector<int> &a){
    int n = a.size();
    if (n <= 1) return;
    assert((n & (n - 1)) == 0);
    static vector<int> revs;
    if ((int)revs.size() != n){
        int lg = __builtin_ctz(n);
        revs.assign(n, 0);
        for(int i = 0; i < n; i++){
            revs[i] = (revs[i >> 1] >> 1) | ((i & 1) << (lg - 1));
        }
    }
    prepare_roots(n);
    for(int i = 0; i < n; i++) if (i < revs[i]) swap(a[i], a[revs[i]]);
    for(int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                int u = a[i + j], v = mult(a[i + j + len], roots[len + j]);
                a[i + j] = add(u, v);
                a[i + j + len] = sub(u, v);
            }
        }
    }
}

void intt(vector<int> &a){
    if ((int)a.size() <= 1) return;
    reverse(a.begin() + 1, a.end());
    ntt(a);
    int iv = inv(a.size());
    for(int &x : a) x = mult(x, iv);
}

void dft(vector<int> &a){
    for(int &x : a) x = normalize(x);
    ntt(a);
}

void idft(vector<int> &a){
    intt(a);
}

vector<int> conv(const vector<int> &a, const vector<int> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int n = nxt_pw(s);
    vector<int> L(a), R(b);
    L.resize(n); R.resize(n);
    dft(L); dft(R);
    for (int i = 0; i < n; i++) L[i] = mult(L[i], R[i]);
    idft(L);
    return {L.begin(), L.begin() + s};
}

vector<int> poly_trim(const vector<int> &a, int sz){
    int n = min<int>(a.size(), sz);
    return vector<int>(a.begin(), a.begin() + n);
}

vector<int> poly_add(const vector<int> &a, const vector<int> &b, int sz = -1){
    if (sz == -1) sz = max((int)a.size(), (int)b.size());

```

```

    vector<int> r(sz);
    for (int i = 0; i < sz; i++){
        if (i < (int)a.size()) r[i] = add(r[i], normalize(a[i]));
        if (i < (int)b.size()) r[i] = add(r[i], normalize(b[i]));
    }
    return r;
}

vector<int> poly_sub(const vector<int> &a, const vector<int> &b, int sz = -1){
    if (sz == -1) sz = max((int)a.size(), (int)b.size());
    vector<int> r(sz);
    for (int i = 0; i < sz; i++){
        if (i < (int)a.size()) r[i] = add(r[i], normalize(a[i]));
        if (i < (int)b.size()) r[i] = sub(r[i], normalize(b[i]));
    }
    return r;
}

vector<int> poly_scalar_mul(vector<int> a, int scalar){
    scalar = normalize(scalar);
    for (auto &x : a) x = mult(x, scalar);
    return a;
}

vector<int> poly_derivative(const vector<int> &a){
    if (a.size() <= 1) return vector<int>{0};
    vector<int> d(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++) d[i - 1] = mult(normalize(a[i]), i);
    return d;
}

vector<int> poly_integral(const vector<int> &a){
    int n = a.size();
    vector<int> res(n + 1);
    for (int i = 0; i < n; i++) res[i + 1] = mult(normalize(a[i]), inv(i + 1));
    return res;
}

vector<int> poly_inv(const vector<int> &F, int m){
    assert(!F.empty() && normalize(F[0]) != 0);
    vector<int> G(1, inv(normalize(F[0])));
    int cur = 1;
    while (cur < m){
        cur <<= 1;
        vector<int> Fcut = poly_trim(F, cur);
        Fcut.resize(cur);
        vector<int> GG = conv(conv(G, G), Fcut);
        G.resize(cur);
        for (int i = 0; i < cur; i++){
            int u = i < (int)GG.size() ? GG[i] : 0;
            G[i] = sub(add(G[i], G[i]), u);
        }
    }
    G.resize(m);
    return G;
}

vector<int> poly_ln(const vector<int> &F, int n){
    assert(!F.empty() && normalize(F[0]) == 1);
    vector<int> prod = conv(poly_derivative(F), poly_inv(F, n));
    prod.resize(max(0, n - 1));
    vector<int> ret = poly_integral(prod);
    ret.resize(n);
    return ret;
}

```

```

vector<int> poly_exp(const vector<int> &A, int n){
    assert(!A.empty() && normalize(A[0]) == 0);
    vector<int> G(1, 1);
    int cur = 1;
    while (cur < n){
        cur <<= 1;
        vector<int> Gcut = poly_trim(G, cur);
        Gcut.resize(cur);
        vector<int> diff = poly_sub(poly_trim(A, cur), poly_ln(Gcut, cur), cur);
        diff[0] = add(diff[0], 1);
        G = conv(G, diff);
        G.resize(cur);
    }
    G.resize(n);
    return G;
}

vector<int> poly_pow(const vector<int> &F, ll k, int n){
    if (n == 0) return {};
    if (k == 0){
        vector<int> r(n);
        r[0] = 1;
        return r;
    }
    int shift = 0;
    while (shift < (int)F.size() && normalize(F[shift]) == 0) shift++;
    if (shift == (int)F.size()) return vector<int>(n, 0);
    if (1LL * shift * k >= n) return vector<int>(n, 0);
    int lead = normalize(F[shift]), ilead = inv(lead);
    vector<int> G;
    for (int i = shift; i < (int)F.size(); i++) G.push_back(mult(normalize(F[i]), ilead));
    G.resize(n);
    vector<int> lnG = poly_ln(G, n);
    for (auto &x : lnG) x = mult(x, normalize(k));
    vector<int> H = poly_exp(lnG, n);
    int coef = fp(lead, k);
    for (auto &x : H) x = mult(x, coef);
    vector<int> res(n);
    for (int i = 0; i + shift * k < n && i < (int)H.size(); i++){
        res[i + shift * k] = H[i];
    }
    return res;
}

vector<int> poly_pow2(vector<int> poly, ll p, int limit = 1e5 + 5){
    vector<int> ans{1};
    while (p){
        if (p & 1) ans = conv(ans, poly);
        poly = conv(poly, poly);
        ans.resize(limit);
        poly.resize(limit);
        p >>= 1;
    }
    return ans;
}

int mod_sqrt(int a){
    a = normalize(a);
    if (a == 0) return 0;
    if (MOD == 2) return a;
    if (fp(a, (MOD - 1) / 2) != 1) return -1;
    if (MOD % 4 == 3) return fp(a, (MOD + 1) / 4);

```

```

    int q = MOD - 1, m = 0;
    while ((q & 1) == 0){
        q >>= 1;
        m++;
    }
    int z = 2;
    while (fp(z, (MOD - 1) / 2) != MOD - 1) z++;
    int c = fp(z, q), x = fp(a, (q + 1) / 2), t = fp(a, q);
    while (t != 1){
        int i = 1, tt = mult(t, t);
        while (tt != 1){
            tt = mult(tt, tt);
            i++;
            if (i == m) return -1;
        }
        int b = fp(c, 1LL << (m - i - 1));
        x = mult(x, b);
        c = mult(b, b);
        t = mult(t, c);
        m = i;
    }
    return min(x, MOD - x);
}

vector<int> poly_sqrt(const vector<int> &F, int n){
    if (F.empty() || n == 0) return {};
    int shift = 0;
    while (shift < (int)F.size() && normalize(F[shift]) == 0) shift++;
    if (shift == (int)F.size()) return vector<int>(n, 0);
    if (shift & 1) return {};
    int c0 = mod_sqrt(F[shift]);
    if (c0 == -1) return {};
    int target = n - shift / 2;
    vector<int> shifted_F(F.begin() + shift, F.end());
    vector<int> G(1, c0);
    int inv2 = inv(2);
    int cur = 1;
    while (cur < target){
        cur <<= 1;
        vector<int> Fcut = poly_trim(shifted_F, cur);
        Fcut.resize(cur);
        vector<int> term = conv(Fcut, poly_inv(G, cur));
        term.resize(cur);
        G.resize(cur);
        for (int i = 0; i < cur; i++) G[i] = mult(add(G[i], term[i]), inv2);
    }
    vector<int> res(n);
    for (int i = 0; i < target && i + shift / 2 < n; i++){
        res[i + shift / 2] = G[i];
    }
    return res;
}

```

NTT_CRT.cpp

Use: conv_crt(a,b,mod) multiplies under arbitrary target_mod (or exact CRT value if mod=0); conv3 returns residues under three NTT primes.

Care: conv_crt with target_mod=0 returns exact reconstructed values while they fit signed 64-bit.

Complexity: Runs three NTT convolutions, so it is about three times normal NTT cost.

When: Use it for arbitrary-mod convolution or exact integer convolution beyond single-mod precision.

```

constexpr int MOD1 = 998244353;
constexpr int MOD2 = 1004535809;
constexpr int MOD3 = 469762049;

template<int MOD, int root>
struct NTT{
    static int normalize(ll x){ x %= MOD; if (x < 0) x += MOD; return x; }
    static int add(int a, int b){ a += b; if (a >= MOD) a -= MOD; return a; }
    static int sub(int a, int b){ a -= b; if (a < 0) a += MOD; return a; }
    static int mult(int a, int b){ return 1LL * a * b % MOD; }
    static int fp(int b, ll p){
        int res = 1;
        while (p){
            if (p & 1) res = mult(res, b);
            b = mult(b, b);
            p >>= 1;
        }
        return res;
    }
    static int inv(int x){ return fp(x, MOD - 2); }
    static int nxt_pw(int n){
        int p = 1;
        while (p < n) p <<= 1;
        return p;
    }
    static vector<int>& root_values(){
        static vector<int> roots{0, 1};
        return roots;
    }
    static void prepare_roots(int n){
        auto &roots = root_values();
        if ((int)roots.size() >= n) return;
        int sz = roots.size();
        roots.resize(n);
        for (int len = sz; len < n; len <<= 1){
            int w = fp(root, (MOD - 1) / (len << 1));
            for (int i = len / 2; i < len; i++){
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = mult(roots[i], w);
            }
        }
    }
    static void ntt(vector<int> &a){
        int n = a.size();
        if (n <= 1) return;
        assert((n & (n - 1)) == 0);
        static vector<int> revs;
        if ((int)revs.size() != n){
            int lg = __builtin_ctz(n);
            revs.assign(n, 0);
            for (int i = 0; i < n; i++){
                revs[i] = (revs[i >> 1] >> 1) | ((i & 1) << (lg - 1));
            }
        }
        prepare_roots(n);
        auto &roots = root_values();
        for (int i = 0; i < n; i++){
            if (i < revs[i]) swap(a[i], a[revs[i]]);

```

```

        }
        for (int len = 1; len < n; len <<= 1){
            for (int i = 0; i < n; i += 2 * len){
                for (int j = 0; j < len; j++){
                    int u = a[i + j], v = mult(a[i + j + len], roots[len + j]);
                    a[i + j] = add(u, v);
                    a[i + j + len] = sub(u, v);
                }
            }
        }
    }
    static void intt(vector<int> &a){
        if ((int)a.size() <= 1) return;
        reverse(a.begin() + 1, a.end());
        ntt(a);
        int iv = inv(a.size());
        for (int &x : a) x = mult(x, iv);
    }
    static void dft(vector<int> &a){
        for (int &x : a) x = normalize(x);
        ntt(a);
    }
    static void idft(vector<int> &a){
        intt(a);
    }
    static vector<int> conv(const vector<int> &a, const vector<int> &b){
        if (a.empty() || b.empty()) return {};
        int s = a.size() + b.size() - 1;
        int n = nxt_pw(s);
        vector<int> L(a), R(b);
        L.resize(n); R.resize(n);
        dft(L); dft(R);
        for (int i = 0; i < n; i++) L[i] = mult(L[i], R[i]);
        idft(L);
        return {L.begin(), L.begin() + s};
    }
};

using NTT1 = NTT<MOD1, 3>;
using NTT2 = NTT<MOD2, 3>;
using NTT3 = NTT<MOD3, 3>;

ll extGCD(ll a, ll b, ll &x, ll &y){
    if (b == 0){
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

ll inv(ll a, ll m){
    ll x, y;
    extGCD(a, m, x, y);
    return (x % m + m) % m;
}

vector<array<ll, 3>> conv3(const vector<ll> &a, const vector<ll> &b){

```

```

vector<int> a1(a.size()), a2(a.size()), a3(a.size());
vector<int> b1(b.size()), b2(b.size()), b3(b.size());
for (int i = 0; i < (int)a.size(); i++){
    a1[i] = NTT1::normalize(a[i]);
    a2[i] = NTT2::normalize(a[i]);
    a3[i] = NTT3::normalize(a[i]);
}
for (int i = 0; i < (int)b.size(); i++){
    b1[i] = NTT1::normalize(b[i]);
    b2[i] = NTT2::normalize(b[i]);
    b3[i] = NTT3::normalize(b[i]);
}
auto c1 = NTT1::conv(a1, b1);
auto c2 = NTT2::conv(a2, b2);
auto c3 = NTT3::conv(a3, b3);
vector<array<ll, 3>> ret(c1.size());
for (int i = 0; i < (int)c1.size(); i++){
    ret[i] = {c1[i], c2[i], c3[i]};
}
return ret;
}
vector<ll> conv_crt(const vector<ll> &a, const vector<ll> &b, ll target_mod = 0){
    if (a.empty() || b.empty()) return {};
    auto c = conv3(a, b);
    ll m1m2 = (ll)MOD1 * MOD2;
    ll i1 = inv(MOD1, MOD2);
    ll i2 = inv(m1m2 % MOD3, MOD3);
    __int128_t all_mods = (__int128_t)MOD1 * MOD2 * MOD3;
    vector<ll> res(c.size());
    for (int i = 0; i < (int)c.size(); i++){
        ll v1 = c[i][0], v2 = c[i][1], v3 = c[i][2];
        ll x = NTT2::sub((int)v2, (int)v1);
        x = NTT2::mult((int)x, (int)i1);
        ll val12 = v1 + x * MOD1;
        ll y = NTT3::sub((int)v3, (int)(val12 % MOD3));
        y = NTT3::mult((int)y, (int)i2);
        __int128_t ans = val12 + (__int128_t)y * m1m2;
        if (ans > all_mods / 2) ans -= all_mods;
        if (target_mod){
            ans %= target_mod;
            if (ans < 0) ans += target_mod;
        }
        res[i] = (ll)ans;
    }
    return res;
}

```

Number_theory.cpp

Use: Use fp/inv_mod/is_prime/factorize/divisors/phi_single directly; call sieve_* before reading spf, phi, mobius, divs, or pfact arrays.

Care: Trial factorization is fine for moderate n; use Pollard Rho for large 64-bit composites.

Complexity: Single-number factorization here is trial division; sieve helpers are $O(n \log \log n)$ or similar.

Pitfall: inv_mod asserts $\gcd(a, \text{mod})=1$; use CRT/exgcd handling when inverse may not exist.

```

ll fp(ll a, ll b, ll mod){
    a %= mod;
    if (a < 0) a += mod;
    ll res = 1 % mod;

```

```

while (b){
    if (b & 1) res = (__int128)res * a % mod;
    a = (__int128)a * a % mod;
    b >>= 1;
}
return res;
}
ll exgcd(ll a, ll b, ll &x, ll &y){
    if (b == 0){
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}
ll inv_mod(ll a, ll mod){
    ll x, y;
    ll g = exgcd(a, mod, x, y);
    assert(abs(g) == 1);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
bool is_prime(ll n){
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (ll i = 5; i <= n / i; i += 6){
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}
vector<pair<ll, int>> factorize_pairs(ll n){
    vector<pair<ll, int>> ret;
    for (ll p : {2, 3, 5}){
        if (n % p == 0){
            int cnt = 0;
            while (n % p == 0){
                n /= p;
                cnt++;
            }
            ret.push_back({p, cnt});
        }
    }
}
static int inc[] = {4, 2, 4, 2, 4, 6, 2, 6};
for (ll p = 7, i = 0; p <= n / p; p += inc[i++ & 7]){
    if (n % p == 0){
        int cnt = 0;
        while (n % p == 0){
            n /= p;
            cnt++;
        }
        ret.push_back({p, cnt});
    }
}
}

```

```

    if (n > 1) ret.push_back({n, 1});
    return ret;
}
vector<ll> factorize(ll n){
    vector<ll> ret;
    for (auto [p, cnt] : factorize_pairs(n)){
        while (cnt--> 0) ret.push_back(p);
    }
    return ret;
}
vector<ll> divisors(ll n){
    vector<ll> ret;
    for (ll i = 1; i <= n / i; i++){
        if (n % i == 0){
            ret.push_back(i);
            if (i != n / i) ret.push_back(n / i);
        }
    }
    return ret;
}
ll phi_single(ll n){
    ll ret = n;
    for (auto [p, cnt] : factorize_pairs(n)){
        ret -= ret / p;
    }
    return ret;
}
ll num_divisors(ll n){
    ll ret = 1;
    for (auto [p, cnt] : factorize_pairs(n)){
        ret *= cnt + 1;
    }
    return ret;
}
ll sum_divisors(ll n){
    ll ret = 1;
    for (auto [p, cnt] : factorize_pairs(n)){
        ll cur = 1, pw = 1;
        for (int i = 0; i < cnt; i++){
            pw *= p;
            cur += pw;
        }
        ret *= cur;
    }
    return ret;
}

vector<int> spf, primes;
void sieve_spf(int n){
    spf.assign(n + 1, 0);
    primes.clear();
    if (n >= 1) spf[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!spf[i]){
            spf[i] = i;
            primes.push_back(i);
        }
    }
    for (int p : primes){
        ll x = 1LL * i * p;

```

```

        if (x > n) break;
        spf[x] = p;
        if (p == spf[i]) break;
    }
}

vector<int> phi;
void sieve_phi(int n){
    phi.resize(n + 1);
    iota(phi.begin(), phi.end(), 0);
    for (int i = 2; i <= n; i++){
        if (phi[i] == i){
            for (int j = i; j <= n; j += i){
                phi[j] -= phi[j] / i;
            }
        }
    }
}

vector<int> mobius;
void sieve_mobius(int n){
    mobius.assign(n + 1, 0);
    vector<int> lp(n + 1), ps;
    if (n >= 1) mobius[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!lp[i]){
            lp[i] = i;
            ps.push_back(i);
            mobius[i] = -1;
        }
        for (int p : ps){
            ll x = 1LL * i * p;
            if (x > n) break;
            lp[x] = p;
            if (p == lp[i]){
                mobius[x] = 0;
                break;
            }
            mobius[x] = -mobius[i];
        }
    }
}

vector<vector<int>> divs;
void sieve_divisors(int n){
    divs.assign(n + 1, {});
    for (int i = 1; i <= n; i++){
        for (int j = i; j <= n; j += i){
            divs[j].push_back(i);
        }
    }
}

vector<vector<int>> pfact;
void sieve_prime_factors(int n){
    pfact.assign(n + 1, {});
    for (int i = 2; i <= n; i++){
        if (pfact[i].empty()){

```

```

        for (int j = i; j <= n; j += i){
            pfact[j].push_back(i);
        }
    }
}

```

Pollard Rho Miller Rabin.cpp

Use: miller_rabin(n) tests 64-bit primality; factorize(n) returns prime factors and factorize_pairs(n) returns {prime, exponent}.

Care: factorize returns unsorted prime factors; sort or call factorize_pairs when you need grouped exponents.

Complexity: Miller-Rabin is fast for 64-bit; Pollard Rho is randomized and usually near $n^{1/4}$.

Pitfall: Because factorization is randomized, retrying can fix rare unlucky splits.

```

using u64 = uint64_t;
using u128 = __uint128_t;

u64 mul_mod(u64 a, u64 b, u64 mod){
    return (u128)a * b % mod;
}

u64 add_mod(u64 a, u64 b, u64 mod){
    a += b;
    if (a >= mod || a < b) a -= mod;
    return a;
}

u64 fp(u64 a, u64 b, u64 mod){
    u64 res = 1 % mod;
    while (b){
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool miller_rabin(u64 n){
    if (n < 2) return false;
    for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){
        if (n % p == 0) return n == p;
    }
    u64 d = n - 1, s = 0;
    while ((d & 1) == 0){
        d >>= 1;
        s++;
    }
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}){
        if (a % n == 0) continue;
        u64 x = fp(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool comp = true;
        for (u64 r = 1; r < s; r++){
            x = mul_mod(x, x, n);
            if (x == n - 1){
                comp = false;
                break;
            }
        }
        if (comp) return false;
    }
}

```

```

    return true;
}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
u64 pollard_rho(u64 n){
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true){
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 y = uniform_int_distribution<u64>(0, n - 1)(rng);
        u64 x = 0, ys = 0, q = 1, g = 1;
        auto f = [&](u64 v){ return add_mod(mul_mod(v, v, n), c, n); };
        for (u64 r = 1; g == 1; r <= 1){
            x = y;
            for (u64 i = 0; i < r; i++) y = f(y);
            for (u64 k = 0; k < r && g == 1; k += 128){
                ys = y;
                for (u64 i = 0; i < min<u64>(128, r - k); i++){
                    y = f(y);
                    u64 dif = x > y ? x - y : y - x;
                    if (dif) q = mul_mod(q, dif, n);
                }
                g = gcd(q, n);
            }
        }
        if (g == n){
            do{
                ys = f(ys);
                u64 dif = x > ys ? x - ys : ys - x;
                g = gcd(dif, n);
            }while (g == 1);
        }
        if (g != n) return g;
    }
}

void factor(u64 n, vector<u64> &ret){
    if (n == 1) return;
    if (miller_rabin(n)){
        ret.push_back(n);
        return;
    }
    u64 d = pollard_rho(n);
    factor(d, ret);
    factor(n / d, ret);
}

vector<u64> factorize(u64 n){
    vector<u64> ret;
    factor(n, ret);
    sort(ret.begin(), ret.end());
    return ret;
}

vector<pair<u64, int>> factorize_pairs(u64 n){
    vector<u64> f = factorize(n);
    vector<pair<u64, int>> ret;
    for (auto p : f){
        if (ret.empty() || ret.back().first != p) ret.push_back({p, 1});
        else ret.back().second++;
    }
    return ret;
}

```

```
}
```

Primitive root.cpp

Use: `primitive_root(p)` returns a primitive root modulo prime `p`.

Care: The modulus `p` must be prime; it factors `p-1` by trial division before testing candidates.

Complexity: Factors `p-1` by trial division, then tests candidates against each prime factor.

Pitfall: There is no primitive root modulo arbitrary composite moduli in this helper.

```
ll fp(ll a, ll b, ll mod){
    a %= mod;
    ll res = 1 % mod;
    while (b){
        if (b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}
// primitive root modulo prime p
ll primitive_root(ll p){
    if (p == 2) return 1;
    ll phi = p - 1, x = phi;
    vector<ll> fact;
    for (ll i = 2; i <= x / i; i++){
        if (x % i == 0){
            fact.push_back(i);
            while (x % i == 0) x /= i;
        }
    }
    if (x > 1) fact.push_back(x);
    for (ll g = 2; ; g++){
        bool ok = true;
        for (ll q : fact){
            if (fp(g, phi / q, p) == 1){
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
}
```

rand.cpp

Use: `rnd(l,r)`, `rndll(l,r)`, and `shuffle_vec(v)` give random integers and shuffles from seeded `rng/rng64`.

Care: The seed uses `steady_clock` plus heap address noise; keep one `rng` if you need reproducible local debugging.

Out: `rnd` and `rndll` are inclusive on both `l` and `r`.

Pitfall: Do not use this RNG for cryptography; it is for randomized algorithms and testing.

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

int rnd(int l, int r){
    return uniform_int_distribution<int>(l, r)(rng);
}
ll rndll(ll l, ll r){
    return uniform_int_distribution<ll>(l, r)(rng64);
}
```

```
}
template<typename T>
void shuffle_vec(vector<T> &v){
    shuffle(v.begin(), v.end(), rng);
}
```

Segmented sieve.cpp

Use: `segmented_sieve(l,r)` returns primes in `[l,r]`, and `segmented_sieve_mask(l,r)` returns primality mask for the interval.

Care: The interval `[l,r]` is inclusive; memory is $O(r-l+1)$, so keep the segment width reasonable.

Complexity: Runs in about $O((r-l+1) \log \log r + \sqrt{r} \log \log \sqrt{r})$.

Pitfall: When `l <= 1` the mask explicitly marks 0 and 1 as non-prime.

```
vector<int> simple_sieve(int n){
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    if (n >= 0) is_prime[0] = false;
    if (n >= 1) is_prime[1] = false;
    for (int i = 2; i <= n; i++){
        if (!is_prime[i]) continue;
        primes.push_back(i);
        if (1LL * i * i <= n){
            for (ll j = 1LL * i * i; j <= n; j += i){
                is_prime[j] = false;
            }
        }
    }
    return primes;
}
vector<bool> segmented_sieve_mask(ll l, ll r){
    if (l > r) return {};
    int root = sqrtl(r);
    while (1LL * (root + 1) * (root + 1) <= r) root++;
    while (1LL * root * root > r) root--;
    vector<int> primes = simple_sieve(root);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p : primes){
        ll start = max(p * p, ((l + p - 1) / p) * p);
        for (ll j = start; j <= r; j += p){
            is_prime[j - l] = false;
        }
    }
    for (ll x = l; x <= min(r, 1LL); x++){
        is_prime[x - l] = false;
    }
    return is_prime;
}
// returns all primes in [l, r]
vector<ll> segmented_sieve(ll l, ll r){
    vector<bool> is_prime = segmented_sieve_mask(l, r);
    vector<ll> primes;
    for (ll i = 0; i < (ll)is_prime.size(); i++){
        if (is_prime[i]) primes.push_back(l + i);
    }
    return primes;
}
```

SOS dp.cpp

Use: For subset/superset transforms vector size must be a power of two; divisor/multiple transforms work on index arrays by value.

Apps: zeta/mobius transforms mutate sums, while gcd_convolution(a,b) and lcm_convolution(a,b) return value-indexed convolutions.

Care: Subset transforms index by bitmask size 2^n ; divisor/multiple transforms expect arrays indexed from 1 to max value.

Complexity: Subset/superset transforms are $O(n \log n)$ where n is number of masks.

Pitfall: gcd/lcm convolution arrays are value-indexed, so index 0 is not part of the number-theory domain.

```
bool is_power_two(int n){
    return n > 0 && (n & (n - 1)) == 0;
}
template<typename T>
void subset_zeta(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask] += a[mask ^ bit];
        }
    }
}
template<typename T>
void subset_mobius(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask] -= a[mask ^ bit];
        }
    }
}
template<typename T>
void superset_zeta(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask ^ bit] += a[mask];
        }
    }
}
template<typename T>
void superset_mobius(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask ^ bit] -= a[mask];
        }
    }
}
vector<int> spf, primes;
void sieve_primes(int n){
    spf.assign(n + 1, 0);
    primes.clear();
```

```
    if (n >= 1) spf[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!spf[i]){
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes){
            ll x = 1LL * i * p;
            if (x > n) break;
            spf[x] = p;
            if (p == spf[i]) break;
        }
    }
}
void ensure_primes(int n){
    if ((int)spf.size() <= n) sieve_primes(n);
}
template<typename T>
void divisor_zeta(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = 1; i * p <= n; i++){
            a[i * p] += a[i];
        }
    }
}
template<typename T>
void divisor_mobius(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = n / p; i >= 1; i--){
            a[i * p] -= a[i];
        }
    }
}
template<typename T>
void multiple_zeta(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = n / p; i >= 1; i--){
            a[i] += a[i * p];
        }
    }
}
template<typename T>
void multiple_mobius(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = 1; i * p <= n; i++){
            a[i] -= a[i * p];
        }
    }
}
```



```

    }
}
template<typename T>
vector<T> gcd_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    multiple_zeta(a);
    multiple_zeta(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    multiple_mobius(a);
    return a;
}
template<typename T>
vector<T> lcm_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    divisor_zeta(a);
    divisor_zeta(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    divisor_mobius(a);
    return a;
}

```

xor basis.cpp

Use: XorBasis<LG> xb; insert(x), can(x), max_xor(x), min_xor(x), kth(k), merge(other), and rank()/size() for linear basis queries.

Care: Call normalize() before kth/count_le if you inserted new vectors; LG must cover the highest bit used.

Complexity: insert/can/max/min are $O(LG)$; kth and count_le need normalized basis.

Pitfall: kth is over representable xor values; account for zero and duplicate subset counts separately.

```

template<int LG = 60>
struct XorBasis{
    array<ll, LG> basis{};
    vector<ll> rref;
    ll cnt = 0;
    int sz = 0;

    void clear(){
        basis.fill(0);
        rref.clear();
        cnt = 0;
        sz = 0;
    }
    bool insert(ll x, bool count = true){
        if (count) cnt++;
        for (int i = LG - 1; i >= 0; i--){
            if (!(x >> i & 1)) continue;
            if (!basis[i]){
                basis[i] = x;
                sz++;
                rref.clear();
                return true;
            }
            x ^= basis[i];
        }
        return false;
    }
    void merge(const XorBasis &o){
        cnt += o.cnt;
    }
}

```

```

        for (int i = 0; i < LG; i++){
            if (o.basis[i]) insert(o.basis[i], false);
        }
    }
    void anding(ll val){
        vector<ll> v;
        for (int i = 0; i < LG; i++){
            if (basis[i] & val) v.push_back(basis[i] & val);
        }
        basis.fill(0);
        rref.clear();
        sz = 0;
        for (ll x : v) insert(x, false);
    }
    int rank() const {
        return sz;
    }
    ll count_distinct() const {
        return sz >= 63 ? LLONG_MAX : (1LL << sz);
    }
    ll count_zero_subsets() const {
        return cnt - sz >= 63 ? LLONG_MAX : (1LL << (cnt - sz));
    }
    bool can(ll x) const {
        for (int i = LG - 1; i >= 0; i--){
            if (x >> i & 1){
                if (!basis[i]) return false;
                x ^= basis[i];
            }
        }
        return true;
    }
    ll max_xor(ll x = 0) const {
        for (int i = LG - 1; i >= 0; i--){
            if (basis[i] && (x ^ basis[i]) > x) x ^= basis[i];
        }
        return x;
    }
    ll min_xor(ll x) const {
        for (int i = LG - 1; i >= 0; i--){
            if (basis[i] && (x ^ basis[i]) < x) x ^= basis[i];
        }
        return x;
    }
    ll min_xor() const {
        if (sz == 0) return 0;
        XorBasis tmp = *this;
        tmp.normalize();
        ll ans = LLONG_MAX;
        for (ll x : tmp.rref){
            ans = min(ans, x);
        }
        return ans;
    }
    void normalize(){
        rref.clear();
        for (int i = 0; i < LG; i++){
            if (!basis[i]) continue;
            ll x = basis[i];

```

```

        for (int j = i - 1; j >= 0; j--){
            if (x >> j & 1) x ^= basis[j];
        }
        rref.push_back(x);
    }
    sort(rref.rbegin(), rref.rend());
}

ll kth(ll k){
    if (rref.empty() && sz) normalize();
    if (sz >= 63 || k >= (1LL << sz)) return -1;
    ll ans = 0;
    for (int i = 0; i < sz; i++){
        if (k >> (sz - i - 1) & 1) ans ^= rref[i];
    }
    return ans;
}

ll count_le(ll val){
    if (val < 0) return 0;
    if (rref.empty() && sz) normalize();
    if (LG < 63 && val >= (1LL << LG)) return sz >= 63 ? LLONG_MAX : (1LL << sz);
    ll ans = 0, mask = 0;
    int j = 0;
    for (int b = LG - 1; b >= 0; b--){
        bool piv = j < sz && (63 - __builtin_clzll(rref[j])) == b;
        int bit = (val >> b) & 1;
        if (piv){
            if (bit){
                if (sz - 1 - j >= 63) return LLONG_MAX;
                ans += 1LL << (sz - 1 - j);
                mask ^= rref[j];
            }
            j++;
        }else{
            int cur = (mask >> b) & 1;
            if (cur < bit){
                if (sz - j >= 63) return LLONG_MAX;
                return ans + (1LL << (sz - j));
            }
            if (cur > bit) return ans;
        }
    }
    return ans + 1;
}
};

```

5 strings

Aho.cpp

Use: AC ac; add_pattern(p) returns id, build(), then search(text) returns start positions for each pattern id.

Care: Alphabet is lowercase a-z; build() must be called after all patterns and before search/advance use.

Complexity: build is $O(\text{total pattern length} * \text{alphabet})$, search is $O(\text{text length plus reported matches})$.

Pitfall: The while over out_link reports all suffix patterns; many matches can make output large.

```
struct AC{
    int N, P;
    static const int A = 26;
    vector<array<int, A>> next;
    vector<int> link, out_link, pat_len;
    vector<vector<int>> out, rev;
    AC(): N(0), P(0) {node();}
    int node() {
        next.push_back(array<int, A>{0});
        rev.push_back({});
        link.emplace_back(0);
        out_link.emplace_back(0);
        out.emplace_back();
        return N++;
    }
    inline int get (char c){ return c - 'a'; }
    int add_pattern(const string & T){
        int u = 0;
        for(auto c : T){
            if(!next[u][get(c)]) next[u][get(c)] = node();
            u = next[u][get(c)];
        }
        out[u].push_back(P);
        pat_len.push_back(T.length());
        return P++;
    }
    void build(){
        queue<int> q; q.push(0);
        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int v = next[u][c];
                if(!v) next[u][c] = next[link[u]][c];
                else{
                    link[v] = u ? next[link[u]][c] : 0;
                    rev[link[v]].push_back(v);
                    out_link[v] = out[link[v]].empty() ? out_link[link[v]] : link[v];
                    q.push(v);
                }
            }
        }
    }
    int advance(int u, char c){
        return next[u][get(c)];
    }
    vector<vector<int>> search(const string & s){
        int n = (int)s.length(), m = (int)pat_len.size();
        int u = 0;
        vector<vector<int>> ret(m);
        for(int i = 0; i < n; i++){
            char c = s[i];
```

```
            u = advance(u, c);
            int x = u;
            while(x != 0){
                for(int v : out[x]) ret[v].push_back(i - pat_len[v] + 1);
                x = out_link[x];
            }
        }
        return ret;
    }
};
```

Bitset string matching.cpp

Use: BitsetMatcher bm(text); occurrences(p) returns 0-indexed starts, count(p) counts matches, and set_char(i,c) updates text.

Care: N is the maximum text length and SIG=256 handles bytes; memory is SIG*N bits, so reduce SIG for small alphabets.

Complexity: Each query costs $O(|\text{pattern}| * N / \text{word_size})$; listing occurrences also costs output scan time.

When: Best for many exact pattern queries on one mostly static text; use KMP/Z if there is only one pattern.

Pitfall: match_bits(p)[i] means p starts at i; only inspect i with $i+|p| \leq \text{text length}$.

```
const int N = 3e5 + 5;
const int SIG = 256;

// Many pattern queries on one text:  $O(|p| * N / \text{word\_size})$ , memory  $O(\text{SIG} * N / \text{word\_size})$ .
// match_bits(p)[i] = 1 iff p occurs starting at 0-indexed position i.
struct BitsetMatcher{
    string s;
    vector<bitset<N>> at;

    BitsetMatcher() : at(SIG) {}
    BitsetMatcher(const string &_s) : at(SIG) {
        build(_s);
    }
    void build(const string &_s){
        s = _s;
        assert((int)s.size() < N);
        for (auto &b : at) b.reset();
        for (int i = 0; i < (int)s.size(); i++){
            at[(unsigned char)s[i]].set(i);
        }
    }
    void set_char(int i, char c){
        assert(0 <= i && i < (int)s.size());
        if (s[i] == c) return;
        at[(unsigned char)s[i]].reset(i);
        s[i] = c;
        at[(unsigned char)s[i]].set(i);
    }
    bitset<N> match_bits(const string &p) const{
        int n = s.size(), m = p.size();
        bitset<N> ret;
        if (m > n) return ret;
        if (m == 0){
            for (int i = 0; i <= n; i++) ret.set(i);
            return ret;
        }
        ret.set();
        for (int i = 0; i < m; i++){
```

```

        ret &= at[(unsigned char)p[i]] >> i;
    }
    return ret;
}

int count(const string &p) const{
    return (int)match_bits(p).count();
}

bool exists(const string &p) const{
    return match_bits(p).any();
}

vector<int> occurrences(const string &p) const{
    int n = s.size(), m = p.size();
    vector<int> ret;
    if (m > n) return ret;
    bitset<N> good = match_bits(p);
    for (int i = 0; i + m <= n; i++){
        if (good[i]) ret.push_back(i);
    }
    return ret;
}
};

```

dynamic_aho.cpp

Use: dynamic_AC ac; insert(pattern), then cnt(text) returns total occurrences of all inserted patterns in text.

Care: Good for online insertions without deletions; cnt(text) sums matches across the logarithmic collection of automata.

Complexity: Each insert rebuilds $O(\log \text{number_of_patterns})$ automata by binary-counter merging.

Pitfall: No deletion support; rebuild from scratch if patterns need to be removed.

```

struct AC{
    int N, P;
    static const int A = 26;
    vector<array<int, A>> next;
    vector<int> link, out_link, cnt;
    vector<vector<int>> out, rev;
    AC(): N(0), P(0) {node();}
    int node() {
        next.push_back(array<int, A>{0});
        rev.push_back({});
        cnt.emplace_back(0);
        link.emplace_back(0);
        out_link.emplace_back(0);
        out.emplace_back();
        return N++;
    }
    inline int get (char c){ return c - 'a'; }
    int add_pattern(const string &T){
        int u = 0;
        for(auto c : T){
            if(!next[u][get(c)]) next[u][get(c)] = node();
            u = next[u][get(c)];
        }
        out[u].push_back(P);
        cnt[u]++;
        return P++;
    }
    void build(){
        queue<int> q; q.push(0);

```

```

        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int v = next[u][c];
                if(!v) next[u][c] = next[link[u]][c];
                else{
                    link[v] = u ? next[link[u]][c] : 0;
                    rev[link[v]].push_back(v);
                    cnt[v] += cnt[link[v]];
                    out_link[v] = out[link[v]].empty() ? out_link[link[v]] : link[v];
                    q.push(v);
                }
            }
        }
    }
    int advance(int u, char c){
        return next[u][get(c)];
    }
    vector<vector<int>> search(const string &s, vector<int>& pat){
        int n = (int)s.length(), m = (int)pat.size();
        int u = 0;
        vector<vector<int>> ret(m);
        for(int i = 0; i < n; i++){
            char c = s[i];
            u = advance(u, c);
            int x = u;
            while(x != 0){
                for(int v : out[x]) ret[v].push_back(i - pat[v] + 1);
                x = out_link[x];
            }
        }
        return ret;
    }
};

```

```

struct dynamic_AC{
    int lg;
    vector<vector<string>> a;
    vector<AC> aho;
    dynamic_AC(int LOG = 20){
        lg = LOG;
        a = vector<vector<string>>(lg);
        aho = vector<AC>(lg);
    }

    void insert(const string &s){
        vector<string> have = {s};
        for(int i = 0; i < lg; i++){
            if(i == lg - 1){
                lg++;
                a.push_back({});
                aho.push_back(AC());
            }
            if(a[i].empty()){
                swap(have, a[i]);
                for(auto & v : a[i]) aho[i].add_pattern(v);
                aho[i].build();
                break;
            }else{

```

```

        have.insert(end(have), begin(a[i]), end(a[i]));
        vector<string>().swap(a[i]);
        aho[i] = AC();
    }
}

ll cnt(const string & s){
    ll answer = 0;
    for(int i = 0; i < lg; i++){
        int state = 0;
        for(auto & v : s){
            state = aho[i].advance(state, v);
            answer += aho[i].cnt[state];
        }
    }
    return answer;
}
};

```

Generalized suffix automaton.cpp

Use: GSAM g(total_len, m); g.init(total_len,m), g.build(strings); lowercase strings and m <= 63 for mask-based methods.

Apps: get_lcs(K) LCS length for first K strings; distinct_substrings() count; match(P) total occurrences; get_kth(K) kth distinct substring.

Apps: shortest_distinguishing() gives min unique length.

Apps: get_all_occurrences(P) gives {string_id,start}; string_specific_occ(P,i) counts in strings[i].

Care: Call build_tree() through build(); mask-based LCS/distinguishing routines need at most 63 strings.

Complexity: Build is linear in total length; occurrence-heavy routines may traverse suffix-link subtrees.

Pitfall: Masks use ll, so K beyond 63 strings needs a different bitset representation.

```

struct GSAM{
    static const int A = 26;
    struct node{
        int len, link;
        bool is_cloned;
        int nxt[A];
        node(){
            memset(nxt, -1, sizeof nxt);
            len = 0;
            link = -1;
            is_cloned = false;
        }
    };
    int sz;
    vector<node> t;
    vector<int> cnt;
    vector<vector<int>> inv, g;
    vector<ll> freq;
    GSAM(){
        GSAM(int n, int m){
            init(n, m);
        }
        void init(int n, int m){
            t.assign(2 * n + 5, node());
            cnt.assign(2 * n + 5, 0);
            freq.assign(2 * n + 5, -1);
            g.assign(2 * n + 5, {});
        }
    };

```

```

        inv.assign(m, {});
        sz = 1;
    }
    int get(char c){
        return c - 'a';
    }
    int extend(char c, int last){
        int c_idx = get(c);
        if (t[last].nxt[c_idx] != -1){
            int q = t[last].nxt[c_idx];
            if (t[last].len + 1 == t[q].len) return q;
            int clone = sz++;
            t[clone] = t[q];
            t[clone].len = t[last].len + 1;
            t[clone].is_cloned = true;
            int p = last;
            while (p != -1 && t[p].nxt[c_idx] == q){
                t[p].nxt[c_idx] = clone;
                p = t[p].link;
            }
            t[q].link = clone;
            return clone;
        }
        int cur = sz++;
        t[cur].len = t[last].len + 1;
        int p = last;
        while (p != -1 && t[p].nxt[c_idx] == -1){
            t[p].nxt[c_idx] = cur;
            p = t[p].link;
        }
        if (p == -1){
            t[cur].link = 0;
        }else{
            int q = t[p].nxt[c_idx];
            if (t[p].len + 1 == t[q].len){
                t[cur].link = q;
            }else{
                int clone = sz++;
                t[clone] = t[q];
                t[clone].len = t[p].len + 1;
                while (p != -1 && t[p].nxt[c_idx] == q){
                    t[p].nxt[c_idx] = clone;
                    p = t[p].link;
                }
                t[q].link = t[cur].link = clone;
                t[clone].is_cloned = true;
            }
        }
        return cur;
    }
    void build_tree(){
        for (int i = 1; i < sz; i++){
            g[t[i].link].push_back(i);
        }
    }
    void build(const vector<string> &strings){
        for (int i = 0; i < (int)strings.size(); i++){
            int lst = 0;
            inv[i].resize(strings[i].size());

```

```

        for (int j = 0; j < (int)strings[i].size(); j++){
            lst = extend(strings[i][j], lst);
            cnt[lst]++;
            inv[i][j] = lst;
        }
    }
    build_tree();
}

ll occ(int i){
    if (~freq[i]) return freq[i];
    ll ret = cnt[i];
    for (auto x : g[i]){
        ret += occ(x);
    }
    return freq[i] = ret;
}

void getMask(int u, vector<ll> &mask){
    for (int v : g[u]){
        getMask(v, mask);
        mask[u] |= mask[v];
    }
}

int get_lcs(int K){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int max_len = 0;
    ll target = (1LL << K) - 1;
    for (int i = 1; i < sz; i++){
        if ((mask[i] & target) == target){
            max_len = max(max_len, t[i].len);
        }
    }
    return max_len;
}

ll distinct_substrings(){
    ll total = 0;
    for (int i = 1; i < sz; i++){
        total += t[i].len - t[t[i].link].len;
    }
    return total;
}

ll match(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    return occ(u);
}

ll calc(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = 1;
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){

```

```

            dp[u] += calc(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}

string get_kth(ll K){
    vector<ll> dp(sz, -1);
    calc(0, dp);
    string res = "";
    int u = 0;
    while (K > 0){
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] != -1){
                int v = t[u].nxt[c];
                if (dp[v] >= K){
                    res += (char)('a' + c);
                    u = v;
                    K--;
                    break;
                }else{
                    K -= dp[v];
                }
            }
        }
    }
    return res;
}

int shortest_distinguishing(){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int min_len = 1e9;
    for (int i = 1; i < sz; i++){
        if (__builtin_popcountll(mask[i]) == 1){
            min_len = min(min_len, t[t[i].link].len + 1);
        }
    }
    return min_len == 1e9 ? -1 : min_len;
}

vector<pair<int, int>> get_all_occurrences(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return {};
        u = t[u].nxt[get(c)];
    }
    vector<vector<pair<int, int>>> state_endpoints(sz);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            state_endpoints[inv[i][j]].push_back({i, j});
        }
    }
    vector<pair<int, int>> res;
    collect(u, (int)P.length(), state_endpoints, res);
    return res;
}

```

```

void collect(int u, int len, vector<vector<pair<int, int>>> &ends, vector<pair<int,
↪ int>> &res){
    for (auto ep : ends[u]){
        res.push_back({ep.first, ep.second - len + 1});
    }
    for (int v : g[u]){
        collect(v, len, ends, res);
    }
}

11 dfsSp(int u, vector<int> &cnt2, vector<ll> &freq2){
    if (freq2[u] != -1) return freq2[u];
    11 ret = cnt2[u];
    for (int v : g[u]){
        ret += dfsSp(v, cnt2, freq2);
    }
    return freq2[u] = ret;
}

11 string_specific_occ(const string &P, int str_idx){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    vector<int> cnt2(sz, 0);
    for (int j = 0; j < (int)inv[str_idx].size(); j++){
        cnt2[inv[str_idx][j]]++;
    }
    vector<ll> freq2(sz, -1);
    return dfsSp(u, cnt2, freq2);
}
};

```

KMP.cpp

Use: KMP(s) builds pi; find_occurrences(pattern,text) returns all 0-indexed starts; automaton(pattern) builds transitions.

Apps: borders(s), period_lengths(s), minimal_period(s), and prefix_occurrences(s) cover common prefix-function applications.

Care: find_occurrences concatenates pattern + '#' + text, so choose a separator not present if alphabet is not lowercase.

Out: prefix_occurrences(s)[len] counts how many times prefix length len appears as a substring of s.

Complexity: KMP and find_occurrences are $O(n+m)$; automaton is $O(\text{pattern_length} * 26)$.

Complexity: borders, periods, minimal_period, and prefix_occurrences are $O(n)$ from the prefix function.

Pitfall: The automaton assumes lowercase a-z transitions.

```

vector<int> KMP(const string &s){
    int n = s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++){
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]){
            j = pi[j - 1];
        }
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> find_occurrences(const string &pat, const string &text){

```

```

    string s = pat + '#' + text;
    vector<int> pi = KMP(s), ret;
    int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++){
        if (pi[i] == m) ret.push_back(i - 2 * m);
    }
    return ret;
}

vector<vector<int>> automaton(string s){ // aut[s.length() + 1][26]
    s += '#';
    int n = s.length();
    vector<int> pi = KMP(s);
    vector<vector<int>> aut(n, vector<int>(26));
    for (int i = 0; i < n; i++){
        for (int j = 0; j < 26; j++){
            if (i > 0 && 'a' + j != s[i]){
                aut[i][j] = aut[pi[i - 1]][j];
            }else{
                aut[i][j] = i + ('a' + j == s[i]);
            }
        }
    }
    return aut;
}

vector<int> borders(const string &s){
    vector<int> pi = KMP(s), ret;
    int x = pi.empty() ? 0 : pi.back();
    while (x > 0){
        ret.push_back(x);
        x = pi[x - 1];
    }
    reverse(ret.begin(), ret.end());
    return ret;
}

vector<int> period_lengths(const string &s){
    int n = s.size();
    if (n == 0) return {};
    vector<int> ret;
    for (int b : borders(s)) ret.push_back(n - b);
    ret.push_back(n);
    sort(ret.begin(), ret.end());
    return ret;
}

int minimal_period(const string &s){
    int n = s.size();
    if (n == 0) return 0;
    vector<int> pi = KMP(s);
    int p = n - pi.back();
    return n % p == 0 ? p : n;
}

vector<int> prefix_occurrences(const string &s){
    int n = s.size();
    vector<int> pi = KMP(s), ret(n + 1, 0);
    for (int x : pi) ret[x]++;
    for (int i = n; i > 0; i--) ret[pi[i - 1]] += ret[i];
    for (int i = 1; i <= n; i++) ret[i]++;
    ret[0] = 0;
    return ret; // ret[len] = occurrences of prefix s[0..len-1] in s
}

```

```
}
```

manacher.cpp

Use: manacher m(s); odd/even radii, ispal(l,r), longest_palindrome(), count_palindromes(), starts, and ends are built.

Out: starts[i] and ends[i] count palindromes starting/ending at i after build(), useful for palindrome DP counts.

Complexity: build is O(n); palindrome checks and longest radii queries are O(1).

Pitfall: ispal(l,r) expects inclusive 0-indexed bounds.

```
struct manacher{
    vector<int> p, odd, even;
    vector<ll> starts, ends;
    manacher(){}
    manacher(const string &S){
        build(S);
    }
    void run_manacher(const string &s){
        int n = s.length();
        p.assign(n, 1);
        int l = 1, r = 1;
        for (int i = 1; i < n; i++){
            p[i] = i < r ? min(r - i, p[l + r - i]) : 1;
            while (i - p[i] >= 0 && i + p[i] < n && s[i - p[i]] == s[i + p[i]]){
                p[i]++;
            }
            if (r < i + p[i]){
                r = i + p[i];
                l = i - p[i];
            }
        }
    }
    void build(const string &s){
        string t = "#";
        t.reserve(s.length() * 2 + 1);
        for (char c : s){
            t += c;
            t += '#';
        }
        run_manacher(t);
        build_radii(s.length());
        build_counts(s.length());
    }
    void build_radii(int n){
        odd.assign(n, 0);
        even.assign(n, 0);
        for(int i = 0; i < n; i++){
            odd[i] = get_longest(i, false) / 2 + 1;
            even[i] = get_longest(i, true) / 2;
        }
    }
    int get_longest(int cen, bool even){
        return p[2 * cen + 1 + even] - 1;
    }
    bool ispal(int l, int r){
        return (r - l + 1) <= get_longest((l + r) >> 1, 1 % 2 != r % 2);
    }
    pair<int, int> longest_palindrome(){
        int best = 0, pos = 0;
```

```
        for (int i = 0; i < (int)p.size(); i++){
            if (p[i] - 1 > best){
                best = p[i] - 1;
                pos = i;
            }
        }
        int l = (pos - best) / 2;
        return {l, l + best - 1};
    }
    ll count_palindromes(){
        ll ret = 0;
        for (int x : p){
            ret += x / 2;
        }
        return ret;
    }
    void build_counts(int n){
        starts.assign(n + 1, 0);
        ends.assign(n + 1, 0);
        for (int i = 0; i < (int)p.size(); i++){
            int len = p[i] - 1;
            if (!len) continue;
            int l = (i - len + 1) / 2;
            int r = (i - 1) / 2;
            starts[l]++;
            starts[r + 1]--;
            int el = i - r - 1;
            int er = i - l - 1;
            ends[el]++;
            ends[er + 1]--;
        }
        for (int i = 1; i < n; i++){
            starts[i] += starts[i - 1];
            ends[i] += ends[i - 1];
        }
        starts.pop_back();
        ends.pop_back();
    }
};
```

palindromic tree.cpp

Use: PalindromicTree pt; build(s) fills all static queries; init()+extend(c) supports online appends, and rollback() removes the last append.

Apps: cnt_dis()/count_distinct and pal_cnt()/count_total count distinct/all palindromic substrings; ending_at(i) and starting_at(i) are 0-indexed counts.

Apps: get_longest()/longest_palindrome returns one longest palindrome; get_all() returns {palindrome, occurrences}; min_pal_partitions() gives minimum palindromic factorization.

Apps: count_crossing_pairs(MOD) returns unordered intersecting palindromic-substring pairs, matching the common CF 17E style formula.

Care: Alphabet is lowercase a-z; call propagate_occurrences() only after final edits because rollback/extend assert before changing propagated counts.

Complexity: Each extend is amortized O(1) for fixed alphabet; build and crossing-pair count are O(n), while start counts are built lazily from the reversed string.

Complexity: min_pal_partitions uses diff/series links and is O(n log n)-style in practice over the series-link chain.

Before: Use build(s) when you need starting_at() or count_crossing_pairs(); online extend keeps ending_at and total counts immediately.

Pitfall: get_all() computes propagated occurrence counts on a copy, so it is safe before/after propagate_occurrences().

```
struct PalindromicTree{
    static const int A = 26;
    struct node{
        int nxt[A], smart_link[A];
        int len, st, en, link, diff, slink, cnt, oc;
        node(){
            memset(nxt, 0, sizeof nxt);
            for (int i = 0; i < A; i++) smart_link[i] = 1;
            len = st = en = link = diff = slink = cnt = oc = 0;
        }
    };

    string s;
    vector<node> t;
    vector<array<int, 4>> changes;
    vector<int> end_cnt, start_cnt;
    vector<ll> tot_pals_hist;
    int sz, last;
    ll tot_pals;
    bool propagated;

    PalindromicTree(){
        init();
    }
    void init(){
        s.clear();
        t.assign(3, node());
        sz = 2;
        last = 2;
        t[1].len = -1;
        t[1].link = t[1].slink = 1;
        t[2].len = 0;
        t[2].link = 1;
        t[2].slink = 1;
        changes.clear();
        end_cnt.clear();
        start_cnt.clear();
        tot_pals_hist.clear();
        tot_pals = 0;
        propagated = false;
    }
    void build(const string &str){
        init();
        for (char c : str) extend(c);
        build_start_counts();
    }
    int extend(char c){
        assert('a' <= c && c <= 'z');
        assert(!propagated);
        start_cnt.clear();
        tot_pals_hist.push_back(tot_pals);
        int cur = last, pos = s.size(), ch = c - 'a';
        s += c;
        if (pos - t[cur].len - 1 < 0 || s[pos - t[cur].len - 1] != c){
            cur = t[cur].smart_link[ch];
        }
    }
};
```

```
if (t[cur].nxt[ch]){
    changes.push_back({last, t[cur].nxt[ch], -1, -1});
    last = t[cur].nxt[ch];
    t[last].oc++;
    tot_pals += t[last].cnt;
    end_cnt.push_back(t[last].cnt);
    return 0;
}
changes.push_back({last, -1, cur, ch});
sz++;
last = sz;
t.emplace_back();
t[cur].nxt[ch] = sz;
t[sz].oc = 1;
t[sz].len = t[cur].len + 2;
t[sz].en = pos;
t[sz].st = pos - t[sz].len + 1;
if (t[sz].len == 1){
    t[sz].link = 2;
    t[sz].smart_link[ch] = 2;
}else{
    t[sz].link = t[t[cur].smart_link[ch]].nxt[ch];
    for (int i = 0; i < A; i++){
        int x = t[sz].link;
        if (pos - t[x].len >= 0 && s[pos - t[x].len] - 'a' == i){
            t[sz].smart_link[i] = x;
        }else{
            t[sz].smart_link[i] = t[x].smart_link[i];
        }
    }
}
t[sz].cnt = 1 + t[t[sz].link].cnt;
t[sz].diff = t[sz].len - t[t[sz].link].len;
t[sz].slink = (t[sz].diff == t[t[sz].link].diff ? t[t[sz].link].slink :
    ↪ t[sz].link);
tot_pals += t[sz].cnt;
end_cnt.push_back(t[sz].cnt);
return 1;
}
void rollback(){
    if (s.empty()) return;
    assert(!propagated);
    s.pop_back();
    end_cnt.pop_back();
    start_cnt.clear();
    auto x = changes.back();
    changes.pop_back();
    int prvlst = x[0], oc_node = x[1], parent = x[2], ch = x[3];
    last = prvlst;
    if (oc_node == -1){
        t[parent].nxt[ch] = 0;
        t.pop_back();
        sz--;
    }else{
        t[oc_node].oc--;
    }
    tot_pals = tot_pals_hist.back();
    tot_pals_hist.pop_back();
}
```

```

int cnt_dis() const{ return sz - 2; }
int count_distinct() const{ return cnt_dis(); }
ll pal_cnt() const{ return tot_pals; }
ll count_total() const{ return pal_cnt(); }
int ending_at(int i) const{
    assert(0 <= i && i < (int)end_cnt.size());
    return end_cnt[i];
}
void build_start_counts(){
    PalindromicTree rev;
    for (int i = (int)s.size() - 1; i >= 0; i--) rev.extend(s[i]);
    start_cnt.assign(s.size(), 0);
    for (int i = 0; i < (int)s.size(); i++) start_cnt[i] = rev.end_cnt[(int)s.size()
    ↪ - 1 - i];
}
int starting_at(int i){
    assert(0 <= i && i < (int)s.size());
    if ((int)start_cnt.size() != (int)s.size()) build_start_counts();
    return start_cnt[i];
}
vector<int> occurrence_counts() const{
    vector<int> occ(sz + 1);
    for (int i = 3; i <= sz; i++) occ[i] = t[i].oc;
    if (propagated) return occ;
    for (int i = sz; i >= 3; i--) occ[t[i].link] += occ[i];
    return occ;
}
void propagate_occurrences(){
    if (propagated) return;
    vector<int> occ = occurrence_counts();
    for (int i = 3; i <= sz; i++) t[i].oc = occ[i];
    propagated = true;
}
string get_longest() const{
    int best = 2;
    for (int i = 3; i <= sz; i++){
        if (t[i].len > t[best].len) best = i;
    }
    if (best == 2) return "";
    return s.substr(t[best].st, t[best].len);
}
string longest_palindrome() const{ return get_longest(); }
vector<pair<string, int>> get_all() const{
    vector<int> occ = occurrence_counts();
    vector<pair<string, int>> res;
    for (int i = 3; i <= sz; i++){
        res.push_back({s.substr(t[i].st, t[i].len), occ[i]});
    }
    return res;
}
int min_pal_partitions() const{
    const int INF = 1e9;
    PalindromicTree pt;
    vector<int> dp(s.size() + 1, INF), g(s.size() + 3, INF);
    dp[0] = 0;
    for (int i = 1; i <= (int)s.size(); i++){
        pt.extend(s[i - 1]);
        dp[i] = INF;

```

```

        if ((int)g.size() <= pt.sz) g.resize(pt.sz + 1, INF);
        for (int v = pt.last; pt.t[v].len > 0; v = pt.t[v].slink){
            int take = pt.t[pt.t[v].slink].len + pt.t[v].diff;
            g[v] = dp[i - take];
            if (pt.t[v].diff == pt.t[pt.t[v].link].diff){
                g[v] = min(g[v], g[pt.t[v].link]);
            }
            dp[i] = min(dp[i], g[v] + 1);
        }
    }
    return dp[s.size()];
}
int min_palindromic_partitions() const{ return min_pal_partitions(); }
ll count_crossing_pairs(ll MOD = (int)1e9 + 7){
    if (s.empty()) return 0;
    if ((int)start_cnt.size() != (int)s.size()) build_start_counts();
    vector<ll> pref(end_cnt.size());
    for (int i = 0; i < (int)end_cnt.size(); i++){
        pref[i] = end_cnt[i];
        if (i) pref[i] += pref[i - 1];
    }
    ll total = pref.back();
    ll all_pairs = (__int128)total * (total - 1) / 2 % MOD;
    ll non_crossing = 0;
    for (int i = 0; i + 1 < (int)s.size(); i++){
        non_crossing = (non_crossing + pref[i] % MOD * (start_cnt[i + 1] % MOD)) %
        ↪ MOD;
    }
    return (all_pairs - non_crossing + MOD) % MOD;
}
};

```

String_Hashing.cpp

Use: Call `init_hash()` once before Hashing; Hashing `h(s)`, `get(l,r)` returns forward hash, `inv(l,r)` reverse hash, and `ispal(l,r)` checks palindrome.

Care: Call `init_hash()` once; `get(l,r)`, `inv(l,r)`, and `ispal(l,r)` are inclusive and 0-indexed.

Complexity: After `init_hash`, build is $O(n)$, push/pop are $O(1)$, and substring hashes are $O(1)$.

Pitfall: N controls maximum precomputed length; increase it before hashing longer strings.

```

using u64 = uint64_t;

const int N = 2e5 + 5;
const u64 md = (1ULL << 61) - 1;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() ^
    (uintptr_t)make_unique<char>().get());
const u64 B = (md >> 2) + rng() % (md >> 1);
inline u64 add(u64 a, u64 b){
    return a + b >= md ? a + b - md : a + b;
}
inline u64 mul(u64 a, u64 b){
    __uint128_t l = (__uint128_t)a * b;
    u64 ret = (l & md) + (l >> 61);
    return ret >= md ? ret - md : ret;
}
u64 binpow(u64 a, u64 b){
    u64 ret = 1;
    while(b){
        if(b & 1) ret = mul(ret, a);

```

```

        a = mul(a, a);
        b >>= 1;
    }
    return ret;
}
const u64 IB = binpow(B, md - 2);
vector<u64> PB(N), PIB(N);
void init_hash(){
    PB[0] = PIB[0] = 1;
    for (int i = 1; i < N; i++){
        PB[i] = mul(PB[i - 1], B);
        PIB[i] = mul(PIB[i - 1], IB);
    }
}
struct Hash : pair<u64, int>{
    Hash(){}
    Hash(const pair<u64, int> &x) : pair(x){}
    Hash(int x) : pair(x, 1){}
    Hash(const string &x){
        for (char c : x) *this = *this + c;
    }
    friend Hash operator+(const Hash &f, const Hash &o){
        return pair{add(mul(f.first, PB[o.second]), o.first), f.second + o.second};
    }
};
struct Hashing{
    vector<Hash> p, rev;
    Hashing(){}
    Hashing(const string &t){
        build(t);
    }
    void build(const string &t){
        clear();
        for (char c : t) push_back(c);
    }
    void push_back(char c){
        if (p.empty()){
            p.push_back(Hash(c));
            rev.push_back(Hash(c));
        }else{
            p.push_back(p.back() + Hash(c));
            rev.push_back(Hash(c) + rev.back());
        }
    }
    void pop_back(){
        assert(!p.empty());
        p.pop_back();
        rev.pop_back();
    }
    Hash get(int l, int r){
        if (l > r) return {};
        if (!l) return p[r];
        return pair{add(p[r].first, md - mul(p[l - 1].first, PB[r - l + 1])), r - l + 1};
    }
    Hash inv(int l, int r){
        if (l > r) return {};
        if (!l) return rev[r];
        u64 val = add(rev[r].first, md - rev[l - 1].first);
        val = mul(val, PIB[l]);
    }
};

```

```

        return pair{val, r - l + 1};
    }
    bool ispal(int l, int r){
        return get(l, r) == inv(l, r);
    }
    int size() const{
        return p.size();
    }
    void clear(){
        p.clear();
        rev.clear();
    }
};

```

Suffixautomaton.cpp

Use: SAM sam; sam.build(s); methods assume lowercase letters and 0-indexed positions.

Apps: distinct_substrings() count; get_kth(K) kth distinct substring; get_kth_all(K) kth substring counting occurrences; match(P) occurrence count; first_occurrence(P) first start or -1; all_occurrences(P) sorted starts; lcs(T) longest common substring length.

Care: build() also builds suffix-link tree data needed by occurrence methods; alphabet is lowercase a-z.

Complexity: Build is $O(n)$; many query methods are $O(\text{length of query})$ plus output size.

Pitfall: get_kth uses lexicographic lowercase order and assumes K is within the number of available substrings.

```

struct SAM{
    static const int A = 26;
    struct node{
        int len, link, firstpos, lstpos;
        bool is_cloned;
        int nxt[A];
        node(){
            memset(nxt, -1, sizeof nxt);
            len = 0;
            link = -1;
            firstpos = 0;
            lstpos = 0;
            is_cloned = false;
        }
    };
    int sz, last;
    vector<node> t;
    vector<int> terminal, inv;
    vector<ll> freq;
    vector<vector<int>> g;
    SAM(){}
    SAM(int n){
        init(n);
    }
    void init(int n){
        t.assign(2 * n + 5, node());
        terminal.assign(2 * n + 5, 0);
        inv.assign(n + 5, 0);
        freq.assign(2 * n + 5, -1);
        g.assign(2 * n + 5, {});
        sz = 1;
        last = 0;
    }
    int get(char c){
        return c - 'a';
    }
};

```

```

}
void extend(char c){
    int cur = sz++;
    t[cur].len = t[last].len + 1;
    t[cur].firstpos = t[cur].lstpos = t[cur].len - 1;
    int p = last;
    while (p != -1 && t[p].nxt[get(c)] == -1){
        t[p].nxt[get(c)] = cur;
        p = t[p].link;
    }
    if (p == -1){
        t[cur].link = 0;
    }else{
        int q = t[p].nxt[get(c)];
        if (t[p].len + 1 == t[q].len){
            t[cur].link = q;
        }else{
            int clone = sz++;
            t[clone] = t[q];
            t[clone].len = t[p].len + 1;
            while (p != -1 && t[p].nxt[get(c)] == q){
                t[p].nxt[get(c)] = clone;
                p = t[p].link;
            }
            t[q].link = t[cur].link = clone;
            t[clone].is_cloned = true;
        }
    }
    last = cur;
}
void build_tree(){
    for (int i = 1; i < sz; i++){
        g[t[i].link].push_back(i);
    }
}
void build(const string &s){
    init(s.size());
    int j = 0;
    for (auto x : s){
        extend(x);
        terminal[last] = j + 1;
        inv[j++] = last;
    }
    build_tree();
}
ll occ(int i){
    if (~freq[i]) return freq[i];
    ll ret = terminal[i] > 0;
    for (auto x : g[i]){
        ret += occ(x);
    }
    return freq[i] = ret;
}
ll distinct_substrings(){
    ll total = 0;
    for (int i = 1; i < sz; i++){
        total += t[i].len - t[t[i].link].len;
    }
    return total;
}

```

```

}
ll calc(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = 1;
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){
            dp[u] += calc(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}
string get_kth(ll K){
    vector<ll> dp(sz, -1);
    calc(0, dp);
    if (K <= 0 || K >= dp[0]) return "";
    string res = "";
    int u = 0;
    while (K > 0){
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] == -1) continue;
            int v = t[u].nxt[c];
            if (dp[v] >= K){
                res += (char)('a' + c);
                u = v;
                K--;
                break;
            }
            K -= dp[v];
        }
    }
    return res;
}
ll calcAll(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = occ(u);
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){
            dp[u] += calcAll(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}
string get_kth_all(ll K){
    vector<ll> dp(sz, -1);
    calcAll(0, dp);
    if (K <= 0 || K > dp[0]) return "";
    string res = "";
    int u = 0;
    while (K > 0){
        bool moved = false;
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] == -1) continue;
            int v = t[u].nxt[c];
            if (dp[v] >= K){
                res += (char)('a' + c);
                u = v;
                K -= occ(v);
                moved = true;
                break;
            }
        }
    }
}

```

```

    }
    K -= dp[v];
}
if (!moved || K <= 0) break;
}
return res;
}
ll match(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    return occ(u);
}
int first_occurrence(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return -1;
        u = t[u].nxt[get(c)];
    }
    return t[u].firstpos - (int)P.length() + 1;
}
void getOcc(int u, int len, vector<int> &res){
    if (terminal[u] > 0){
        res.push_back(terminal[u] - len);
    }
    for (int v : g[u]){
        getOcc(v, len, res);
    }
}
vector<int> all_occurrences(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return {};
        u = t[u].nxt[get(c)];
    }
    vector<int> res;
    getOcc(u, (int)P.length(), res);
    sort(res.begin(), res.end());
    return res;
}
int lcs(const string &T){
    int u = 0, l = 0, max_len = 0;
    for (char c : T){
        int idx = get(c);
        while (u && t[u].nxt[idx] == -1){
            u = t[u].link;
            l = t[u].len;
        }
        if (t[u].nxt[idx] != -1){
            u = t[u].nxt[idx];
            l++;
        }
        max_len = max(max_len, l);
    }
    return max_len;
}
}

```

```
};
```

Trie_strings.cpp

Use: Trie tr; insert(s), erase(s), count(s), count_prefix(s), contains(s), kth_string(k), and autocomplete(prefix,limit).

Apps: count_prefix gives autocomplete count; autocomplete returns distinct suggestions in lexicographic order.

Care: erase(s) returns false if the word is absent; counts keep duplicate insertions, not just boolean presence.

Out: kth_string(k) is 1-indexed and counts duplicates; autocomplete lists each stored word once even if ending count is larger.

Complexity: insert, erase, count, and prefix queries are O(length of string); kth_string is O(answer length * 26).

Complexity: autocomplete costs O(prefix length + printed characters * 26), capped by limit when provided.

Pitfall: The implementation assumes lowercase letters; adapt child size and get mapping for other alphabets.

```

struct Trie{
    vector<array<int, 26>> nxt;
    vector<int> cnt, ending;
    Trie(){
        init();
    }
    void init(){
        nxt.clear();
        cnt.clear();
        ending.clear();
        node();
    }
    int node(){
        nxt.push_back(array<int, 26>{0});
        cnt.push_back(0);
        ending.push_back(0);
        return (int)nxt.size() - 1;
    }
    void insert(const string &s){
        int u = 0;
        cnt[u]++;
        for (char c : s){
            int idx = c - 'a';
            if (!nxt[u][idx]) nxt[u][idx] = node();
            u = nxt[u][idx];
            cnt[u]++;
        }
        ending[u]++;
    }
    bool erase(const string &s){
        if (!count(s)) return false;
        int u = 0;
        cnt[u]--;
        for (char c : s){
            int idx = c - 'a';
            u = nxt[u][idx];
            cnt[u]--;
        }
        ending[u]--;
        return true;
    }
    int go(const string &s){
        int u = 0;
        for (char c : s){
            int idx = c - 'a';

```

```

        if (!nxt[u][idx]) return -1;
        u = nxt[u][idx];
    }
    return u;
}

int count(const string &s){
    int u = go(s);
    return u == -1 ? 0 : ending[u];
}

int count_prefix(const string &s){
    int u = go(s);
    return u == -1 ? 0 : cnt[u];
}

bool contains(const string &s){
    return count(s) > 0;
}

string kth_string(long long k){
    if (k <= 0 || k > cnt[0]) return "";
    string ret;
    int u = 0;
    while (true){
        if (k <= ending[u]) return ret;
        k -= ending[u];
        for (int c = 0; c < 26; c++){
            int v = nxt[u][c];
            if (!v) continue;
            if (k > cnt[v]){
                k -= cnt[v];
            }else{
                ret += char('a' + c);
                u = v;
                break;
            }
        }
    }
}

void collect(int u, string &cur, vector<string> &ret, int limit){
    if (limit >= 0 && (int)ret.size() >= limit) return;
    if (ending[u]) ret.push_back(cur);
    if (limit >= 0 && (int)ret.size() >= limit) return;
    for (int c = 0; c < 26; c++){
        int v = nxt[u][c];
        if (!v) continue;
        cur += char('a' + c);
        collect(v, cur, ret, limit);
        cur.pop_back();
        if (limit >= 0 && (int)ret.size() >= limit) return;
    }
}

vector<string> autocomplete(string prefix, int limit = -1){
    vector<string> ret;
    int u = go(prefix);
    if (u == -1 || limit == 0) return ret;
    collect(u, prefix, ret, limit);
    return ret;
}

void clear(){
    init();
}

```

```
};
```

Z_algorithm.cpp

Use: zfunction(s) returns z[i], the longest prefix of s matching s starting at i.

Care: For pattern matching, run zfunction(pattern + '#' + text) and test z positions at least pattern length.

Complexity: zfunction runs in $O(n)$.

Out: z[0] is left as 0 in this implementation, so do not expect z[0]=n.

```

vector<int> zfunction(const string& s){
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; i++){
        if(i < r) z[i] = min(r - i, z[i - l]);
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if(i + z[i] > r){
            r = i + z[i];
            l = i;
        }
    }
    return z;
}

```

6 DP Optimizations and Misc

divide and conquer dp.cpp

Use: Set n and k , edit `add(i)/rem(i)` so `cur=cost(L,R)`, then `compute()`; `answer()` returns `dp[n]` after k layers.

Care: This maintains cost by moving `[L,R]`; `add/rem` may use current `L/R` exactly like Mo-style pointer movement.

Complexity: Each layer is $O(n \log n)$ divide-and-conquer states plus the amortized moving-window cost changes.

Recurrence: The template computes `dp[mid][layer]=min dp[split-1][layer-1]+cost(split,mid)`.

Pitfall: The optimization is only valid when optimal split points are monotone; otherwise use plain DP or another optimization.

```
const int N = 1e5 + 5;
const ll inf = (1LL << 62);

// dp[i][layer] = minimum cost to cover prefix [1..i] using this many layers.
// Transition shape: dp[mid][it] = min_{split <= mid} dp[split - 1][it ^ 1] + cost(split,
// ↪ mid).
// Valid only when the optimal split point is monotone as mid increases.
int n, k, it = 1, L = 1, R = 0;
ll dp[N][2], cur = 0;

void add(int i){
    (void)i;
    // i has just entered the current interval [L, R].
    // Update cur so it equals cost(L, R).
}

void rem(int i){
    (void)i;
    // i is still inside [L, R] and is about to leave.
    // Update cur so it equals the interval after removing i.
}

void reset_window(){
    L = 1;
    R = 0;
    cur = 0;
}

void move(int l, int r){
    while(R < r) add(++R);
    while(L > l) add(--L);
    while(R > r) rem(R--);
    while(L < l) rem(L++);
}

void compute_layer(){
    vector<array<int, 4>> q;
    q.reserve(2 * n);
    q.push_back({1, n, 1, n}); // {answer range l,r, allowed opt range lx,rx}
    while(!q.empty()){
        auto [l, r, lx, rx] = q.back();
        q.pop_back();
        int mid = (l + r) >> 1, opt = lx;
        dp[mid][it] = inf;
        for(int split = lx; split <= min(rx, mid); split++){
            move(split, mid);
            ll val = dp[split - 1][it ^ 1] + cur;
            if(val < dp[mid][it]){
                dp[mid][it] = val;
                opt = split;
            }
        }
    }
}
```

```
        if(l <= mid - 1) q.push_back({l, mid - 1, lx, opt});
        if(mid + 1 <= r) q.push_back({mid + 1, r, opt, rx});
    }
}

void compute(){
    for(int i = 1; i <= n; i++){
        dp[i][0] = dp[i][1] = inf;
    }
    dp[0][0] = dp[0][1] = 0;
    it = 1;
    for(int layer = 1; layer <= k; layer++){
        for(int i = 1; i <= n; i++) dp[i][it] = inf;
        reset_window();
        compute_layer();
        it ^= 1;
    }
}

ll answer(){
    return dp[n][it ^ 1];
}
```

Custom hashing.cpp

Use: Use `unordered_map<K,V,custom_hash>` for integers or `unordered_map<pair<ll,ll>,V,pair_hash>` for pairs to resist hacks.

Care: Use these hashes only in unordered containers; ordered maps/sets do not need them and cannot use them directly.

When: Use it for adversarial `unordered_map/unordered_set` tests with integer or pair keys.

Pitfall: Custom hash changes bucket distribution, not equality; pair equality still uses normal pair comparison.

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return (size_t) splitmix64(x + FIXED_RANDOM);
    }
};

struct pair_hash {
    size_t operator()(const pair<ll,ll>& p) const noexcept {
        uint64_t a = (uint64_t) p.first;
        uint64_t b = (uint64_t) p.second;
        uint64_t z = a + 0x9e3779b97f4a7c15ULL + (b << 6) + (b >> 2);
        return (size_t) custom_hash::splitmix64(z);
    }
};
```

Josephus.cpp

Use: `josephus_kth_removed(n,k,m)` returns the m -th removed 1-indexed label; survivor/order helpers cover common variants.

Care: $k=1$ is handled directly; `josephus_order(n,k)` uses Fenwick and returns the whole deletion order in 1-indexed labels.

Complexity: `kth_removed/survivor` use jump compression and are fast for large n ; `josephus_order` is $O(n \log n)$.

Out: survivor1 returns the 1-indexed survivor; survivor0_slow is the classic $O(n)$ 0-indexed recurrence.

```
// Returns the m-th removed person, 1-indexed labels, 1 <= m <= n.
// People are removed by counting k alive people starting from person 1.
ll josephus_kth_removed(ll n, ll k, ll m){
    assert(n >= 1 && k >= 1 && 1 <= m && m <= n);
    m = n - m;
    if (k == 1) return n - m;
    ll alive = m;
    while (alive < n){
        ll jump = (ll)((__int128)alive - m + k - 2) / (k - 1));
        if ((__int128)alive + jump > n) jump = n - alive;
        else if (jump == 0) jump = 1;
        alive += jump;
        m = (ll)((m + (__int128)jump * k) % alive);
    }
    return m + 1;
}

ll josephus_survivor1(ll n, ll k){
    return josephus_kth_removed(n, k, n);
}

ll josephus_survivor0_slow(ll n, ll k){
    assert(n >= 1 && k >= 1);
    ll ans = 0;
    for (ll i = 1; i <= n; i++){
        ans = (ans + k) % i;
    }
    return ans;
}
```

```
struct FenwickJosephus{
    int n;
    vector<int> bit;
    FenwickJosephus(){
        FenwickJosephus(int _n){ init(_n); }
    void init(int _n){
        n = _n;
        bit.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) add(i, 1);
    }
    void add(int i, int v){
        for (; i <= n; i += i & -i) bit[i] += v;
    }
    int kth(int k){
        int pos = 0;
        for (int pw = 1 << __lg(n); pw; pw >>= 1){
            int nxt = pos + pw;
            if (nxt <= n && bit[nxt] < k){
                pos = nxt;
                k -= bit[nxt];
            }
        }
        return pos + 1;
    }
};

vector<int> josephus_order(int n, ll k){
    assert(n >= 1 && k >= 1);
    FenwickJosephus fw(n);
    vector<int> order;
```

```
    order.reserve(n);
    ll cur = 0;
    for (int alive = n; alive >= 1; alive--){
        cur = (cur + k - 1) % alive;
        int pos = fw.kth((int)cur + 1);
        order.push_back(pos);
        fw.add(pos, -1);
    }
    return order;
}
```

Permutation cycles.cpp

Use: Helpers use 0-indexed permutations with $p[i]$ =image of i ; cycle order represents $v \rightarrow \text{next}(v)$.

decompose: perm_decompose(p) gets a permutation p and returns its disjoint cycles in orbit order; $O(n)$.

restore: perm_restore(n,cycles) gets cycle lists and returns p with each element mapped to the next element; $O(\text{total length})$.

power: perm_power_cycles(p,k) gets p and $k \geq 0$, then returns the cycle decomposition of p^k ; $O(n)$.

root: perm_root_cycles(p,k,toggle) gets p and $k > 0$, returns cycles of q with $q^k = p$ or empty if impossible.

swap list: perm_transpositions(cycles) returns swaps (c0,ci) for each cycle, count n-number_of_cycles; $O(\text{total length})$.

Root detail: For a length L cycle in p^k , preimages are formed by merging equal-length cycles according to $\text{gcd}(L,k)$.

Toggle: toggle tries one legal 2^* need merge to change transposition parity when the problem asks for a parity target.

Pitfall: The helpers do not validate that p is a permutation; duplicates or out-of-range images break the orbit assumptions.

```
vector<vector<int>> perm_decompose(const vector<int> &p){
    int n = p.size();
    vector<vector<int>> cycles;
    vector<bool> vis(n);
    for (int i = 0; i < n; i++){
        if (vis[i]) continue;
        vector<int> cycle;
        int u = i;
        while (!vis[u]){
            cycle.push_back(u);
            vis[u] = true;
            u = p[u];
        }
        cycles.push_back(cycle);
    }
    return cycles;
}

vector<int> perm_restore(int n, const vector<vector<int>> &cycles){
    vector<int> p(n);
    for (const auto &cycle : cycles){
        int m = cycle.size();
        for (int i = 0; i < m; i++){
            p[cycle[i]] = cycle[(i + 1) % m];
        }
    }
    return p;
}

vector<vector<int>> perm_power_cycles(const vector<int> &p, ll k){
    assert(k >= 0);
    auto cycles = perm_decompose(p);
```



```

    vector<vector<int>>> ans;
    for (const auto &cycle : cycles){
        int len = cycle.size();
        int step = k % len;
        int g = gcd(step, len);
        for (int i = 0; i < g; i++){
            vector<int> cur;
            for (int j = i, cnt = 0; cnt < len / g; cnt++, j = (j + step) % len){
                cur.push_back(cycle[j]);
            }
            ans.push_back(cur);
        }
    }
    return ans;
}

vector<vector<int>>> perm_root_cycles(const vector<int> &p, ll k, bool toggle = false){
    assert(k > 0);
    int n = p.size();
    vector<vector<vector<int>>>> by_len(n + 1);
    for (auto &cycle : perm_decompose(p)){
        by_len[(int)cycle.size()].push_back(cycle);
    }
    vector<vector<int>>> ans;
    for (int len = 1; len <= n; len++){
        if (by_len[len].empty()) continue;
        ll tmp = k, need = 1;
        int g = gcd((ll)len, tmp);
        while (g != 1){
            need *= g;
            tmp /= g;
            g = gcd((ll)len, tmp);
        }
        if ((ll)by_len[len].size() % need != 0) return {};
        int id = 0, block = need;
        if (toggle && tmp % 2 == 0 && (ll)by_len[len].size() >= 2 * need){
            int merged = 2 * block;
            int sz = merged * len;
            vector<int> cycle(sz);
            for (int x = 0; x < merged; x++){
                for (int y = 0; y < len; y++){
                    cycle[(x + 1LL * y * k) % sz] = by_len[len][x][y];
                }
            }
            ans.push_back(cycle);
            id = merged;
            toggle = false;
        }
        for (int i = id; i < (int)by_len[len].size(); i += block){
            int sz = block * len;
            vector<int> cycle(sz);
            for (int x = 0; x < block; x++){
                for (int y = 0; y < len; y++){
                    cycle[(x + 1LL * y * k) % sz] = by_len[len][i + x][y];
                }
            }
            ans.push_back(cycle);
        }
    }
    return ans;
}

```

```

}

vector<pair<int, int>> perm_transpositions(const vector<vector<int>>> &cycles){
    vector<pair<int, int>> ans;
    for (const auto &cycle : cycles){
        for (int i = (int)cycle.size() - 1; i > 0; i--){
            ans.push_back({cycle[0], cycle[i]});
        }
    }
    return ans;
}

```

Lattice count under line

Use: lattice_cnt(a,b,c) counts nonnegative integer pairs (x,y) with $ax+by \leq c$, assuming $a,b > 0$.

Idea: Euclidean descent groups many y-values at once, so it avoids looping over c/b terms.

Care: If $a=0$ or $b=0$ the feasible set is infinite when $c \geq 0$; this snippet asserts instead of returning it.

Complexity: $O(\log \max(a,b))$ arithmetic steps, with the answer stored in ll.

```

ll lattice_cnt(ll a, ll b, ll c) {
    assert(a >= 0 && b >= 0);
    if(c < 0) return 0;
    if(a == 0 || b == 0) {
        assert(0); // infinite solutions when c >= 0
        return -1;
    }
    if(a > b) swap(a, b);
    ll ans = 0;
    while(c >= 0) {
        ll k = b / a;
        ll l = b % a;
        ll f = c / b;
        ll e = c % b / a;
        ll g = c % b % a;
        ans += (f + 1) * (e + 1) + (f + 1) * f / 2 * k;
        c = f * l - a + g;
        b = a;
        a = l;
    }
    return ans;
}

```

Minimum number of squares

Use: min_squares(n) returns the minimum k in $\{0,1,2,3,4\}$ with n as a sum of k nonnegative squares.

Theorems: Two squares uses prime exponents $p \equiv 3 \pmod 4$; three squares uses Legendre $n \neq 4^a(8b+7)$.

Care: This version factors by trial division; swap can_two_squares to Pollard Rho if n is a huge 64-bit composite.

Complexity: $O(\sqrt{n})$ for the two-square test, $O(\log n)$ for the one/three/four-square decisions.

```

bool is_square_ll(ll n) {
    if(n < 0) return false;
    ll r = sqrtl((long double)n);
    while((__int128)(r + 1) * (r + 1) <= n) ++r;
    while((__int128)r * r > n) --r;
    return r * r == n;
}

bool can_two_squares(ll n) {

```

```

    if(n < 0) return false;
    for(ll p = 2; (__int128)p * p <= n; p += (p == 2 ? 1 : 2)) {
        if(n % p) continue;
        int e = 0;
        while(n % p == 0) n /= p, e++;
        if(p % 4 == 3 && (e & 1)) return false;
    }
    return n % 4 != 3;
}
bool can_three_squares(ll n) {
    if(n < 0) return false;
    while(n % 4 == 0) n /= 4;
    return n % 8 != 7;
}
int min_squares(ll n) {
    assert(n >= 0);
    if(n == 0) return 0;
    if(is_square_ll(n)) return 1;
    if(can_two_squares(n)) return 2;
    if(can_three_squares(n)) return 3;
    return 4; // Lagrange four-square theorem
}

```

Pell equation

Use: fundamental_pell(n) returns the smallest positive solution of $x^2 - n \cdot y^2 = 1$, or {1,0} for bad/square n.

Generate: pell_solutions(n,k) multiplies by the fundamental unit and returns the first k positive solutions in order.

Care: Values may be enormous, so the snippet uses a tiny nonnegative BigInt; the period generation itself uses signed 64-bit n.

Complexity: O(period) to get the fundamental solution; generating each next solution is one big-integer multiplication step.

```

struct BigInt {
    static const int base = 1000000000;
    vector<int> d;

    BigInt(ll x = 0) {
        while(x > 0) d.push_back(x % base), x /= base;
    }
    bool zero() const {
        return d.empty();
    }
    void trim() {
        while(!d.empty() && d.back() == 0) d.pop_back();
    }
    string str() const {
        if(d.empty()) return "0";
        string s = to_string(d.back());
        for(int i = (int)d.size() - 2; i >= 0; i--) {
            string cur = to_string(d[i]);
            s += string(9 - cur.size(), '0') + cur;
        }
        return s;
    }
};
ostream& operator << (ostream &os, const BigInt &x) {
    return os << x.str();
}

```

```

}
bool operator == (const BigInt &a, const BigInt &b) {
    return a.d == b.d;
}
bool operator < (const BigInt &a, const BigInt &b) {
    if(a.d.size() != b.d.size()) return a.d.size() < b.d.size();
    for(int i = (int)a.d.size() - 1; i >= 0; i--) {
        if(a.d[i] != b.d[i]) return a.d[i] < b.d[i];
    }
    return false;
}
BigInt operator + (BigInt a, const BigInt &b) {
    int n = max(a.d.size(), b.d.size());
    a.d.resize(n, 0);
    ll carry = 0;
    for(int i = 0; i < n; i++) {
        ll cur = carry + a.d[i] + (i < (int)b.d.size() ? b.d[i] : 0);
        a.d[i] = cur % BigInt::base;
        carry = cur / BigInt::base;
    }
    if(carry) a.d.push_back(carry);
    return a;
}
BigInt operator * (const BigInt &a, ll m) {
    assert(m >= 0);
    if(a.zero() || m == 0) return 0;
    BigInt res;
    res.d.resize(a.d.size());
    ll carry = 0;
    for(int i = 0; i < (int)a.d.size(); i++) {
        __int128 cur = (__int128)a.d[i] * m + carry;
        res.d[i] = cur % BigInt::base;
        carry = cur / BigInt::base;
    }
    while(carry > 0) {
        res.d.push_back(carry % BigInt::base);
        carry /= BigInt::base;
    }
    return res;
}
BigInt operator * (ll m, const BigInt &a) {
    return a * m;
}
BigInt operator * (const BigInt &a, const BigInt &b) {
    if(a.zero() || b.zero()) return 0;
    vector<__int128> tmp(a.d.size() + b.d.size());
    for(int i = 0; i < (int)a.d.size(); i++) {
        for(int j = 0; j < (int)b.d.size(); j++) {
            tmp[i + j] += (__int128)a.d[i] * b.d[j];
        }
    }
    BigInt res;
    res.d.resize(tmp.size());
    __int128 carry = 0;
    for(int i = 0; i < (int)tmp.size(); i++) {
        tmp[i] += carry;
        res.d[i] = tmp[i] % BigInt::base;
        carry = tmp[i] / BigInt::base;
    }
}

```

```

while(carry > 0) {
    res.d.push_back(carry % BigInt::base);
    carry /= BigInt::base;
}
res.trim();
return res;
}

ll isqrt_ll(ll n) {
    ll r = sqrtl((long double)n);
    while((__int128)(r + 1) * (r + 1) <= n) ++r;
    while((__int128)r * r > n) --r;
    return r;
}

vector<ll> continued_fraction_sqrt(ll n) {
    ll a0 = isqrt_ll(n);
    if(a0 * a0 == n) return {a0};
    vector<ll> a = {a0};
    ll m = 0, d = 1, cur = a0;
    do {
        m = d * cur - m;
        d = (n - m * m) / d;
        cur = (a0 + m) / d;
        a.push_back(cur);
    } while(cur != 2 * a0);
    return a; // a[1..] is the periodic part
}

pair<BigInt, BigInt> fundamental_pell(ll n) {
    if(n <= 0) return {1, 0};
    auto cf = continued_fraction_sqrt(n);
    if(cf.size() == 1) return {1, 0};
    int period = (int)cf.size() - 1;
    int terms = (period % 2 == 0 ? period : 2 * period);
    BigInt p2 = 0, p1 = 1, q2 = 1, q1 = 0;
    for(int i = 0; i < terms; i++) {
        ll ai = (i == 0 ? cf[0] : cf[1 + (i - 1) % period]);
        BigInt p = ai * p1 + p2;
        BigInt q = ai * q1 + q2;
        p2 = p1; p1 = p;
        q2 = q1; q1 = q;
    }
    return {p1, q1};
}

vector<pair<BigInt, BigInt>> pell_solutions(ll n, int k) {
    vector<pair<BigInt, BigInt>> ans;
    auto [x1, y1] = fundamental_pell(n);
    if(k <= 0) return ans;
    if(y1.zero()) return {{1, 0}};
    BigInt x = 1, y = 0;
    while(k--) {
        BigInt nx = x * x1 + (y * y1) * n;
        BigInt ny = x * y1 + y * x1;
        ans.push_back({nx, ny});
        x = nx; y = ny;
    }
    return ans;
}

```

Subsequences product at least K

Use: count_product_at_least(a,K) counts subsequences with product $\geq K$ modulo $1e9+7$; $a[i]$ must be positive.

Idea: Count all subsequences, subtract product $\leq K-1$, and switch between front/back DP around $\sqrt{K}$.

Care: Empty subsequence product is treated as 1; if $K \leq 1$, every subsequence is counted.

Complexity: $O(n \cdot \sqrt{K})$ states when K fits normal contest limits; memory is the same order.

```

const int PRODUCT_MOD = 1e9 + 7;

struct ProductSubseqCounter {
    int n, SQ;
    ll lim;
    vector<int> a;
    vector<vector<int>> dp_back, dp_front;

    int addmod(int x, int y) {
        x += y;
        if(x >= PRODUCT_MOD) x -= PRODUCT_MOD;
        return x;
    }

    int small_product(int i, int p) { // product of chosen suffix <= p
        if(p <= 0) return 0;
        if(i == 0) return p >= 1;
        int &ret = dp_back[i][p];
        if(ret != -1) return ret;
        ret = small_product(i - 1, p);
        ret = addmod(ret, small_product(i - 1, p / a[i]));
        return ret;
    }

    int front_product(int i, int p) { // current product p is still < SQ
        if(i == 0) return p <= lim;
        int &ret = dp_front[i][p];
        if(ret != -1) return ret;
        ret = front_product(i - 1, p);
        ll nxt = 1LL * p * a[i];
        if(nxt < SQ) ret = addmod(ret, front_product(i - 1, (int)nxt));
        else ret = addmod(ret, small_product(i - 1, (int)(lim / nxt)));
        return ret;
    }

    int solve(const vector<int> &v, ll K) {
        n = (int)v.size();
        if(K <= 1) {
            int all = 1;
            for(int i = 0; i < n; i++) all = addmod(all, all);
            return all;
        }
        lim = K - 1;
        SQ = sqrtl((long double)lim) + 3;
        a.assign(n + 1, 1);
        for(int i = 1; i <= n; i++) {
            assert(v[i - 1] > 0);
            a[i] = v[i - 1];
        }
        dp_back.assign(n + 1, vector<int>(SQ, -1));
        dp_front.assign(n + 1, vector<int>(SQ, -1));
        int all = 1;
        for(int i = 0; i < n; i++) all = addmod(all, all);
    }
}

```

```

        return (all - front_product(n, 1) + PRODUCT_MOD) % PRODUCT_MOD;
    }
};
int count_product_at_least(const vector<int> &a, ll K) {
    ProductSubseqCounter solver;
    return solver.solve(a, K);
}

```

Blue red hackenbush

Use: `blue_red_hackenbush(n, edges, root)` returns winners {Chef-first, Ksen-first}; color 0 is Chef/black, color 1 is Ksen/white.

Game: Deleting an edge removes the detached subtree, so each rooted edge contributes a dyadic surreal value to its parent.

Care: Edges are 1-indexed tree edges {u,v,color}; the implementation roots the tree iteratively to avoid recursion depth issues.

Complexity: $O(n \log n)$ from small-to-large merging of fractional dyadic bits.

```

struct Surreal {
    ll val = 0, w = 0; // value = val + sum 2^(key+w)
    map<int, int> frac;

    void clear() {
        val = w = 0;
        frac.clear();
    }

    void add_bit(int x) {
        frac[x]++;
        auto it = frac.find(x);
        while(it != frac.end() && it->second != 1) {
            if(it->second > 1) {
                frac[it->first + 1] += it->second >> 1;
                it->second &= 1;
            }
            auto old = it++;
            if(old->second == 0) frac.erase(old);
        }
    }

    void divide_by_power(int x) {
        for(int i = 0; i < x && val != 0; i++) {
            if(val & 1) add_bit(i - w);
            val >>= 1;
        }
        w -= x;
    }

    void operator += (const Surreal &other) {
        val += other.val;
        for(auto [bit, cnt] : other.frac) {
            if(cnt) add_bit(bit + other.w - w);
        }
        while(!frac.empty() && frac.rbegin()->first + w >= 0) {
            int e = frac.rbegin()->first + w;
            if(frac.rbegin()->second) val += (1LL << e);
            frac.erase(prev(frac.end()));
        }
    }

    void take_black_edge() {
        ll p = max(1LL, 1 - val);
    }
}

```

```

        if(frac.empty() && p + val == 1) ++p;
        val += p;
        divide_by_power((int)p - 1);
    }

    void take_white_edge() {
        ll p = max(1LL, 1 + val);
        if(val - p == -1) ++p;
        val -= p;
        divide_by_power((int)p - 1);
    }

    int sign() const {
        if(val != 0) return val > 0 ? 1 : -1;
        return frac.empty() ? 0 : 1;
    }
};

pair<string, string> blue_red_hackenbush(int n, vector<array<int, 3>> edges, int root = -1) {
    vector<vector<pair<int, int>>> g(n + 1);
    for(auto [u, v, c] : edges) {
        g[u].push_back({v, c});
        g[v].push_back({u, c});
    }
    vector<int> par(n + 1, 0), pcol(n + 1, 0), id(n + 1);
    vector<Surreal> a(n + 1);
    iota(id.begin(), id.end(), 0);
    vector<int> order = {root};
    par[root] = -1;
    for(int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for(auto [v, c] : g[u]) if(v != par[u]) {
            par[v] = u;
            pcol[v] = c;
            order.push_back(v);
        }
    }
    for(int i = (int)order.size() - 1; i > 0; i--) {
        int u = order[i], p = par[u];
        if(pcol[u] == 0) a[id[u]].take_black_edge();
        else a[id[u]].take_white_edge();
        if(a[id[u]].frac.size() > a[id[p]].frac.size()) swap(id[u], id[p]);
        a[id[p]] += a[id[u]];
        a[id[u]].clear();
    }
    int s = a[id[root]].sign();
    if(s > 0) return {"Chef", "Chef"};
    if(s < 0) return {"Ksen", "Ksen"};
    return {"Ksen", "Chef"};
}

```